

Mémoire de fin d'études

l'obtention du diplôme d'ingénieur en informatique

Option: Systèmes d'Information et Technologies (SIT)

Prédiction des Demandes de Livraison via Apprentissage Profond et Détection du Concept Drift

Réalisée par:

Mr. Raid BOURZAM (SIT)

Mr. Zakaria BELHADJ (SIT)

Encadrée par:

Mme. Leila HAMDAD (ESI)

Mme. Asma CHADER (ESI)

Mme. Yasmine RIHANE
(OURQUILANE)

Dédicaces

À MES PARENTS, À MES FRÈRES ET SŒURS, À
MES ENSEIGNANTS, À MES AMIS, À MOI, MERCI.

Remerciements

Avant tout, nous commençons par remercier Allah, le Miséricordieux, de nous avoir donné la force et le courage pour mener à bien ce travail à son terme.

Nous tenons à remercier nos encadrantes, Dr Hamdad, Dr Chader et Mme Rihane, d'être toujours disponibles pour nous guider et orienter avec leurs précieux conseils et assistance tout au long de ce projet.

Nous exprimons notre profonde gratitude et nos plus sincères reconnaissances à nos familles et nos amies pour leur soutien inconditionnel, tout au long de nos années d'études. Leur présence à nos côtés a été notre source de motivation et de réconfort.

Aux membres de jury, nous adressons nos sincères remerciements pour les efforts qu'ils font en prenant le temps de lire et d'évaluer ce travail.

À toute l'équipe d'Ourquilane, nous adressons nos sincères remerciements pour votre soutien constant, votre confiance, ainsi que pour les opportunités d'apprentissage et de collaboration que vous nous avez offertes tout au long de ce projet.

Enfin, nous adressons nos remerciements à tous ceux qui ont participé de près ou de loin à la réalisation de ce modeste travail.

Résumé

Ce projet aborde le défi de la prévision des volumes de livraison pour *Yalidine EL DJAZAIR SERVICES*, une entreprise de logistique confrontée à des difficultés dans l'allocation des ressources, notamment lors des pics d'activité saisonniers. Face aux limitations d'un système de prévision externe coûteux et peu flexible, l'objectif est de développer une solution interne basée sur l'apprentissage profond capable d'anticiper avec précision les demandes par station. Bien que l'approche privilégie les techniques d'apprentissage profond, l'étude adopte une démarche comparative multi-modèles reconnaissant que certaines stations peuvent bénéficier d'approches plus simples selon leurs spécificités locales. Ainsi, la solution intègre des modèles de réseaux de neurones (*MLP*, *MLP-SUL*) et de L'apprentissage profond (*LSTM*) comme approche principale, tout en incluant une technique de Machine Learning traditionnel (*XGBoost*) et une approche de statistique (*ARIMA*) pour s'adapter à la diversité des 182 stations du réseau. Le *MLP-SUL* constitue une variante du perceptron multicouche adaptée spécifiquement à notre contexte, basée sur une approche issue de la littérature de recherche. La méthodologie *CRISP-DM* guide cette étude, depuis la compréhension des besoins métier jusqu'à la mise en œuvre d'un système intégrant la détection de concept drift pour s'adapter aux évolutions de la demande. L'analyse des processus existants a révélé une sous-estimation chronique des besoins en livreurs et des problématiques de conformité réglementaire. La solution proposée vise à automatiser les prévisions, optimiser l'affectation des ressources, assurer la conformité avec la loi algérienne sur la protection des données 18-07, et réduire les coûts opérationnels, en s'appuyant sur un algorithme de sélection automatique qui identifie le modèle optimal pour chaque station spécifique.

Mots-clés : Prévision de la demande, séries temporelles, apprentissage profond, concept drift, logistique.

ملخص

يتناول هذا المشروع تحدي التنبؤ بأحجام التسليم لشركة Validine الجزائرية للخدمات، وهي شركة لوجستية تواجه صعوبات في تخصيص الموارد، خاصة خلال ذروة النشاط الموسمي. في مواجهة قيود نظام التنبؤ الخارجي المكلف وقليل المرونة، الهدف هو تطوير حل داخلي قائم على التعلم العميق قادر على توقع الطلبات بدقة لكل محطة. رغم أن النهج يفضل تقنيات التعلم العميق، تتبنى الدراسة منهجية مقارنة متعددة النماذج تقرر بأن محطات معينة قد تستفيد من مناهج أبسط وفقاً لخصائصها المحلية. وبالتالي، يدمج الحل نماذج الشبكات العصبية (MLP، MLP-SUL) والتعلم العميق (LSTM) كنهج رئيسي، مع تضمين تقنية التعلم الآلي التقليدي (XGBoost) ونهج إحصائي (ARIMA) للتكيف مع تنوع محطات الشبكة البالغة 182 محطة. يشكل MLP-SUL متغيراً من البرسبترون متعدد الطبقات مكيف خصيصاً لسياقنا، مستند إلى نهج مستمد من الأدبيات البحثية. تُرشد منهجية CRISP-DM هذه الدراسة، من فهم احتياجات العمل إلى تصميم نظام يدمج كشف انحراف المفهوم للتكيف مع تطور الطلب. كشف تحليل العمليات الموجودة عن نقص مزمن في تقدير احتياجات عمال التوصيل ومشاكل الامتثال التنظيمي. يهدف الحل المقترح إلى أتمتة التنبؤات، وتحسين تخصيص الموارد، وضمان الامتثال للقانون الجزائري لحماية البيانات 07-18، وتقليل التكاليف التشغيلية، بالاعتماد على خوارزمية اختيار تلقائية تحدد النموذج الأمثل لكل محطة محددة.

الكلمات المفتاحية: التنبؤ بالطلب، السلاسل الزمنية، التعلم العميق، انحراف المفهوم، اللوجستيات.

Liste des Sigles et Acronymes

ADWIN : Adaptive Windowing
ANN : Artificial Neural Networks (Réseaux de Neurones Artificiels)
API : Application Programming Interface
ARIMA : Autoregressive Integrated Moving Average (Moyenne Mobile Intégrée Autorégressive)
B2B : Business-to-Business
B2C : Business-to-Customer
BPMN : Business Process Model and Notation
CVFDT : Concept-adapting Very Fast Decision Tree
DDM : Drift Detection Method
DKL : Divergence de Kullback-Leibler
EDDM : Early Drift Detection Method
GRU : Gated Recurrent Unit
HDDM : Hoeffding Drift Detection Method
ISO : International Organization for Standardization
ITA : Information-Theoretic Approach
KSWIN : Kolmogorov-Smirnov Windowing
LLDD : Learning Link-based Drift Detection
LSTM : Long Short-Term Memory (Réseaux de neurones à mémoire longue)
MAE : Erreur Absolue Moyenne (Mean Absolute Error)
MAPE : Erreur Absolue Moyenne en Pourcentage (Mean Absolute Percentage Error)
ML : Machine Learning (Apprentissage Automatique)
MLP : Perceptron Multicouche (Multilayer Perceptron)
MPE : Erreur en Pourcentage Moyenne (Mean Percentage Error)
MSE : Erreur Quadratique Moyenne (Mean Squared Error)
Padd : Parallel Activations Drift Detector
RMSE : Racine de l'Erreur Quadratique Moyenne (Root Mean Squared Error)
RNN : Réseaux de Neurones Récurrents (Recurrent Neural Networks)
SIT : Systèmes d'Information et Technologi
SMAPE : Erreur Absolue Moyenne Symétrique en Pourcentage
SVM : Support Vector Machine
SVR : Support Vector Regression
VFDT : Very Fast Decision Tree

Table des Matières

Introduction Générale.....	1
Contexte du projet.....	3
PARTIE I	4
Chapitre 01 Généralités sur les séries temporelles et leurs méthodes de prévision.....	5
1.1. Introduction.....	5
1.2. Les séries temporelles.....	5
1.2.1. Caractéristiques d'une série temporelle.....	6
1.2.2. Décomposition d'une série temporelle.....	7
1.3. Vue globale sur les Approches principales de prédiction.....	8
1.3.1. Approche Qualitative.....	8
1.3.2. Approche Quantitative.....	9
1.4. Métriques d'évaluation.....	21
1.4.1. Erreurs dépendantes de l'échelle.....	21
1.4.2. Erreurs en pourcentage.....	22
1.5. Conclusion.....	23
Chapitre 02 Détection et Caractérisation du Concept Drift.....	24
2.1. Introduction.....	24
2.2. Définition du Concept Drift.....	24
2.3. Sources du concept drift.....	24
2.3.1. Le changement dans le profil des données.....	24
2.3.2. Changement dans la relation prédictive.....	25
2.3.3. Combinaison des deux (Dérive Hybride).....	25
2.4. Types de Concept Drift.....	26
2.5. Méthodes de Détection du Concept Drift.....	27
2.5.1. Cas supervisé.....	29
2.5.2. Cas Non-Supervisé.....	32
2.6. Adaptation aux drifts (changements).....	32
2.6.1. Entraînement de nouveaux modèles pour les drifts globales.....	32
2.6.2. Utilisation d'ensembles de modèles pour les drifts récurrents.....	33
2.6.3. Ajustement des modèles existants pour les drifts locales.....	34
2.6.4. Conclusion.....	34
PARTIE I I	35
Chapitre 03 Étude de l'existant et analyse des besoins.....	36
3.1. Introduction.....	36
3.2. Présentation du domaine et de l'organisme d'accueil.....	36
3.2.1. Présentation générale de l'entreprise.....	36
3.3. Analyse des processus existants.....	38
3.3.1. Composantes clés de la chaîne logistique Yalidine.....	38

3.3.2. Cheminement d'un Colis.....	39
3.3.3. Flux Logistiques et Processus de Prévision.....	40
3.3.4. Cycle de vie d'un colis chez Yalidine.....	41
3.4. Problématique.....	42
3.5. Objectifs finaux du système cible.....	42
3.6. Exigences Techniques et Ressources.....	43
3.7. Parties prenantes du projet.....	43
3.8. Gestion des prévisions existante.....	44
3.9. Diagnostic opérationnel et impact sur le processus de livraison.....	49
3.10. Recensement des Besoins Client.....	50
3.11. Conclusion.....	50
Chapitre 04 Conception.....	51
4.1. Introduction.....	51
4.2. Méthodologie de travail.....	51
4.2.1. Compréhension du métier.....	52
4.2.2. Compréhension des données.....	52
4.2.3. Préparation des données.....	52
4.2.4. Modélisation.....	52
4.2.5. Évaluation.....	52
4.2.6. Déploiement.....	52
4.3. Architecture Globale.....	53
4.4. Architecture Détaillée.....	55
4.4.1. Module de collecte et traitement des données.....	55
4.4.2. Module d'Apprentissage et Entraînement.....	61
4.4.3. Module de Prédiction et Visualisation.....	66
4.4.4. Module de détection du Concept drift.....	66
4.5. Conclusion.....	69
Chapitre 05 Réalisation.....	70
5.1. Introduction.....	70
5.2. Choix technologiques.....	70
5.2.1. Langages et frameworks utilisés.....	70
5.2.2. Outils de gestion des données.....	74
5.2.3. Prétraitement et transformation des données.....	75
5.3. Architecture technique de la solution.....	76
5.4. Implémentation des modèles de prédiction.....	77
5.4.1. Méthode d'entraînement.....	77
5.4.2. Optimisation des hyperparamètres.....	78
5.4.3. Stockage des Résultats.....	78
5.4.4. Détection du concept drift.....	79
5.5. Conclusion.....	80
Chapitre 06 Tests et Résultats.....	81
6.1. Introduction.....	81

6.2. Apprentissage des modèles.....	81
6.2.1. Performance du modèle final.....	81
6.2.2. Algorithmes d'optimisation.....	82
6.2.3. Feature Engineering.....	83
6.3. Tests sur les prédictions.....	84
6.3.1. Court terme.....	85
6.3.2. Long terme.....	86
6.3.3. Meilleur modèle vs Prophet par station.....	86
6.3.4. Performances entre développement et production.....	87
6.3.5. Statistiques sur les modèles.....	89
6.4. Détection du Concept Drift.....	92
6.4.1. DDM.....	92
6.4.2. ADWIN.....	93
6.5. Conclusion.....	93
Conclusion Generale.....	94
Bibliographie.....	96
Annexes.....	100
7.1. Gestion de projet.....	100
7.1.1. Planning.....	100
7.1.2. Contraintes liées au projet.....	100

Liste des Figures

Figure 1.1	Exemple des deux types des séries temporelles.....	3
Figure 1.2	Caractéristiques d'une série temporelle.....	4
Figure 1.3	Classification des méthodes de prédiction de la demande.....	6
Figure.1.4	Représentation schématique d'un modèle ARIMA.....	9
Figure.1.5	Représentation schématique d'un modèle XGBoost.....	11
Figure.1.6	Structure d'un neurone artificiel (Laqrichi, 2015 p.36).....	12
Figure.1.7	Un exemple de perceptron multicouche (MLP).....	13
Figure.1.8	Visualisation du modèle de MLP-SUL (Kim & Hwang, 2023).....	14
Figure 1.9	Architecture simplifiée d'un réseau de neurones MLP-SUL.....	15
Figure 1.10	Architecture RNN (Colah 2015).....	16
Figure 1.11	Cellule LSTM (Colah 2015).....	17
Figure 1.12	Comparaison de l'algorithme SGD avec et sans momentum (Du, 2019).....	19
Figure 2.1	Première source du concept drift (Lu et al. 2020).....	25
Figure 2.2	Deuxième source du concept drift (Lu et al.2020).....	25
Figure 2.3	Troisième source du concept drift (Lu et al., 2020).....	26
Figure 2.4	Types de concept drift (Lu et al., 2020).....	27
Figure 2.5	Architecture globale d'un détecteur de concept drift (Lu et al., 2020).....	29
Figure 2.6	Entraînement de nouveaux modèles pour les drifts globales.....	34
Figure 2.7	Utilisation d'ensembles de modèles pour les drifts récurrents.....	35
Figure 2.8	Ajustement des modèles existants pour les drifts locales.....	36
Figure 3.1	Logo de l'organisme d'accueil Ourquillane.....	39
Figure 3.2	Organigramme de l'organisme d'accueil Ourquillane.....	39
Figure 3.3	Logo de l'organisme client Yalidine.....	40
Figure 3.4	Exemple de suivi de colis en temps réel sur le site web de Yalidine.....	40
Figure 3.5	Diagramme BPMN du processus de livraison actuel de Yalidine.....	43
Figure 3.6	Cycle de vie d'un colis chez Yalidine.....	44
Figure 3.7	Logo de Tableau Cloud.....	45
Figure 3.8	Diagramme BPMN du processus actuel de prévision du nombre de colis.....	45
Figure 3.9	Exemple d'importation des données de livraison sur Tableau Cloud.....	46
Figure 3.10	Exemple de visualisation des données sur Tableau Cloud.....	47
Figure 3.11	Exemple de prévision pour l'agence d'El Khroub sur Tableau cloud.....	47
Figure 3.12	Exemple de prévision pour l'agence Ali Mendjeli sur Tableau cloud.....	48
Figure 3.13	Options de génération de la prévision avec Tableau cloud.....	49
Figure 3.14	Modèle de prévision et métriques de qualité.....	49
Figure 4.1	Méthodologie CRISP-DM.....	54
Figure 4.2	Architecture globale du système de prévision des volumes de livraison.....	57
Figure 4.3	Exemple de représentation des données brutes depuis l'API sous format JSON.....	58
Figure 4.4	Exemple de structure des données agrégées par jour et par station sous format CSV.....	59
Figure 4.5	Moyenne quotidienne des colis de Yalidine traités par wilaya et par jour.....	60
Figure 4.6	Moyenne des livraisons mensuelles par commune de départ.....	61

Figure 4.7 Pipeline de prétraitement des données.....	62
Figure 4.8 Données en format csv après Enrichissement.....	63
Figure 4.9 Événements - Année 2023.....	64
Figure 4.10 Algorithme de sélection automatique de modèle par station.....	65
Figure 4.11 Vue globale de l'algorithme du sélection des modèles.....	66
Figure 4.12 Illustration du principe de Random Search pour l'optimisation des hyperparamètres..	67
Figure 4.13 Module de Prédiction et Visualisation.....	69
Figure 4.14 Module de détection du Concept drift.....	70
Figure 4.15 Algorithme de DDM.....	71
Figure 4.16 Algorithme de ADWIN.....	72
Figure 5.1 Logo de Python.....	73
Figure 5.2 PyTorch.....	74
Figure 5.3 NumPy.....	74
Figure 5.4 Matplotlib.....	74
Figure 5.5 Scikit-learn.....	75
Figure 5.6 River.....	75
Figure 5.7 VSCode.....	75
Figure 5.8 GitHub.....	75
Figure 5.9 draw.io.....	75
Figure 5.10 Google Drive.....	76
Figure 5.11 Zotero.....	76
Figure 5.12 LaTeX.....	76
Figure 5.13 Pandas.....	77
Figure 5.14 Architecture Technique (Applicative) de notre solution.....	79
Figure 6.1 Perte d'entraînement du MLP-SUL – Corner El Harrach.....	84
Figure 6.2 Comparaison des optimiseurs (perte de validation du MLP-SUL) – Corner El Harrach..	86
Figure 6.3 Comparaison des prédictions avec et sans feature engineering.....	87
Figure 6.4 Comparaison des prédictions des modèles sur 10 jours – Station Corner El Harrach.....	88
Figure 6.5 Prédictions des modèles sur 30 jours – Corner El Harrach.....	89
Figure 6.6 Prédictions MLP-SUL vs Prophet – Corner El Harrach.....	90
Figure 6.7 Performance réelle en développement – Corner El Harrach.....	91
Figure 6.8 Performance réelle en production – Corner El Harrach.....	91
Figure 6.9 Comparaison du MSE des modèles – Corner El Harrach.....	92
Figure 6.10 Répartition des modèles sélectionnés comme meilleurs prédicteurs.....	93
Figure 6.11 Sélections des modèles (développement et production).....	93
Figure 6.12 Taux de correspondance des meilleurs modèles (développement vs production).....	94
Figure 6.13 Scores pondérés MMR par modèle.....	95
Figure 7.1 Planning prévisionnel du projet.....	104

Liste des Tables

Tableau 3.1 Statuts des colis et leurs descriptions.....	44
Tableau 3.2 Rôles des acteurs et impacts attendus du projet.....	53
Tableau 5.1 Liste des principales bibliothèques Python utilisées et leurs descriptions.....	75
Tableau 5.2 Outils logiciels utilisés pour le développement et la gestion du projet.....	76
Tableau 5.3 Spécifications matérielles des machines utilisées lors du développement.....	77
Tableau 6.1 Résultats de performance des modèles pour un échantillon de stations.....	85
Tableau 6.2 Statut de dérive détecté par DDM sur un échantillon de stations.....	96

Introduction Générale

Dans un contexte économique en constante évolution, la logistique s'est affirmée comme un enjeu stratégique majeur, particulièrement dans le secteur florissant du e-commerce. En Algérie, la croissance exponentielle des achats en ligne impose aux entreprises de repenser leurs stratégies afin de garantir un service de livraison efficace et optimisé. Au cœur de cette dynamique, YALIDINE EL DJAZAIR SERVICES, leader national de la livraison express, est confrontée au défi d'adapter sa demande aux fluctuations imprévisibles.

L'anticipation précise du volume de colis est un levier essentiel pour optimiser l'allocation des ressources humaines et matérielles, tout en améliorant la qualité du service client. Une prévision imprécise engendre des conséquences économiques majeures : une surestimation mène à des coûts superflus et un gaspillage de ressources, tandis qu'une sous-estimation provoque une incapacité à absorber les pics de demande, notamment lors d'événements comme le Ramadan et l'Aïd, où les volumes de livraison augmentent drastiquement.

La prévision des demandes de livraison est un pilier fondamental de la gestion logistique moderne. Elle consiste à anticiper précisément le volume de colis à livrer en analysant les données historiques, les tendances saisonnières et les facteurs économiques. Cette discipline s'appuie sur l'analyse des séries chronologiques, c'est-à-dire des observations collectées à intervalles réguliers et ordonnées chronologiquement, permettant de modéliser les variations d'une variable en fonction du temps.

Le défi principal réside dans la nature dynamique des données de livraison, marquée par des variations saisonnières, des tendances émergentes et des événements exceptionnels. Cette complexité est amplifiée par le Concept Drift, un phénomène d'évolution des relations statistiques entre les variables au fil du temps, qui rend les modèles prédictifs obsolètes et exige une adaptation constante. Ce drift affecte la performance des modèles et constitue un défi majeur pour maintenir la prévisibilité de nos systèmes.

Face à cette problématique complexe, l'objectif principal de ce projet est de concevoir et développer un système de prévision intelligent et adaptatif, intégrant des techniques avancées d'apprentissage automatique et d'apprentissage profond, afin d'anticiper avec une précision optimale les demandes de livraison pour YALIDINE EL DJAZAIR SERVICES. Notre solution propose une approche comparative, évaluant diverses familles d'algorithmes : des méthodes statistiques aux techniques d'apprentissage automatique et de deep learning, en se concentrant sur la détection et la gestion du Concept Drift pour maintenir la performance prédictive.

Notre méthodologie s'appuie sur une approche rigoureuse inspirée du modèle CRISP-DM, comprenant une analyse approfondie des besoins métier, une collecte et un

prétraitement minutieux des données, ainsi que le développement, l'optimisation et l'évaluation comparative de modèles prédictifs. Cette démarche scientifique nous permet d'identifier les algorithmes les plus performants pour chaque contexte spécifique, tout en intégrant des mécanismes de détection de drift pour assurer la robustesse et la pérennité de la solution.

Ce manuscrit est structuré en plusieurs parties pour présenter de manière exhaustive notre démarche et nos contributions. Après cette introduction générale, la première partie, dédiée à l'état de l'art, explorera en profondeur deux domaines cruciaux : d'une part, les fondements de la prévision des séries chronologiques un choix justifié par la nature temporelle de nos données, telles que les volumes de colis à livrer, qui évoluent au fil du temps. Il est donc indispensable de maîtriser les méthodes spécifiques à ce type de données pour garantir la fiabilité des prévisions, et d'autre part, les mécanismes de détection et d'adaptation au Concept Drift. La section suivante se concentrera sur l'étude de l'existant et l'analyse des besoins, incluant un recensement détaillé des attentes de YALIDINE et une compréhension fine de leurs processus logistiques actuels. Les chapitres suivants détaillent la conception de notre solution, depuis l'architecture globale jusqu'aux choix des algorithmes, et enfin la réalisation et les tests de l'implémentation, avec une analyse approfondie des performances obtenues. Le rapport se conclura par un résumé des réalisations et une discussion sur les perspectives futures. L'objectif ultime est de fournir un outil décisionnel fiable et adaptatif, capable de transformer les données historiques en intelligence prédictive actionnable pour améliorer l'efficacité opérationnelle et la satisfaction client, tout en s'adaptant continuellement aux évolutions du marché et aux changements comportementaux des consommateurs algériens.

Contexte du projet

Le secteur de la livraison est au cœur d'une transformation profonde et rapide, exacerbée par l'explosion du e-commerce et les attentes client toujours plus élevées en matière de rapidité et de fiabilité. Dans ce contexte, la complexité logistique n'a cessé de croître, rendant impérative une maîtrise rigoureuse des coûts et une optimisation sans précédent des opérations. Face à ces défis, l'adoption d'une logistique moderne – caractérisée par l'intégration technologique (IA, Big Data), l'optimisation des processus, la transparence globale et une adaptabilité accrue – est devenue non seulement un avantage compétitif, mais une nécessité stratégique pour les entreprises du secteur.

C'est dans cette optique que Yalidine EL DJAZAIR SERVICES, acteur majeur de la livraison en Algérie, a initié une démarche de modernisation de ses opérations. Leur objectif principal est de rationaliser et d'améliorer l'efficacité de leurs processus, avec un focus crucial sur la gestion du dernier kilomètre et l'organisation de leurs livreurs "Dispatch App". Pour concrétiser cette vision ambitieuse, Yalidine a fait appel à un partenaire de confiance, la startup Ourquilane.

Dans le cadre de ce projet, notre mission est de développer un système de prévision avancé capable de prédire le nombre de colis entrants pour chaque station de Yalidine. À cet effet, et dans le strict respect de la Loi 18-07 relative à la protection des personnes physiques dans le traitement des données à caractère personnel, ainsi que de la certification ISO 27001 en matière de sécurité de l'information, Yalidine nous fournira un API dédié aux données de colis uniquement, excluant toute donnée personnelle.

Il est fondamental de souligner que ces données de colis représentent des séries temporelles. Par conséquent, pour une conception et une implémentation robustes de ce système de prévision, il sera essentiel de mobiliser des connaissances approfondies en analyse des séries temporelles et en maîtrise de leurs méthodes de prévision spécifiques. Ce système servira de fondation pour des calculs précis, notamment pour déterminer le nombre optimal de colis par livreur, évaluer les performances individuelles de chaque livreur et ainsi optimiser l'ensemble de la chaîne de distribution.

PARTIE I

Etat de l'art

Chapitre 01

Généralités sur les séries temporelles et leurs méthodes de prévision

1.1. Introduction

La prévision du nombre de colis est un cas typique de série temporelle, où l'analyse des données se fait dans le temps pour identifier des motifs et anticiper l'avenir. Ce chapitre explorera les caractéristiques fondamentales des séries temporelles, ainsi que les méthodes de prévision les plus pertinentes issues de la littérature pour une anticipation efficace de la demande logistique.

1.2. Les séries temporelles

Les séries chronologiques représentent simplement des suites de données recueillies dans l'ordre du temps, à intervalles réguliers. Elles servent à observer comment un phénomène évolue, que ce soit heure par heure, jour après jour ou mois après mois. Ce qui fait leur particularité, c'est que chaque valeur dépend souvent de celles qui l'ont précédée, ce qui permet de repérer des tendances, des cycles ou des changements soudains. Travailler avec des séries chronologiques, c'est donc chercher à comprendre le passé pour mieux anticiper ce qui peut arriver ensuite, que ce soit pour gérer des stocks, prévoir une demande ou suivre l'évolution d'un indicateur important.

Deux types principaux sont rencontrés comme illustré dans la *Figure 1.1*:

- **Série temporelle univariée:** Il s'agit d'une suite de mesures d'une seule caractéristique, relevées à intervalles réguliers.
- **Série temporelle multivariée:** Ce type de série rassemble plusieurs indicateurs suivis simultanément dans le temps. Chaque variable peut être influencée par son propre passé, mais aussi par l'évolution des autres variables. Par exemple, on peut enregistrer chaque heure la consommation d'électricité, la température et l'humidité dans un bâtiment.

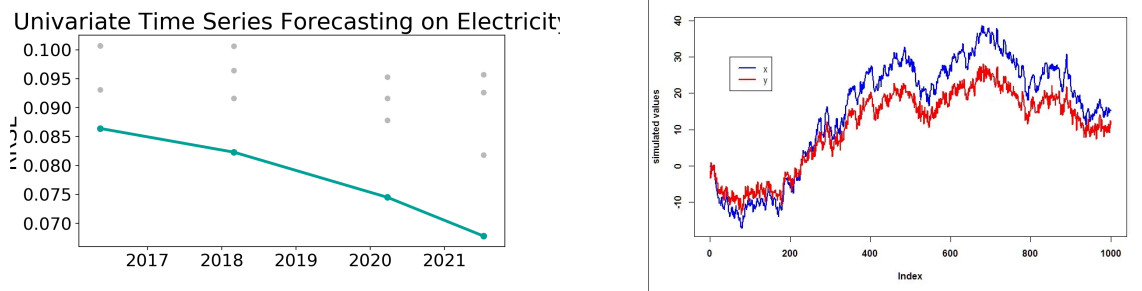


Figure 1.1 Exemple des deux types des séries temporelles

1.2.1. Caractéristiques d'une série temporelle

La performance d'un modèle de prédiction est fortement influencée par les caractéristiques des données. Leur identification est cruciale pour le choix des approches de modélisation. Les caractéristiques majeures illustrées dans la **Figure 1.2** incluent :

- **Stationnarité:** Une série temporelle est dite stationnaire lorsque ses propriétés statistiques fondamentales, telles que sa moyenne et sa variance ne varient pas au fil du temps. De nombreux modèles statistiques classiques requièrent la stationnarité, et bien que les modèles de Deep Learning soient plus flexibles, la transformation d'une série non stationnaire en série stationnaire (*par différenciation, par exemple*) peut parfois améliorer leur performance.
- **Saisonnalité:** La saisonnalité se réfère à des variations cycliques prévisibles qui se répètent à des intervalles de temps fixes (*par exemple, annuels, mensuels, hebdomadaires*). Un exemple typique est l'augmentation des ventes de glaces en été. La prise en compte de la saisonnalité est cruciale pour de nombreuses tâches de prévision.
- **Cycle:** Hausses et baisses dans les données qui n'ont pas une fréquence fixe ou une périodicité régulière.
- **Tendance:** La tendance décrit l'évolution à long terme, croissante ou décroissante, de la série. Elle représente la direction générale vers laquelle la série évolue sur une période étendue, indépendamment des fluctuations à plus court terme comme la saisonnalité.
- **Bruit:** Le bruit représente les fluctuations aléatoires et inexpliquées qui subsistent après avoir pris en compte la tendance, la saisonnalité et les cycles. Idéalement, dans un bon modèle, ce résidu devrait s'apparenter à un bruit blanc, c'est-à-dire une séquence d'erreurs aléatoires sans structure identifiable.

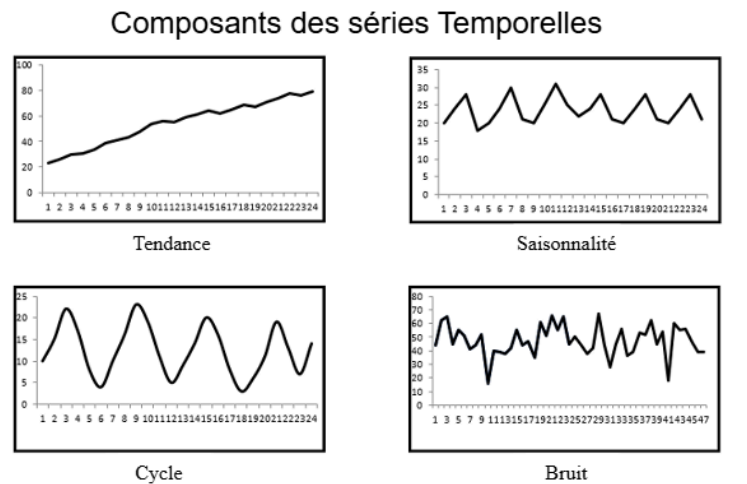


Figure 1.2 Caractéristiques d'une série temporelle

1.2.2. Décomposition d'une série temporelle

Décomposer une série chronologique consiste à la diviser en plusieurs éléments de base qui expliquent son comportement. On distingue principalement deux façons de représenter cette décomposition :

1.2.2.1. Combinaison additive

Lorsque l'ampleur des fluctuations (saisonniers ou autour de la tendance) ne varie pas avec le niveau de la série. La série s'écrit :

$$Y_t = T_t + S_t + C_T + I_T \quad (1.1)$$

1.2.2.2. Combinaison multiplicative

Dans ce cas, les variations dépendent du niveau de la série. Plus la valeur globale est élevée, plus les fluctuations saisonnières ou autour de la tendance sont marquées.

$$Y_t = T_t \cdot S_t \cdot C_t \cdot I_t \quad (1.2)$$

Où Y_t représente les données, S_t désigne le composant de saisonnalité, T_t désigne le composant de tendance, C_t désigne le composant de cycle, I_t désigne le composant de bruit, pour une période t .

Dans les deux cas, on cherche à isoler plusieurs composantes : la tendance générale, les effets saisonniers, les cycles irréguliers, et enfin le bruit, c'est-à-dire les variations imprévisibles. Cette démarche permet de mieux comprendre la structure de la série et d'améliorer la qualité des prévisions.

1.3. Vue globale sur les Approches principales de prédiction

Cette partie présente les différentes techniques de prévision de la demande, qui se répartissent en deux grandes catégories : les approches qualitatives et quantitatives comme illustré dans la **Figure 1.3**. Les méthodes qualitatives sont particulièrement utiles lorsque les données chiffrées du passé sont rares, inaccessibles ou peu pertinentes. Bien que ce projet soit principalement axé sur les méthodes quantitatives en vue de l'automatisation, il est essentiel d'aborder également les approches qualitatives. En effet, l'expertise humaine est souvent irremplaçable pour ajuster, valider ou enrichir les prévisions, notamment pour faire face à des situations exceptionnelles ou imprévues. (Kilger & Wagner, 2015).

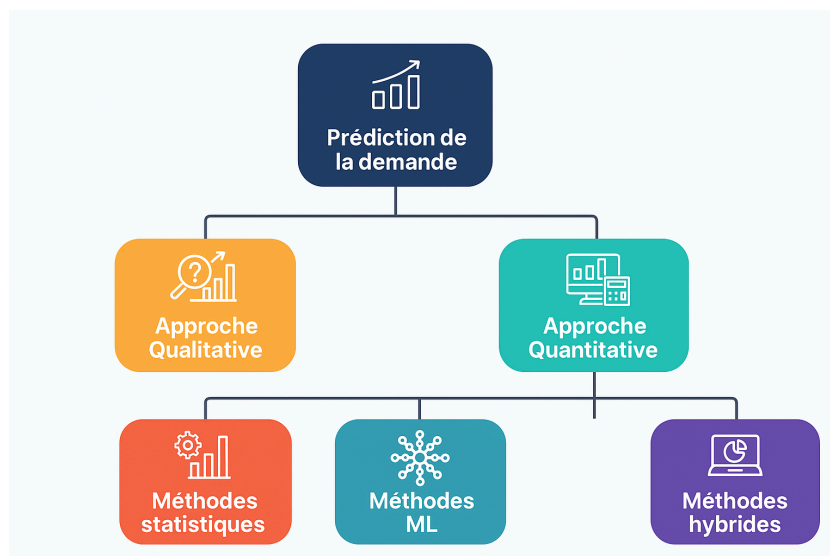


Figure 1.3 Classification des méthodes de prédiction de la demande

1.3.1. Approche Qualitative

On a recours aux méthodes qualitatives lorsque les données historiques sont absentes, insuffisantes ou jugées non pertinentes. Nous aborderons dans cette section quelques-unes des approches les plus reconnues dans la littérature.

1.3.1.1. Méthode Delphi

La méthode Delphi est une technique structurée pour obtenir l'avis consensuel d'un groupe d'experts. Elle se déroule de manière itérative et anonyme, sous la conduite d'un facilitateur.

Voici le processus simplifié comme cité par *Wright, 1991 (P 235-251)*:

1. Les experts répondent individuellement à des questionnaires.
2. Le facilitateur synthétise anonymement les réponses.
3. Cette synthèse est partagée avec les experts.
4. Chaque expert réévalue son jugement à la lumière des avis collectifs, sans influence directe.
5. Le processus se répète jusqu'à ce que les avis convergent.
6. Le résultat final est l'agrégation de ce consensus stable.

1.3.1.2. Prévision par analogie

La prévision par analogie est une approche qualitative qui consiste à estimer le comportement futur d'un nouvel élément ou un produit en se basant sur l'historique d'éléments similaires du passé. Son principe est simple : si deux produits partagent des caractéristiques communes, leur demande devrait suivre une tendance comparable. Cette méthode est donc particulièrement utile pour anticiper la demande de nouveautés, pour lesquelles aucune donnée historique n'est disponible.

Sa mise en œuvre repose fortement sur le jugement humain, car il faut d'abord identifier les "*analogues*" les plus pertinents, puis décider de la manière de combiner leurs historiques de ventes pour générer la prévision.

Le processus structuré, proposé par *Green et al. (2007)*, s'appuie généralement sur un panel d'experts. Ces derniers identifient plusieurs analogies possibles, évaluent leur pertinence, puis un animateur synthétise leurs analyses pour construire une prévision finale, souvent en pondérant les estimations selon la force de chaque analogie.

1.3.2. Approche Quantitative

Les approches quantitatives exploitent les données historiques pour modéliser la demande et produire des prévisions chiffrées. Elles sont divisées en :

1.3.2.1. Méthodes Statistiques classiques

Les modèles classiques sont la base pour prévoir les séries temporelles. Ils recherchent des relations, surtout simples (*linéaires*), dans les données passées d'une série. Ils sont reconnus pour leur facilité à comprendre, leur fiabilité et leur rapidité de calcul. Pour bien les utiliser, il faut d'abord analyser la série (*ses tendances, saisons, etc ...*). Un concept clé est la stationnarité, ce qui signifie que les caractéristiques de la série (comme sa moyenne, sa variance) restent stables dans le temps. Ci-dessous les méthodes classiques les plus prédominantes dans la littérature cités par *Brockwell & Davis, (2013)*:

Modèle Autorégressif (AR): Ce modèle, appelé AutoRégressif, postule que la valeur actuelle d'une série temporelle X_t peut être exprimée comme une fonction linéaire de ses propres valeurs passées. Il est particulièrement adapté pour capturer la dépendance d'une observation par rapport à ses antécédents directs.

On le note $AR(p)$, où la lettre ' p ' représente un nombre entier positif qui est l'ordre du modèle, indiquant le nombre de valeurs passées considérées.

$$X_t = c + \sum_{i=1}^p \varphi_i X_{t-i} + \varepsilon_t \quad (1.3)$$

Modèle à Moyenne Mobile (MA): Contrairement au modèle AR, le modèle de *Moyenne Mobile* explique la valeur actuelle d'une série comme une combinaison linéaire de ses erreurs de prédiction passées. Cela signifie que les chocs aléatoires ou les surprises des périodes précédentes ont un impact sur la valeur actuelle de la série.

On le note $MA(q)$, où la lettre ' q ' représente un nombre entier positif qui est l'ordre du modèle, indiquant le nombre de termes d'erreur passés pris en compte.

$$X_t = \mu + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \quad (1.4)$$

Moyenne mobile intégrée autorégressive (Autoregressive Integrated Moving Average - ARIMA) : Formalisé par *George Box et Gwilym Jenkins 1970*, le modèle ARIMA étend le modèle ARMA, pour traiter les séries temporelles non stationnaires, en particulier celles qui présentent une tendance. Le I pour “*Intégrée*” signifie que la série originale a été différenciée une ou plusieurs fois pour la rendre stationnaire avant d'appliquer un modèle ARMA. On le note $ARIMA(p,d,q)$ comme illustré dans la **Figure 1.4**, p est l'ordre de la partie Autorégressive, d est le degré de différenciation nécessaire pour rendre la série stationnaire, et q est l'ordre de la partie Moyenne Mobile.

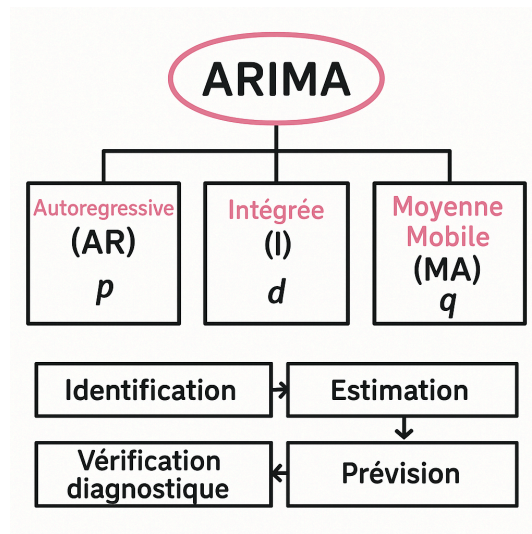


Figure.1.4 Représentation schématique d'un modèle ARIMA.

Lissage Exponentiel : Cette famille de méthodes de prévision consiste à calculer une moyenne pondérée des observations passées. La particularité est que les poids diminuent de manière exponentielle, ce qui signifie que les données les plus récentes ont une influence bien plus forte sur la prévision que les données plus anciennes.

Le *Lissage Exponentiel Simple* (LES) est la forme la plus basique de cette méthode. Il est spécifiquement adapté pour prédire des séries temporelles qui ne montrent ni tendance claire (*pas d'augmentation ou de diminution constante*) ni saisonnalité (*pas de cycles réguliers comme annuels ou mensuels*).

Sa formule de calcul est la suivante :

$$s_t = \alpha y_t + (1 - \alpha) s_{t-1} \quad (1.5)$$

Dans cette formule :

- S_t représente la nouvelle valeur lissée (ou la prévision) calculée pour le moment actuel(t).
- Y_t est l'observation réelle (la vraie valeur de la série) à l'instant t .
- S_{t-1} est la valeur lissée calculée juste avant ($t-1$).
- Le paramètre α (alpha), est le facteur de lissage, une valeur comprise entre 0 et 1. Il détermine la part d'importance donnée à la nouvelle observation (Y_t) par rapport à l'ancienne valeur lissée (S_{t-1}) :
 - Si α est proche de 1, la prévision réagit très vite aux dernières observations.
 - Si α est proche de 0, la prévision est plus stable et moins influencée par les fluctuations récentes, se basant davantage sur le passé lissé

1.3.2.2. Méthodes d'Apprentissage Automatique (Machine Learning)

Les méthodes d'apprentissage automatique (*Machine Learning*) sont parmi les plus utilisées dans la prévision des séries chronologiques, ainsi que pour la problématique de prédiction de la demande et du nombre de colis. Elles permettent de détecter des motifs complexes et non linéaires dans les données, offrant ainsi une meilleure précision que les approches purement statistiques dans de nombreux cas.

Support Vector Machine (SVM): est une méthode d'apprentissage supervisée, adaptée à la régression (SVR), qui cherche à trouver la fonction optimale maximisant la marge entre elle et les points de données les plus proches (vecteurs de support). Grâce à l'utilisation de fonctions noyau, le SVM gère les relations non linéaires, ce qui le rend idéal pour les séries temporelles complexes et bruitées.

Dans la littérature, *Kim (2003)* a évalué la performance du SVM pour la prévision d'indices boursiers. Ses résultats indiquent que le SVM surpasse les réseaux de neurones à rétropropagation et les modèles statistiques traditionnels en termes de précision, notamment sur des séries financières caractérisées par une forte volatilité et des non-linéarités marquées.

XGBoost : Introduit par *Chen & Guestrin (2016)* comme schématisé dans la **Figure 1.5**, est un algorithme d'ensemble basé sur le gradient boosting d'arbres de décision. Il est largement utilisé pour la prévision de séries temporelles multivariées, notamment dans la logistique, grâce à sa capacité à intégrer des variables exogènes et à gérer des volumes importants de données.

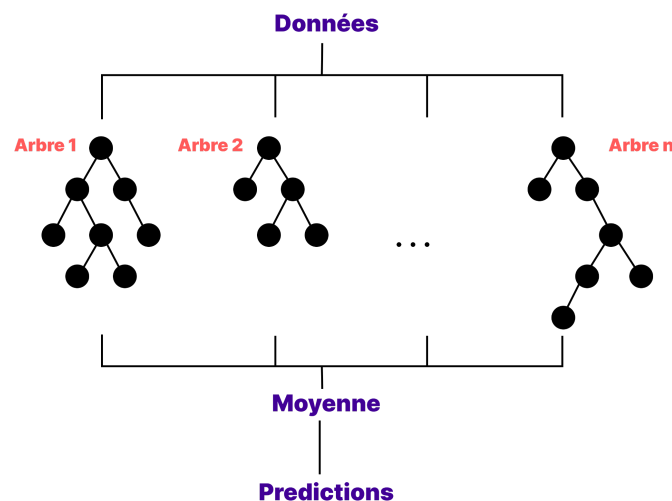


Figure.1.5 Représentation schématique d'un modèle XGBoost

Une étude scientifique récente menée par Aiyegbeni Gifty *et al.*, 2024. Elle visait à créer des modèles capables de prédire le prix de clôture de l'action pour le lendemain en se basant uniquement sur les prix de clôture passés, *XGBoost* atteint un R^2 de 99,35 %, surpassant *LSTM* (96,87 %) et *ARIMA* (66,37 %) pour la prédiction de séries boursières, avec les plus faibles *MAE* et *RMSE*.

1.3.2.3. Méthodes d'Apprentissage Profond(Deep Learning - DL)

Dans cette section nous détaillerons les méthodes d'apprentissage profond et de réseaux de neurones les plus cités dans la littérature, mais avant d'en entamer, on doit en expliquer certains termes :

Éléments de base

L'apprentissage profond est une branche du Machine Learning qui s'appuie sur des modèles de calcul organisés en multiples couches. L'idée, popularisée par des pionniers, est de permettre à la machine d'apprendre des représentations de plus en plus abstraites des données, couche après couche (LeCun *et al.*, 2015)

À la base de tout cela se trouve le *Réseau de Neurones Artificiels* (ANN). Inspiré du cerveau humain, un ANN est un système composé de processeurs élémentaires interconnectés (Touzet, 1992) – les neurones – qui travaillent ensemble. L'information circule à travers le réseau de manière structurée :

1. Les données brutes sont fournies à une couche d'entrée.
2. Elles traversent ensuite une ou plusieurs couches cachées, où les transformations et l'apprentissage s'opèrent.
3. Enfin, un résultat est produit par la couche de sortie.

Le terme "*profond*" fait simplement référence à la présence de plusieurs couches cachées. Comme l'explique (Chollet, 2018), la profondeur d'un modèle n'est rien d'autre que le nombre de couches qui le composent. Un réseau avec plus de trois couches est déjà considéré comme profond.

Neurone Artificiel (Perceptron): L'unité de base de tout réseau de neurones est le neurone artificiel, aussi appelé *perceptron*. Inspiré du neurone biologique, son fonctionnement est simple (illustré dans la **Figure 1.6**) :

1. Il reçoit plusieurs valeurs en entrée x_i , chacune associée à un poids w_i qui représente son importance.
2. Il calcule la somme pondérée de ces entrées et y ajoute une valeur de décalage, le biais b .

3. Ce résultat est ensuite passé à travers une fonction d'activation f qui décide de la valeur finale y que le neurone enverra à la couche suivante.

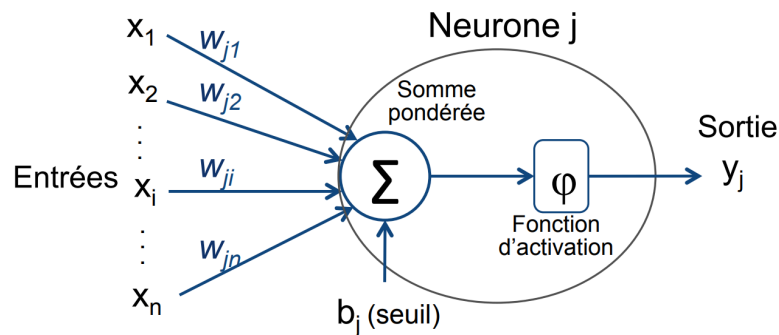


Figure.1.6 Structure d'un neurone artificiel (Laqrichi, 2015 p.36).

Cette *fonction d'activation* agit comme un interrupteur ou un régulateur, transformant le signal pour introduire des non-linéarités, ce qui permet au réseau d'apprendre des relations complexes.

Fonction	Fonction Linéaire	Sigmoid	ReLu	Softmax
Expression	$f(x)=a.x$	$f(x) = \frac{1}{1 + e^{-x}}$	$f(x)=\max(0, x)$	$f(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$

Tableau 1.1 - Expressions de quelques fonctions d'activation.

Après avoir expliqué les concepts de base, nous allons maintenant présenter les méthodes de prévision de séries temporelles les plus reconnues, fréquemment citées dans la littérature scientifique :

Perceptron multicouche (MLP) : C'est un réseau de neurones de base qui fonctionne en propagation avant (*feedforward*) : l'information circule simplement de l'entrée vers la sortie, en passant par des couches intermédiaires comme illustré dans la **Figure 1.7**. Il réalise des calculs complexes pour trouver des relations non linéaires dans les séries temporelles, ce qui lui permet de détecter des schémas que les modèles linéaires classiques ne verraient pas.

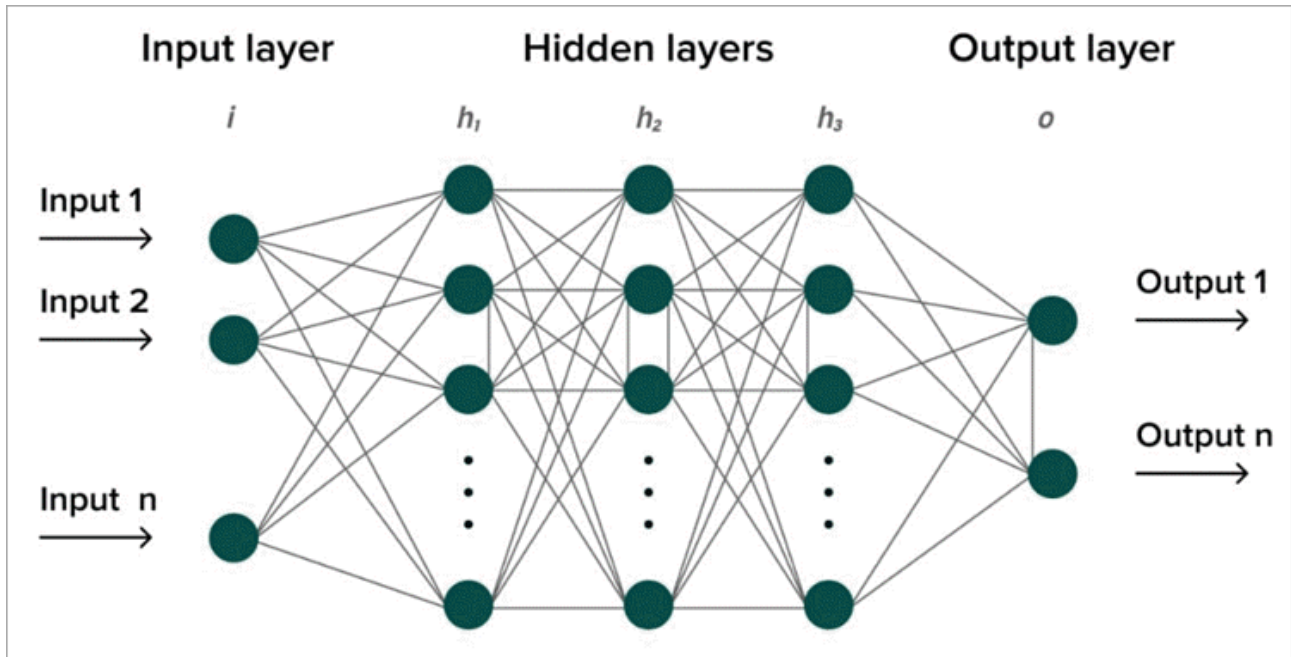


Figure.1.7 Un exemple de perceptron multicouche

Dans la littérature, il existe le *MLP-SUL* (Kim & Hwang, 2023), est une évolution du Perceptron Multicouche classique, spécifiquement conçue pour améliorer sa gestion des dépendances complexes dans les données séquentielles. Cette innovation repose sur l'intégration de deux mécanismes clés : la couche de sélection et la couche de mise à jour comme illustré dans la **Figure 1.8** et **Figure1.9**

- La couche de sélection fonctionne comme un mécanisme d'attention ou de *gating*. Elle évalue dynamiquement l'importance (la pertinence) de chaque variable d'entrée à un instant donné, attribuant des poids appris aux différentes caractéristiques. Cela permet au modèle de filtrer activement les informations moins pertinentes et de concentrer ses ressources de traitement sur les *features* les plus informatives, optimisant l'extraction de signaux contextuels des données brutes.
- La couche de mise à jour intervient après ce processus de sélection. Son rôle est de transformer et d'intégrer les informations filtrées et pondérées par la couche de sélection dans la représentation interne du modèle. Elle agit comme un module de raffinement qui met à jour les états ou les *features* latentes du réseau, permettant ainsi au MLP de s'adapter et de réagir de manière plus précise aux dynamiques des séries temporelles. Cette couche permet de moduler l'influence des données d'entrée sur les couches suivantes du réseau.

Ce modèle a été appliqué avec succès à la prévision de la demande postale en *Corée du Sud*, atteignant un *MAPE* inférieur à 0,15 lors des pics saisonniers, et surpassant *LSTM* et la régression linéaire.

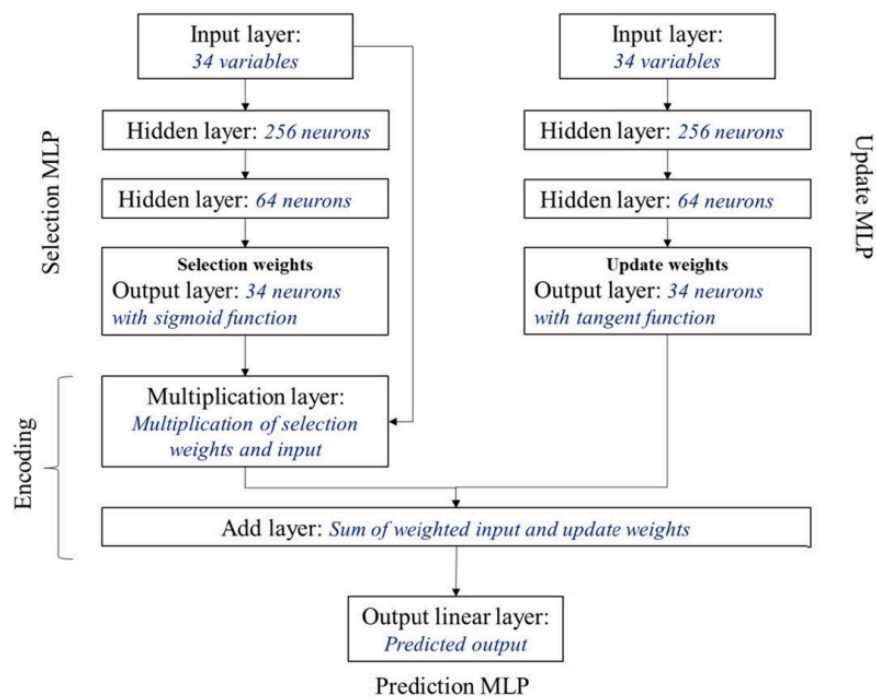


Figure.1.8 Visualisation du modèle de MLP-SUL (Kim & Hwang, 2023)

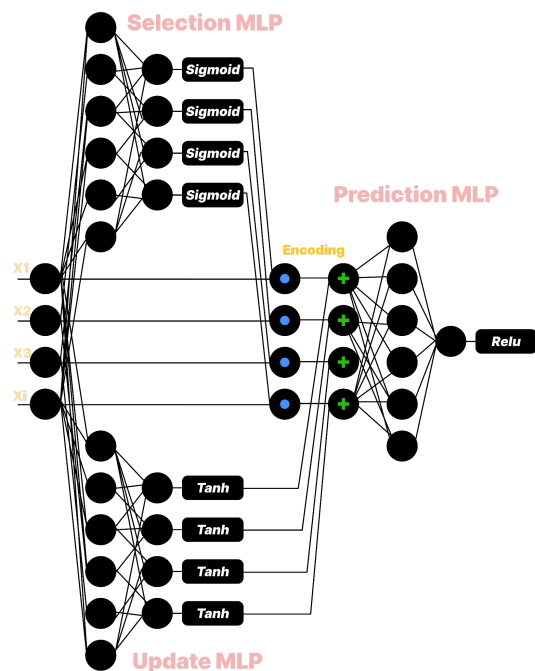


Figure 1.9 Architecture simplifiée d'un réseau de neurones MLP-SUL

Dans la continuité de ces avancées, l'architecture TSMixer Chen et al., (2023) est un autre exemple marquant de l'intérêt et de l'efficacité des MLP pour la prévision de séries temporelles. Ce modèle a la particularité d'utiliser uniquement des MLP pour prédire plusieurs séries temporelles à la fois (*prévision multivariée*) dans un contexte industriel.

Son innovation est de traiter les dépendances complexes via des MLP en deux "mixages" :

- Le mixage temporel analyse les liens dans le temps pour une même série.
- Le mixage de caractéristiques (variables) relie les données entre différentes séries au même instant.

En appliquant ces "mixages" de manière innovante, *TSMixer* prouve que des architectures *MLP* plus simples peuvent être très performantes, même face à des modèles plus complexes comme les *LSTM*.

Réseaux de Neurones Récurrents (RNN) : Les réseaux de neurones récurrents (RNN) comme illustré dans la **Figure 1.10** sont utilisés pour traiter des données séquentielles. Ils conservent une mémoire des états passés, ce qui les rend adaptés à la prévision de séries temporelles présentant des dépendances courtes. Cependant, les RNN classiques souffrent du problème du gradient évanouissant, ce qui limite leur efficacité sur de longues séquences. Les variantes avancées comme *LSTM* et *GRU* ont été développées pour pallier ces limites.

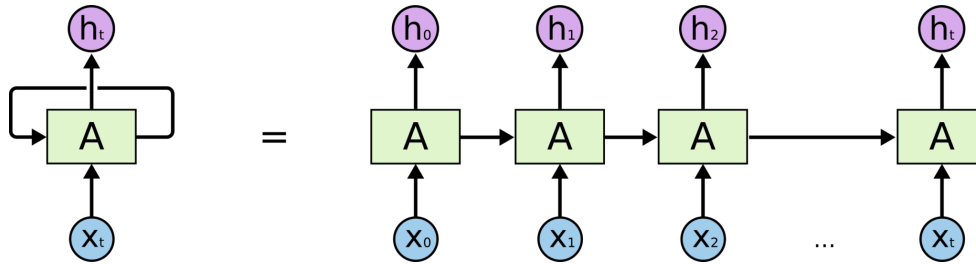


Figure 1.10 Architecture RNN (Colah 2015)

Long Short-Term Memory (LSTM) : Les réseaux de neurones à mémoire longue (*LSTM*), proposés par *Hochreiter & Schmidhuber* en 1997, est une architecture avancée de réseaux récurrents conçue pour modéliser efficacement les dépendances à long terme dans les séries temporelles, en surmontant le problème du gradient évanouissant qui limite les (*RNN réseaux récurrents classiques*). Grâce à ses cellules mémoire et à ses portes d'entrée, d'oubli et de sortie comme illustré dans la **Figure 1.11**.

$$\text{(Porte d'oubli)} \quad f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1.6)$$

$$\text{(Porte d'entrée)} \quad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (1.7)$$

$$\text{(MAJ de l'état)} \quad C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (1.8)$$

$$\text{(Porte de sortie)} \quad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (1.9)$$

$$\text{(État caché)} \quad h_t = o_t \cdot \tanh(C_t) \quad (1.10)$$

Le *LSTM* contrôle dynamiquement le flux d'information, ce qui lui permet de retenir ou d'oublier sélectivement des informations pertinentes sur de longues périodes. Cette capacité en fait un modèle particulièrement adapté à la prévision de séries temporelles complexes présentant tendances, saisonnalités ou événements extrêmes. Toutefois, malgré ses performances reconnues, il nécessite un entraînement coûteux en ressources et peut s'avérer sensible au choix des hyperparamètres.

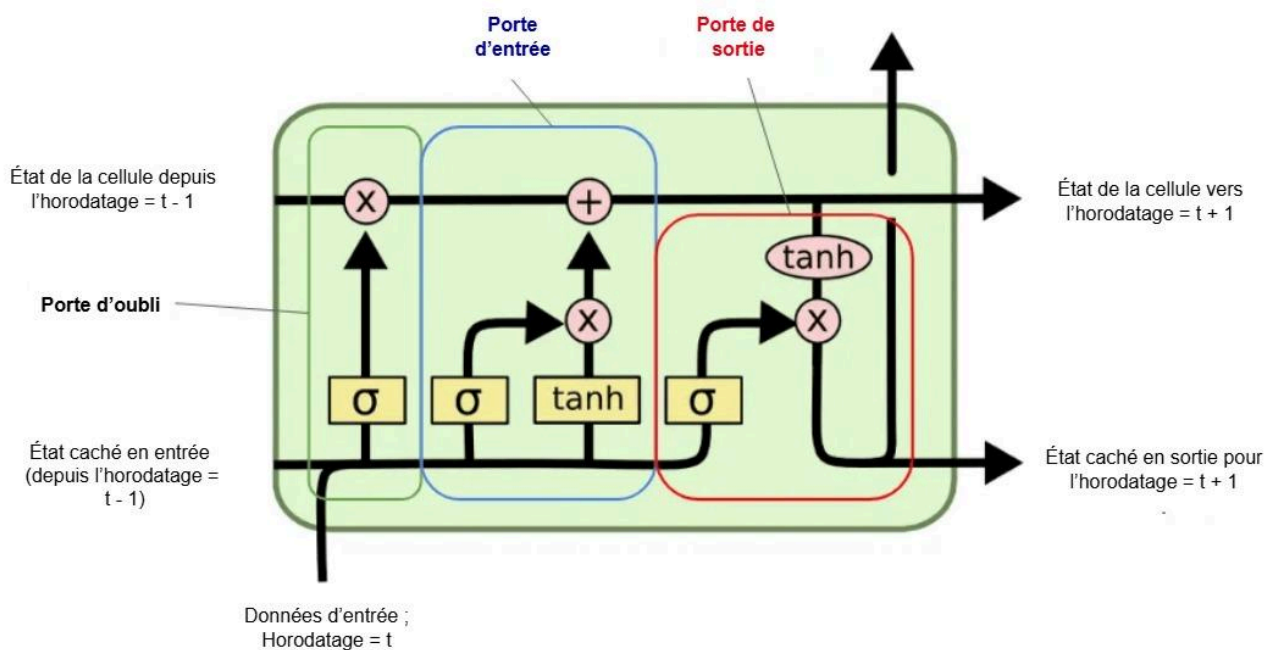


Figure 1.11 Cellule LSTM (Colah 2015)

Les réseaux LSTM (Long Short-Term Memory) ont démontré une efficacité remarquable pour la prévision de séries temporelles complexes.

Par exemple, dans le domaine de l'hydrologie, *BELASLA et al.* (2024) ont réalisé une étude sur la prévision des précipitations sur 120 mois. Les résultats ont montré que les architectures LSTM multicouches, intégrant des mécanismes comme le **dropout**, surpassent les méthodes classiques en capturant les dépendances à long terme dans les données. Dans l'industrie, (*Uber Engineering Team, 2017*) a utilisé des LSTM empilés (c'est-à-dire plusieurs couches de LSTM superposées) pour la prévision de la demande de courses, intégrant des variables exogènes telles que la météo et les jours fériés, ce qui a permis une amélioration significative des erreurs de prévision (jusqu'à 36% de réduction du SMAPE) même lors d'événements exceptionnels, confirmant ainsi la robustesse et la pertinence des LSTM pour des applications réelles et complexes.

Ces études illustrent la robustesse du LSTM face aux défis de prévision dans des contextes variés où les dépendances temporelles et les motifs non linéaires sont prédominants.

Prophet (Meta, 2017) : Développé par l'équipe *Core Data Science de Meta*, Prophet est un outil open source conçu pour la prévision de séries temporelles avec saisonnalités complexes, tendances non linéaires et effets de jours fériés. Contrairement aux modèles de deep learning, Prophet repose sur un modèle additif décomposable inspiré des travaux de *Cleveland et al.* sur les *STL (Seasonal-Trend Décomposition)*, combiné à une approche bayésienne pour l'ajustement automatique des hyperparamètres.

Ce qui est, une étude comparative sur la prévision du nombre d'hospitalisations (*Frontiers in Public Health*, 2022) a montré que le *LSTM* obtient la meilleure précision (*MAPE et RMSE les plus faibles*), suivi de *Prophet*, alors que *SARIMA* arrivait en dernier. De même, des expérimentations industrielles confirment que *Prophet* reste le choix privilégié pour des prévisions rapides et interprétables, tandis que *LSTM* offre de meilleures performances lorsque la structure des données est complexe ou fortement bruitée.

Techniques d'optimisation

Dans cette sous-section, nous détaillerons les méthodes d'optimisation utilisées pour entraîner les modèles de séries temporelles, notamment les réseaux de neurones.

Descente de Gradient Stochastique (SGD) : C'est est la méthode de base pour entraîner les réseaux de neurones. Au lieu d'analyser toutes les données à la fois, le *SGD* met à jour les paramètres du modèle petit à petit, en utilisant de petits groupes de données aléatoires (appelés "*mini-batch*") à chaque étape. Malgré sa rapidité, le *SGD* classique a des limites : il peut être lent à trouver la meilleure solution, osciller fortement autour d'elle, et avoir du mal à sortir des zones pièges (*minimums locaux ou zones plates*). Pour surmonter ces problèmes et accélérer l'entraînement, des techniques d'optimisation plus avancées ont été développées.

Optimiseurs Avancés

Dans cette sous section, nous allons détailler quelques optimiseurs connus dans la littérature pour les réseaux de neurones

Momentum : Cette méthode cherche à accélérer la convergence en donnant à la descente de gradient une sorte de "*mémoire*". Plutôt que de baser sa direction uniquement sur le gradient à l'instant présent, elle intègre une fraction des directions prises lors des étapes précédentes. On peut imaginer une boule de neige qui dévale une pente : elle accumule de l'élan (*le momentum*), ce qui l'aide à passer outre les petits obstacles (*les minimums locaux*)

et à accélérer dans les longues descentes régulières. L'importance de cet élan est contrôlée par un facteur, généralement autour de 0.9, qui accélère et stabilise la convergence (Du, 2019), comme le montre la **Figure 1.12**.

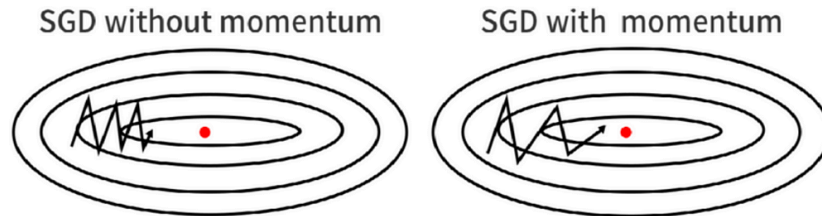


Figure 1.12 Comparaison de l'algorithme SGD avec et sans momentum (Du, 2019)

RMSProp (Root Mean Square Propagation) : L'idée centrale de RMSProp est de se débarrasser du problème d'un taux d'apprentissage unique et constant. Cette technique ajuste dynamiquement le taux d'apprentissage pour chaque paramètre du modèle, de manière individuelle. Elle le fait en se basant sur la magnitude des gradients récents : si un paramètre subit de fortes variations, son taux d'apprentissage est réduit pour éviter les oscillations et la divergence, tandis que s'il évolue peu, son taux est augmenté pour accélérer sa convergence.

Adam (Adaptive Moment Estimation) : Proposé dans Kingma and Ba, (2014) considéré comme une des méthodes les plus robustes, Adam combine intelligemment les avantages du Momentum et de RMSProp.

- De Momentum, il hérite de l'utilisation des gradients passés pour accélérer le processus et naviguer plus efficacement dans le paysage de la fonction de coût.
- De RMSProp, il emprunte l'adaptation individuelle du taux d'apprentissage pour chaque paramètre.

1.3.2.4. Méthodes Hybrides

Les méthodes hybrides combinent les forces des modèles statistiques classiques, tels que ARIMA, avec celles des approches d'apprentissage automatique ou profond, comme les réseaux de neurones artificiels (ANN) ou les architectures LSTM, afin d'améliorer la précision globale des prévisions. Cette combinaison permet de modéliser efficacement à la fois les composantes linéaires (*capturées par ARIMA*) et les dynamiques non linéaires complexes (*modélisées par ANN ou LSTM*). Dans la littérature, Zhang et al. (2003) ont proposé l'un des premiers modèles hybrides ARIMA-ANN pour la prévision de séries temporelles, démontrant que la combinaison d'un modèle linéaire (ARIMA) pour capturer les tendances et d'un réseau de neurones artificiel (ANN) pour modéliser les composantes non linéaires permet d'obtenir une précision supérieure à celle de chaque modèle pris

individuellement, notamment sur des séries économiques et financières.

Plus récemment, *Jin et al. (2023)* ont comparé les performances de modèles *ARIMA*, *LSTM* et hybrides *ARIMA-LSTM* pour la prévision de séries financières, montrant que le modèle hybride *ARIMA-LSTM* atteint un R^2 de 0,99 et des erreurs (*MSE*, *RMSE*, *MAE*) nettement inférieures à celles des modèles simples, confirmant l'intérêt des approches hybrides pour intégrer à la fois les dynamiques linéaires et non linéaires des séries temporelles complexes

1.4. Métriques d'évaluation

L'évaluation d'un modèle prédictif en séries temporelles, cruciale pour estimer la perte sur des données inconnues selon *Shcherbakov et al., (2013)*. nécessitant une attention particulière à la dépendance des observations, utilise des métriques spécifiques pour mesurer la précision des prévisions, où une erreur désigne la différence entre la valeur observée et sa prévision, représentant la composante imprévisible plutôt qu'une faute.

1.4.1. Erreurs dépendantes de l'échelle

Les erreurs de prévision peuvent être mesurées sur la même échelle que les données, ce qui rend ces métriques inadaptées à la comparaison entre séries de différentes unités.

Les deux mesures dépendantes de l'échelle les plus courantes sont :

1.4.1.1. Erreur Absolue Moyenne (MAE)

La *MAE* est la moyenne des valeurs absolues des erreurs de prévision. Elle est définie comme suit :

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (1.11)$$

La *MAE* est moins sensible aux valeurs aberrantes que la *RMSE* et est exprimée dans la même unité que la variable cible, ce qui facilite son interprétation (*Chai & Draxler, 2014*).

1.4.1.2. Erreur quadratique moyenne (MSE)

La *MSE* est la moyenne des carrés des écarts entre les valeurs prédites et observées. Elle pénalise fortement les grandes erreurs, ce qui la rend sensible aux valeurs aberrantes. La *MSE* est exprimée dans le carré de l'unité de la variable cible, ce qui peut rendre son interprétation moins intuitive *MSE*, Elle est définie comme suit :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1.12)$$

1.4.1.3. Racine de l'erreur quadratique moyenne (RMSE)

La *RMSE* est la racine carrée de la *MSE*, ce qui permet de ramener l'unité de la métrique à celle de la variable prédite. Elle est largement utilisée pour évaluer les modèles de séries temporelles, car elle fournit une mesure directe de l'écart-type des erreurs de prévision, facilitant ainsi l'interprétation *RMSE*. Elle est définie comme suit :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1.13)$$

La *RMSE* donne plus de poids aux grandes erreurs en raison du terme quadratique. Le choix entre *MAE* et *RMSE* dépend de l'importance que l'on accorde aux grandes erreurs.

1.4.2. Erreurs en pourcentage

Les erreurs en pourcentage sont sans unité, ce qui les rend adaptées à la comparaison entre différentes séries. Ces mesures incluent l'Erreur en Pourcentage Moyenne *MPE*, l'Erreur Absolue Moyenne en Pourcentage *MAPE* et l'Erreur Absolue Moyenne Symétrique en Pourcentage *SMAPE*

1.4.2.1. Erreur Absolue Moyenne en Pourcentage (MAPE)

La *MAPE* exprime l'erreur absolue moyenne en pourcentage des valeurs observées. Elle exprime l'erreur absolue moyenne en pourcentage des valeurs observées. Elle est définie comme suit :

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (1.14)$$

Cependant, cette métrique présente des limites : elle devient infinie ou indéfinie si une valeur observée y_i est nulle, et elle peut générer des valeurs extrêmes lorsque les observations sont proches de zéro. De plus, elle suppose que l'unité de mesure a un zéro significatif, ce qui n'est pas toujours le cas.

1.4.2.2. Erreur Absolue Moyenne Symétrique en Pourcentage (SMAPE)

La *SMAPE* est une alternative à la *MAPE* qui tente de corriger certains de ses inconvénients en utilisant la moyenne des valeurs absolues observées et prédites au

dénominateur. *SMAPE*, Elle est définie comme suit :

$$SMAPE = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|) / 2} \quad (1.15)$$

Bien que conçue pour être symétrique, la *SMAPE* peut toujours être affectée par les valeurs nulles et a ses propres limitations d'interprétation. *SMAPE* peut toujours être affectée par les valeurs nulles et a ses propres limitations d'interprétation

Le choix d'une métrique d'évaluation appropriée dépend fortement des caractéristiques spécifiques des données et des objectifs de la prévision. Il est souvent recommandé d'utiliser une combinaison de plusieurs métriques pour obtenir une vue d'ensemble complète de la performance du modèle.

1.5. Conclusion

Ce chapitre a introduit les séries temporelles et leurs méthodes de prévision. Cependant la fiabilité de ces résultats reste un défi face au *Concept Drift*, un phénomène qui sera détaillé dans le chapitre suivant.

Chapitre 02

Détection et Caractérisation du Concept Drift

2.1. Introduction

Ce chapitre présentera le concept drift, un phénomène où les données évoluent constamment, affectant la fiabilité des modèles d'apprentissage automatique, et en détaillera les sources et méthodes de détection.

2.2. Définition du Concept Drift

Le concept drift survient lorsque la relation entre les données d'entrée (X) et ce que le modèle prédit (y) change avec le temps. Ce phénomène, introduit par *Schlimmer et Granger (1986)*, rend les modèles obsolètes car leurs "règles" apprises ne correspondent plus à la réalité. Formellement, la distribution de probabilité liant X à y n'est plus la même. Elle s'écrit comme cela :

$$\exists t : P_t(X, y) \neq P_{t+1}(X, y) \quad (2.1)$$

2.3. Sources du concept drift

Pour identifier l'origine du concept drift, on décompose la situation en deux éléments clés : la distribution des données d'entrée $P_t(X)$ et la relation prédictive qui les lie à la variable à prédire $P_t(X, y)$. Une dérive survient si l'un de ces éléments, ou les deux, change.

2.3.1. Le changement dans le profil des données

Ici, ce sont les données d'entrée elles-mêmes qui changent, mais la règle de décision du modèle reste la même ($P_t(X)$ change, $P_t(y | X)$ non). Cette dérive n'affecte pas la frontière de décision du modèle et est appelée dérive virtuelle *Ramírez-Gallego et al. (2017)*. Le modèle est juste confronté à des données pour lesquelles il n'a pas été directement préparé. Voir la **Figure 2.1** pour une illustration.

Exemple : Un modèle formé sur des commentaires en français reçoit soudainement une majorité de commentaires en anglais. La définition de la *toxicité* reste identique, mais le format des données d'entrée a changé.

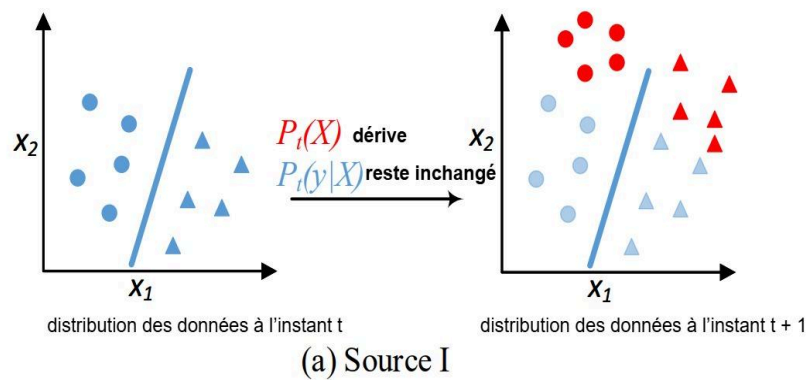


Figure 2.1 Première source du concept drift (Lu et al. 2020)

2.3.2. Changement dans la relation prédictive

Dans ce cas, c'est la manière dont les caractéristiques prédisent la cible qui évolue. Les règles du jeu ont été modifiées. Voir la **Figure 2.2** pour une illustration.

Par exemple : Un modèle de détection de fraude bancaire signale une transaction étrangère comme suspecte. Suite à une nouvelle réglementation, cette même transaction devient normale. La règle de détection a changé, pas les données.

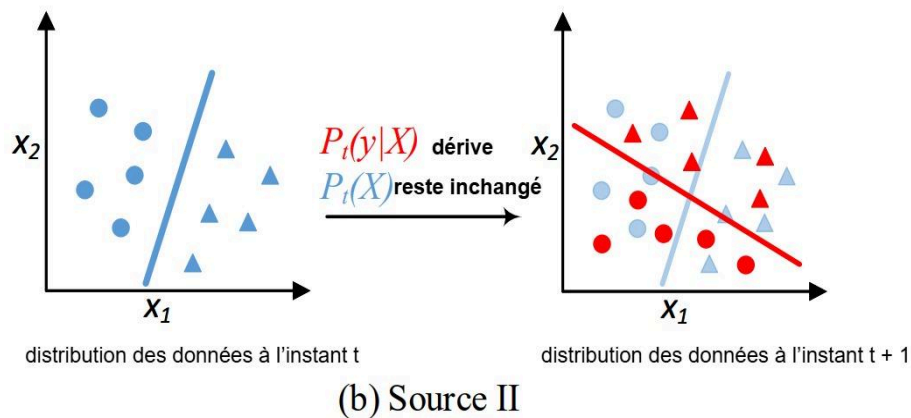


Figure 2.2 Deuxième source du concept drift (Lu et al.2020)

2.3.3. Combinaison des deux (Dérive Hybride)

Dans la plupart des situations réelles, les deux phénomènes se produisent simultanément, le profil des données d'entrée et la relation prédictive changent en même temps.

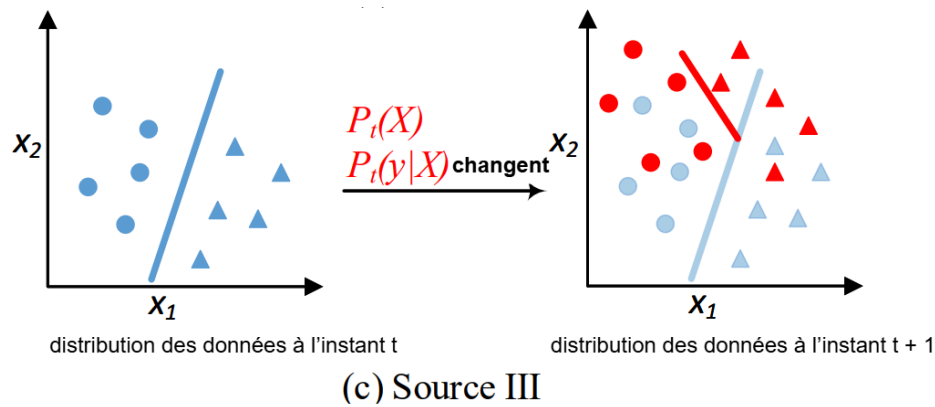


Figure 2.3 Troisième source du concept drift (Lu et al., 2020)

2.4. Types de Concept Drift

Le concept drift se produit lorsque les propriétés statistiques de la variable cible que le modèle tente de prédire changent de manière imprévue avec le temps. Cette variation peut se manifester à différentes vitesses et selon divers schémas, comme l'illustre la **Figure 2.4** pour des données unidimensionnelles. En se basant sur les travaux de *Lemaire et al. (2015)*, on distingue principalement quatre types de dérive

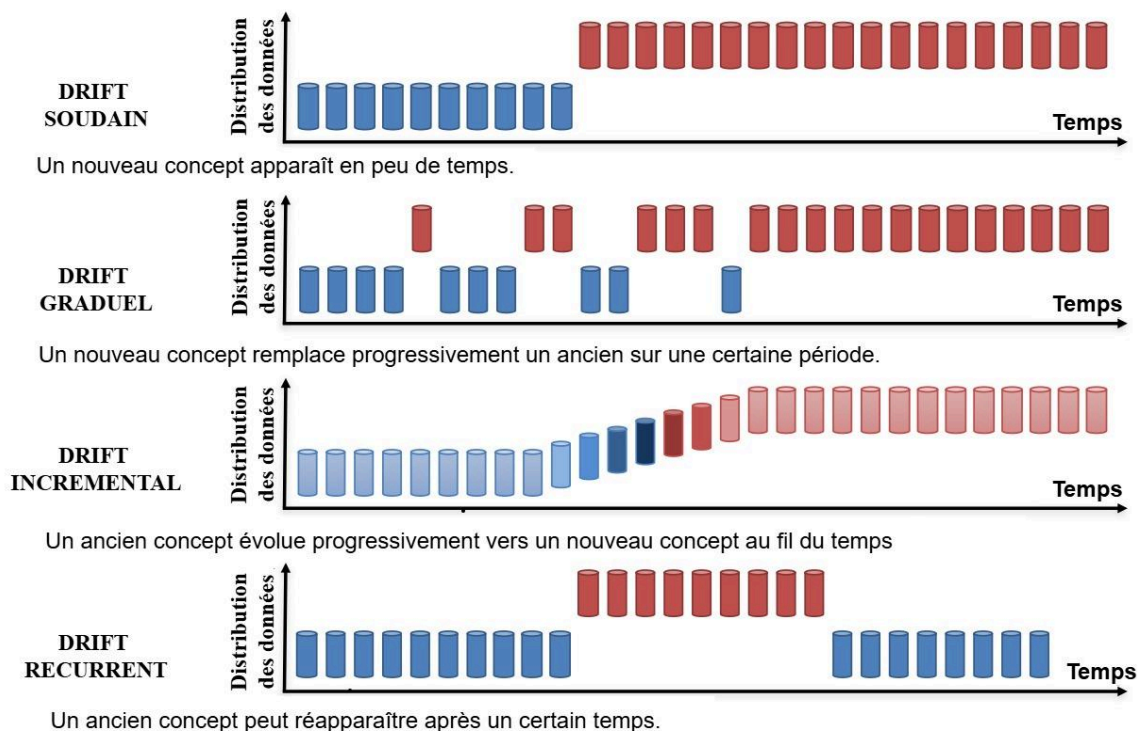


Figure 2.4 Types de concept drift (Lu et al., 2020)

- **Drift soudain** : Le concept S_i est remplacé d'un coup par un autre concept S_j . Le changement est brutal et immédiat. Par exemple, un modèle de fréquentation de restaurants devient obsolète du jour au lendemain suite à un confinement dû à la COVID-19.
- **Drift graduel** : Les deux concepts (S_i et S_j) coexistent un moment. Petit à petit, le nouveau concept prend le dessus. Imaginons un modèle de détection de fraude qui doit s'adapter au passage progressif du paiement en espèces au paiement mobile en Algérie.
- **Drift incrémental** : Le changement est très lent, presque invisible au quotidien, mais significatif sur le long terme. C'est un cas particulier de la dérive graduelle. Un modèle de rendement agricole, par exemple, s'ajuste aux légères augmentations annuelles de la température moyenne.
- **Drift récurrent** : Un ancien concept S_i revient après un certain temps, sans que ce soit forcément régulier. périodique.

2.5. Méthodes de Détection du Concept Drift

La détection du Concept Drift vise à identifier les points ou intervalles temporels où ces changements surviennent. Un cadre général de détection est illustré dans la **Figure 2.5** et comprend :

1. Récupération des données (Data Collection/Retrieval)

La tâche consiste à extraire des éléments de données à partir de flux de données. Pour mener à bien les tâches d'analyse des flux de données, il est nécessaire de comprendre comment organiser les morceaux de données en modèles significatifs ou en connaissances, car une seule instance de données ne peut pas contenir suffisamment d'informations pour en déduire la distribution générale *Ramirez-Gallego et al.,(2017)*

2. Modélisation des données (Data Modeling/Abstraction)

Vise à abstraire les données récupérées en identifiant les caractéristiques critiques ou les statistiques résumées qui sont sensibles aux changements de distribution. Cette étape, bien que facultative, est souvent mise en œuvre pour réduire la dimensionnalité des données, optimisant ainsi les exigences de stockage et la vitesse de traitement en ligne.

3. Calcul des tests statistiques (Statistical Test Calculation)

Est l'estimation de la distance ou la mesure de la dissimilarité. Elle établit des statistiques de test pour le test d'hypothèse et évalue la gravité du drift. Elle est considérée comme la composante la plus difficile de la détection des concept drift. La question de la définition d'une mesure de dissimilarité fiable et précise n'est pas résolue. L'évaluation du clustering peut également utiliser une mesure de dissimilarité.

4. Test d'hypothèse (Hypothesis Testing)

Cette étape évalue la signification statistique du changement quantifié à l'étape précédente. Elle utilise un test d'hypothèse spécifique pour déterminer si le changement observé est significatif ou s'il relève du bruit aléatoire. Le test d'hypothèse permet de calculer un intervalle de confiance, indiquant la probabilité que le changement soit bien un Concept Drift et non un artefact dû au hasard ou à un biais d'échantillonnage. Sans cette validation statistique, les valeurs brutes des statistiques de test de l'étape 3 sont insuffisantes pour une détection fiable

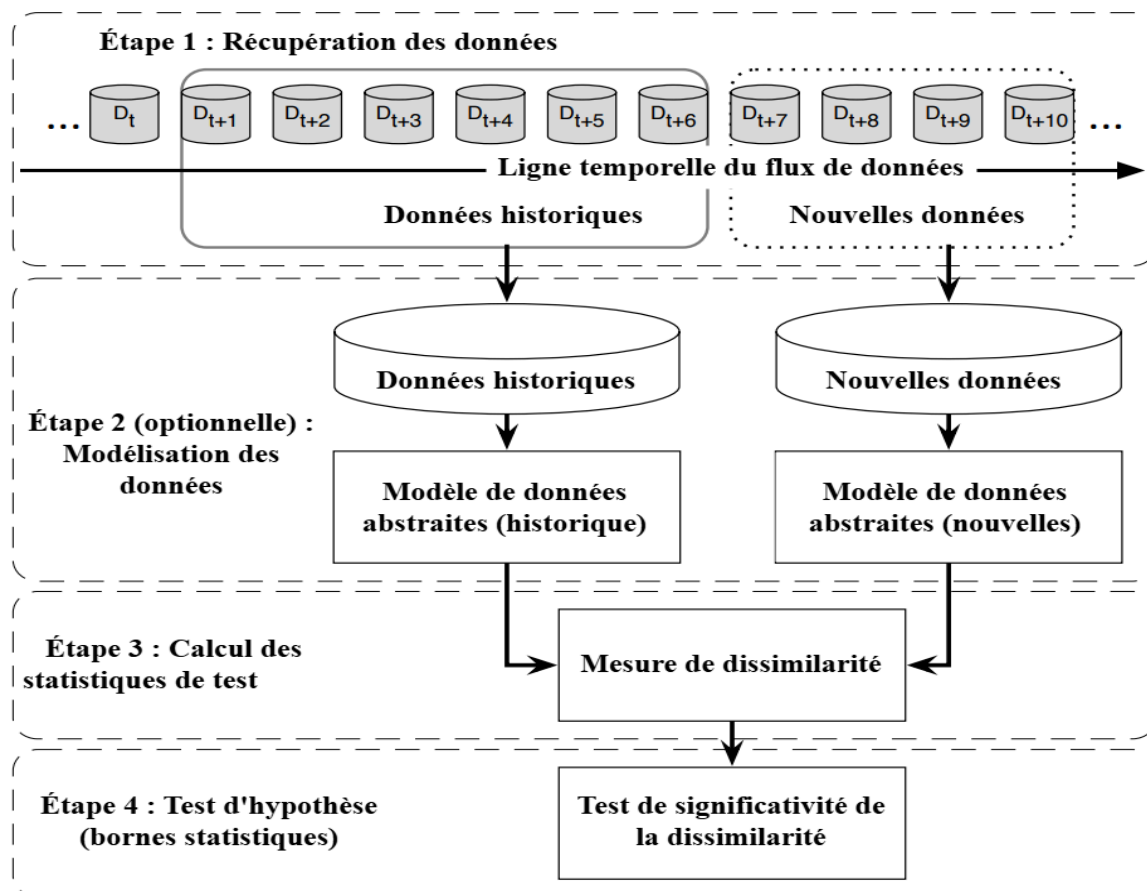


Figure 2.5 Architecture globale d'un détecteur de concept drift (Lu et al., 2020)

Une fois le cadre général de détection du drift présenté, il est important de distinguer les méthodes selon la disponibilité des étiquettes dans le flux de données.

En effet, deux grands cas se présentent dans la littérature :

2.5.1. Cas supervisé

En contexte supervisé, les étiquettes de classe sont disponibles. Les algorithmes sont classés selon deux catégories :

2.5.1.1. Détection basée sur le taux d'erreur

Ces méthodes surveillent les changements dans le taux d'erreur du modèle. Un drift est signalé si cette erreur augmente significativement.

Drift Detection Method (DDM): Un algorithme fréquemment cité *Gama et al., (2014)*. Le DDM détermine si le taux d'erreur agrégé dans une fenêtre a significativement augmenté. Lorsque le changement atteint un seuil de confiance, un nouveau modèle est développé, et le modèle précédent est remplacé si le niveau de drift est atteint. Le DDM utilise le taux d'erreur comme statistique de test. Soit p_i l'indicateur d'erreur pour l'instance i ($p_i=1$ si erreur, 0 sinon).

Le DDM surveille le taux d'erreur moyen :

$$\hat{p}_t = \frac{1}{t} \sum_{i=1}^t p_i \quad (2.2)$$

et son écart-type :

$$\hat{s}_t = \sqrt{\frac{\hat{p}_t(1 - \hat{p}_t)}{t}} \quad (2.3)$$

Une dérive est suspectée (*niveau d'alerte*) si : $\hat{p}_t + \hat{s}_t \geq p_{\min} + 2s_{\min}$ et confirmée (*niveau de dérive*) si : $\hat{p}_t + \hat{s}_t \geq p_{\min} + 3s_{\min}$ où $p_{\min}$ et $s_{\min}$ sont les valeurs minimales de $\hat{p}_t$ et $\hat{s}_t$ observées jusqu'à présent.

LLDD (Learning Link-based Drift Detection): *Gama and Castillo, (2006)*, Le DDM traditionnel surveille l'erreur globale du modèle. LLDD, lui, va plus loin en se concentrant sur les modèles qui sont des arbres de décision. Un arbre de décision prend des décisions en plusieurs étapes, chaque étape étant un "nœud" où une condition est vérifiée (ex: "le client est-il majeur ?").

LLDD analyse les changements d'erreur spécifiquement dans ces nœuds ou branches de l'arbre. Si l'erreur augmente soudainement seulement pour les données qui passent par un certain nœud (*par exemple, pour un groupe de clients avec une caractéristique particulière*), cela signale une dérive localisée. On comprend alors précisément quelle *règle* ou quel sous-ensemble de données est affecté par le drift.

EDDM (Early Drift Detection Method) : *Baena-García et al., (2006)* EDDM est conçu pour détecter la dérive plus rapidement que DDM. Au lieu de se baser uniquement sur la moyenne des erreurs, il s'intéresse à la distance (le nombre d'instances) entre deux prédictions correctes consécutives du modèle.

Lorsque le modèle commence à faire plus d'erreurs, la probabilité d'avoir deux prédictions correctes d'affilée diminue, et la "*distance*" entre elles (*c'est-à-dire le nombre d'erreurs ou d'instances incorrectes entre deux succès*) augmente. Une augmentation significative de cette distance est un signe précoce que le modèle perd de sa précision et qu'une dérive est en cours, parfois avant même que le taux d'erreur global ne change de manière flagrante.

HDDM (Hoeffding Drift Detection Method) *Frías-Blanco et al., (2015)*: HDDM vise à rendre la détection plus robuste et fiable. Il utilise l'inégalité de Hoeffding, un principe mathématique qui permet de calculer la probabilité qu'une moyenne observée (comme le taux d'erreur) soit proche de sa vraie valeur.

En s'appuyant sur cette inégalité, HDDM peut définir des seuils d'alerte et de dérive plus solides et statistiquement justifiés. Cela réduit considérablement le risque de fausses alarmes (*détecter un drift qui n'existe pas*) et rend la détection plus sûre, car elle est moins sensible aux fluctuations aléatoires des données.

2.5.1.2.Détection basée sur la distribution de données

Ces algorithmes opèrent en comparant la distribution des données d'une fenêtre de référence (*données historiques*) avec celle d'une fenêtre courante (*nouvelles données*) . L'objectif est de voir si la répartition des informations (*les "caractéristiques" ou features*) a subi un changement significatif. Si un tel changement est détecté, c'est ce qu'on appelle un Concept Drift. Une alerte est alors émise, signalant que le modèle pourrait avoir besoin d'être mis à jour ou ré-entraîné pour s'adapter à cette nouvelle réalité. Ces méthodes sont particulièrement utiles car elles peuvent souvent identifier quelles informations spécifiques ont changé et à quel moment ce changement s'est produit.

Approche basée sur la théorie de l'information (ITA): Proposée par Yu *et al.* (2021), utilise une structure d'arbre, typiquement un *KdqTree* dans certaines implémentations de flux de données), pour partitionner l'espace des caractéristiques des données historiques et récentes en cellules discrètes. La divergence de Kullback-Leibler est utilisée pour mesurer la différence de densité de probabilité (notée θ_{ITA}) entre les deux fenêtres pour chaque cellule, définie comme suit :

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (2.4)$$

Où $P(x)$ représente la densité de probabilité dans la cellule pour la fenêtre courante, et $Q(x)$ pour la fenêtre de référence. Une dérive est suspectée si la divergence agrégée dépasse un certain seuil. Un test d'hypothèse Bootstrap peut être utilisé pour évaluer la signification statistique de la dérive détectée.

Le *KdqTree*, par exemple, surveille les caractéristiques entrantes en construisant un arbre de référence sur une fenêtre initiale, puis compare les données des fenêtres suivantes à cet arbre pour détecter la dérive.

2.5.1.3. Méthodes basées sur les tests statistiques de changement

Ces algorithmes identifient des points de rupture ou des changements significatifs dans les propriétés statistiques d'un flux de données (*erreurs du modèle ou caractéristiques d'entrée*). Voici un exemple notable :

ADWIN (*Adaptive Windowing*): Proposé par Bifet *et al.* (2007), ADWIN est un détecteur de changement non paramétrique qui maintient une fenêtre glissante de taille variable. Il divise cette fenêtre en deux sous-fenêtres (w_0, w_1) et teste si leurs moyennes (*ou autres statistiques*) sont significativement différentes. Si c'est le cas, la sous-fenêtre la plus ancienne (w_0) est abandonnée. La condition de coupure est souvent basée sur l'inégalité de Hoeffding :

$$|\text{mean}(W_0) - \text{mean}(W_1)| > \epsilon_{\text{cut}} \quad (2.5)$$

où

$$\epsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \ln \left(\frac{4}{\delta} \right)} \quad (2.6)$$

avec m lié à la taille des sous-fenêtres, et δ un niveau de confiance.

2.5.2. Cas Non-Supervisé

Dans un contexte non supervisé, où les étiquettes de classe ne sont pas disponibles (*ou leur accès est coûteux, limité ou retardé*), la détection du "concept drift" doit se baser uniquement sur les propriétés statistiques des données d'entrée (*caractéristiques*)

- **Traitement par modèle fenêtre (*Window-based Processing*)**: Selon la revue de *Lu et al. (2020)*, la stratégie prédominante en détection non supervisée est l'utilisation implicite ou explicite de modèles de fenêtres. Ces méthodes comparent les distributions statistiques des données dans une fenêtre courante à celles d'une fenêtre de référence. Le défi majeur réside dans la détermination de la taille optimale de la fenêtre : une fenêtre trop petite détecte rapidement les dérives soudaines mais est très sensible au bruit et peut générer de fausses alarmes; une fenêtre trop grande est robuste en l'absence de dérive mais réagit lentement aux changements, qu'ils soient soudains ou incrémentaux.
- ***Parallel Activations Drift Detector (Padd)***: Proposé par *Komorniczak & Ksieniewicz (2024)*, le *padd* est une méthode de détection de dérive non supervisée qui utilise les activations d'un réseau de neurones non entraîné (*initialisé aléatoirement*). Le flux de données est divisé en lots. La détection de dérive est basée sur des tests statistiques comparant les distributions des activations passées et actuelles pour toutes les sorties du réseau neuronal. Les paramètres critiques sont le niveau de signification α du test statistique et un seuil indiquant la fraction de tests devant signaler une différence pour déclencher une détection de dérive. Cette approche ne nécessite pas d'accès aux étiquettes pour mettre à jour un modèle en cas de dérive, contrairement à d'autres méthodes basées sur la confiance d'un classifieur entraîné.

2.6. Adaptation aux drifts (changements)

Cette section détaille les stratégies principales utilisées dans la littérature pour adapter les modèles d'apprentissage automatique face aux évolutions des données (*dérive de concept*) afin de maintenir leur performance.

2.6.1. Entraînement de nouveaux modèles pour les drifts globales

Pour les dérives significatives qui affectent l'ensemble du concept sous-jacent, l'approche la plus directe consiste souvent à entraîner un nouveau modèle à partir de zéro en utilisant les données les plus récentes comme illustré dans la **Figure 2.6**. Cette stratégie de réapprentissage global est particulièrement bien adaptée aux dérives soudaines et majeures, où les connaissances acquises par le modèle précédent sont devenues largement obsolètes.

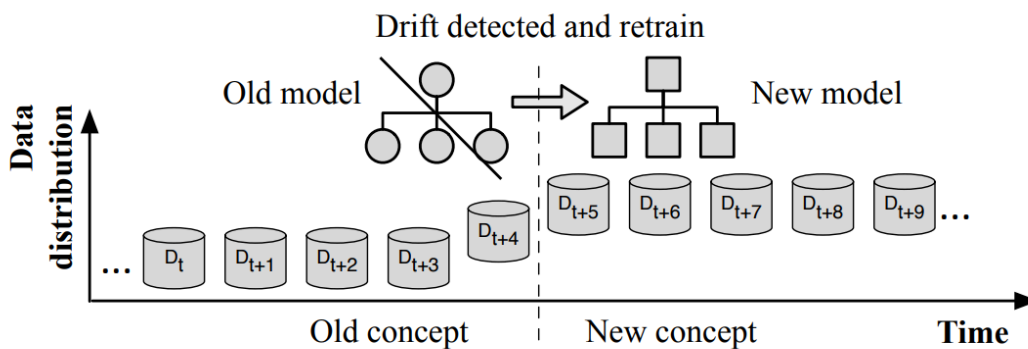


Figure 2.6 Entraînement de nouveaux modèles pour les drifts globales

2.6.2. Utilisation d'ensembles de modèles pour les drifts récurrents

Quand les données d'un modèle changent souvent, ou que d'anciens types de données reviennent, il est très utile d'utiliser des équipes de modèles (*des ensembles*) comme illustré dans la **Figure 2.7**.

L'idée est de ne pas avoir un seul modèle, mais plusieurs modèles qui travaillent ensemble. Quand la nature des données change, on peut soit ajouter un nouveau modèle à l'équipe, entraîné sur les données récentes, soit ajuster l'importance (*le poids*) de chaque modèle existant dans l'équipe selon sa performance actuelle. Cela permet de garder les connaissances anciennes tout en s'adaptant aux nouvelles, assurant que le modèle reste efficace même face à des changements imprévisibles et récurrents. Par exemple :

- **Leveraging Bagging**, proposé par *Bifet et al. (2010)*, Cet algorithme fonctionne comme une équipe qui recrute de nouveaux membres en permanence. Il ajoute régulièrement de nouveaux modèles entraînés sur les données les plus récentes. En même temps, il s'assure que les modèles qui ne sont plus très performants sur les données actuelles ont moins d'influence ou sont même retirés de l'équipe. L'objectif est de toujours avoir une équipe à jour et efficace.
- D'autres méthodes, comme *Learn++.NSE Elwell & Polikar (2011)*, Cette méthode est différente : au lieu d'ajouter ou de retirer des modèles, elle ajuste l'importance (*le poids*) de chaque modèle au sein de l'équipe. Si un modèle est très bon pour prédire les données récentes, son poids augmente, ce qui signifie que son "avis" compte davantage. Si un modèle est moins pertinent, son poids diminue. Cette adaptation dynamique permet à l'ensemble de réagir efficacement, que les changements dans les données soient lents et progressifs ou rapides et soudains.

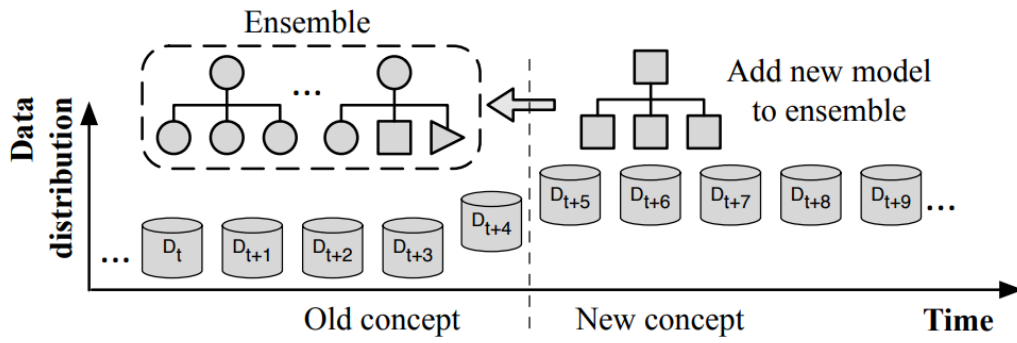


Figure 2.7 Utilisation d'ensembles de modèles pour les drifts récurrents

2.6.3. Ajustement des modèles existants pour les drifts locales

Lorsque les changements n'affectent qu'une partie spécifique d'un modèle ou des régions précises des données, une mise à jour locale ou incrémentale est souvent plus efficace et moins coûteuse qu'un réapprentissage complet comme illustré dans la **Figure 2.8**. Par exemple, dans les arbres de décision, seuls les nœuds ou branches impactés peuvent être ajustés. Les algorithmes comme VFDT et son extension CVFDT (Domingos et Hulten 2000) illustrent cette approche. Le CVFDT, notamment, utilise une fenêtre glissante pour surveiller les scissions de l'arbre et peut remplacer un ancien sous-arbre par un nouveau, plus précis et adapté aux données récentes, en cas de dérive détectée.

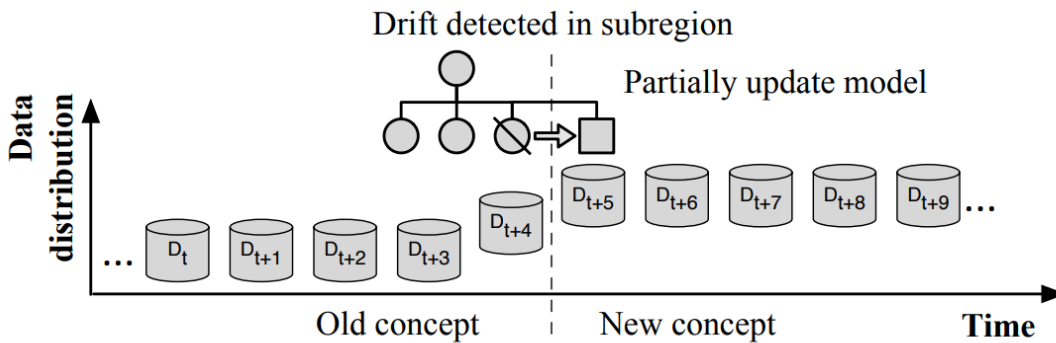


Figure 2.8 Ajustement des modèles existants pour les drifts locales

2.6.4. Conclusion

Ce chapitre présente les mécanismes de détection et les méthodes d'adaptation permettant aux modèles d'apprentissage automatique de gérer le concept drift et d'assurer la fiabilité de leurs prévisions.

PARTIE I I
Contribution

Chapitre 03

Étude de l'existant et analyse des besoins

3.1. Introduction

Ce chapitre entame une étude approfondie de l'existant au sein de Yalidine EL DJAZAIR SERVICES. En examinant leurs processus métiers actuels, notre analyse vise à identifier clairement la problématique centrale liée aux limites des outils en place. Cette démarche est essentielle pour définir avec précision les objectifs et les besoins spécifiques nécessaires au développement d'une nouvelle solution.

3.2. Présentation du domaine et de l'organisme d'accueil

3.2.1. Présentation générale de l'entreprise

3.2.1.1. Ourquilane



Figure 3.1 Logo de l'organisme d'accueil Ourquilane

Une startup dynamique spécialisée dans le développement de solutions logicielles novatrices. OURQUILANE s'engage à transformer et moderniser divers secteurs via des plateformes numériques, le marketing digital, et des outils d'aide à la décision. Parmi ses projets : HR Force (Gestion RH pour Yalidine), FLEET GO (gestion de flotte), ITOP (gestion IT), CASH Box (opérations financières), Route Planner (planification de tournées) et CS Care (gestion des réclamations). C'est l'entité porteuse de ce projet.

3.2.1.2. Organigramme d'OURQUILANE

La **Figure 3.2** illustre la structure organisationnelle d'Ourquilane. Pour ce projet, notre équipe, bien que localisée au sein du département de Développement Informatique, endosse les responsabilités de Data Scientists. Cette fonction transversale implique une collaboration essentielle avec les autres pôles (Recherche et Développement, Administration, Commercial) pour garantir l'atteinte des objectifs.

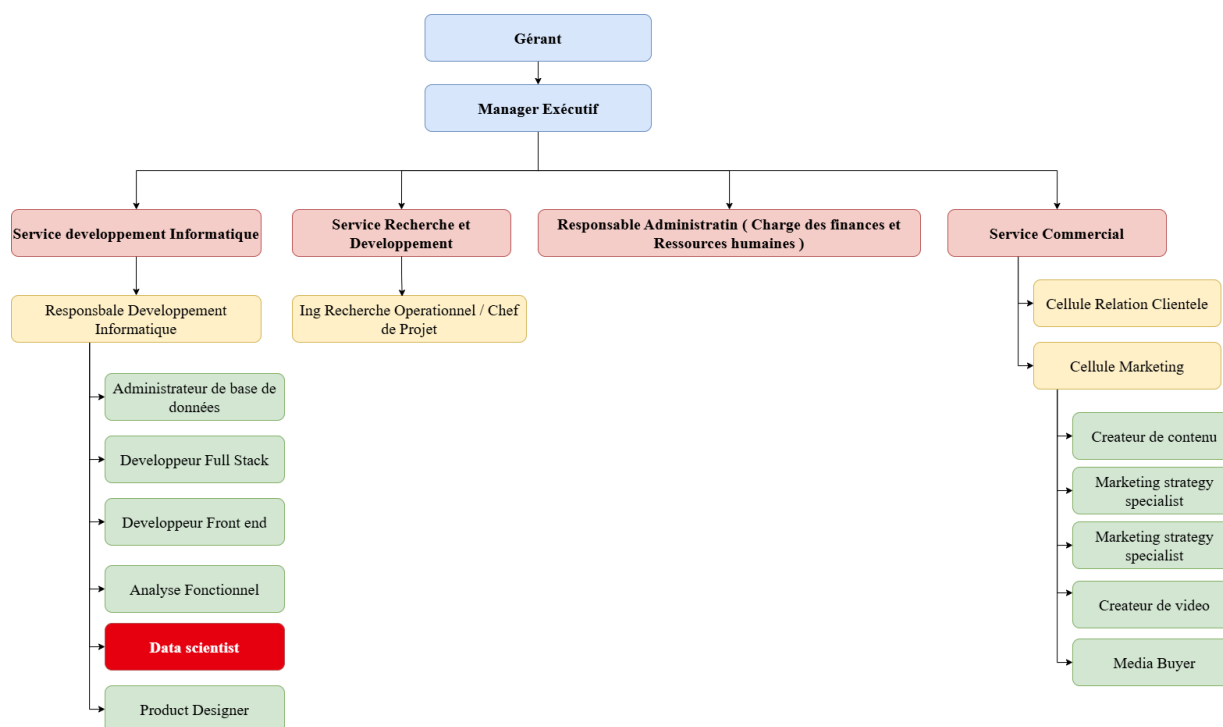


Figure 3.2 Organigramme de l'organisme d'accueil Ourquilane

3.2.1.3. Yalidine



Figure 3.3 Logo de l'organisme client Yalidine

YALIDINE EL DJAZAIR SERVICES est le leader de la livraison express en Algérie. L'entreprise assure une logistique nationale de pointe. Ses services incluent le courrier B2B, les livraisons e-commerce B2C et le recouvrement avec paiement à la livraison (COD : Cash on Delivery). Reconnue pour son suivi en temps réel qui garantit la sécurité et la rapidité des envois illustré dans la **Figure 3.4**, Yalidine s'engage pour l'excellence, comme en attestent ses certifications ISO 9001 (Qualité), ISO 14001 (Environnement) et ISO 27001 (Sécurité de l'information).

 [Accueil](#) [Se connecter](#) [Nos agences](#) [A propos](#) [Services](#) [Nos qualités](#) [Contact](#) [Devis / créer](#) [Suivre mon colis](#)

Le suivi du colis **yal-DNEB99**

Date du statut	Le statut	Localisation
dimanche 01 décembre à 08:37	Livré Actuel	El Bouni - Annaba
dimanche 01 décembre à 03:01	Centre	El Bouni - Annaba
dimanche 01 décembre à 00:36	Tentative échouée (Client no-show)	El Bouni - Annaba
dimanche 01 décembre à 00:36	Tentative échouée (Client no-show)	El Bouni - Annaba
samedi 30 novembre à 13:43	En attente du client	El Bouni - Annaba
samedi 30 novembre à 10:01	Centre	El Bouni - Annaba
samedi 30 novembre à 07:48	Transfert	Seraïdi - Annaba
samedi 30 novembre à 06:51	Centre	Seraïdi - Annaba
jeudi 28 novembre à 15:11	Vers Wilaya	Sétif - Sétif
jeudi 28 novembre à 14:33	Expédié	Sétif - Sétif
jeudi 28 novembre à 13:40	En préparation	

Figure 3.4 Exemple de suivi de colis en temps réel sur le site web de Yalidine.

3.3. Analyse des processus existants

Pour comprendre la prévision du nombre de colis, il est essentiel de détailler la structure opérationnelle de Yalidine. L'entreprise utilise une architecture logistique décentralisée et hiérarchisée à l'échelle nationale. Dans le cadre de nos missions de travail, nous avons visité le siège de Yalidine, qui intègre une station logistique, ainsi que d'autres stations. Ces visites nous ont permis de poser des questions et de mieux comprendre le processus de livraison, ainsi que l'impact réel de notre projet sur les clients.

3.3.1. Composantes clés de la chaîne logistique Yalidine

La chaîne logistique de Yalidine est organisée autour des entités suivantes, qui travaillent ensemble pour acheminer les colis à travers le pays :

- **Hub (Centre de Tri Principal)** : Ce sont les plateformes principales du réseau national. Ils reçoivent les colis de différentes provenances, effectuent un tri important, puis les redistribuent vers les Centres Régionaux. Yalidine possède quatre Hubs stratégiquement situés :
 - Oued Smar (Alger) : couvre le nord du pays.
 - Relizane : pour l'ouest.
 - Constantine : Pour l'est.
 - Ghardaïa : Couvre le Grand Sud.

- **Centre régional** : Chaque centre régional est une entité logistique qui gère plusieurs stations de livraison. Il reçoit les colis consolidés des Hubs et organise leur répartition vers les stations qui lui sont rattachées.
- **Centre (Station)** : C'est l'unité de proximité, responsable de la livraison du dernier kilomètre. Chaque station couvre un ensemble de communes et est le point d'attache unique de ses livreurs. Sa mission principale est de recevoir les colis du Centre Régional et d'organiser l'affectation précise des colis aux livreurs pour les tournées quotidiennes. C'est à ce niveau que la précision de la prévision des volumes est la plus cruciale pour notre projet. Yalidine dispose d'un réseau dense de 182 stations couvrant les 55 wilayas.
- **Région** : Ce terme désigne un regroupement géographique plus large, composé de plusieurs stations. Ces stations peuvent appartenir à différentes wilayas ou à la même wilaya.
- **Wilaya** : La wilaya est une division administrative en Algérie. Yalidine assure une couverture complète de toutes les 55 wilayas du territoire national.

3.3.2. Cheminement d'un Colis

Pour illustrer cette organisation, reprenons l'exemple d'une commande expédiée de Chéraga (Alger) vers la Station de Fernandville (Oran) :

1. **Origine (Chéraga - Station d'origine)** : Le colis est collecté et créé dans le système de livraison.
2. **Hub Nord (Oued Smar, Alger)** : Le colis transite par le Hub d'Oued Smar pour le tri et le regroupement par wilaya ou région.
3. **Transfert Inter-Hubs (Oued Smar vers Relizane)** : Le colis est acheminé du Hub d'Oued Smar vers le Hub de Relizane, qui dessert l'Ouest, ou directement vers le Centre Régional/station de destination
4. **Distribution Régionale (Hub de Relizane vers Centre Régional d'Oran)** : Du Hub de Relizane, le colis est envoyé au Centre régional d'Oran.
5. **Distribution Locale (Centre Régional d'Oran vers Station de Fernandville)** : Le Centre Régional d'Oran transfère le colis à la Station de Fernandville, sa zone de couverture finale.

6. **Livraison Finale (Station de Fernandville)** : À la Station de Fernandville, le colis est attribué à un livreur pour la livraison à domicile (*Home Desk*) ou mis à disposition pour un retrait en station (*Stop Desk*).

3.3.3. Flux Logistiques et Processus de Prévision

Le diagramme BPMN présenté en **Figure 3.5** illustre les étapes clés du cheminement d'un colis au sein de la chaîne logistique de Yalidine, depuis la demande initiale jusqu'à sa livraison finale ou son retour.

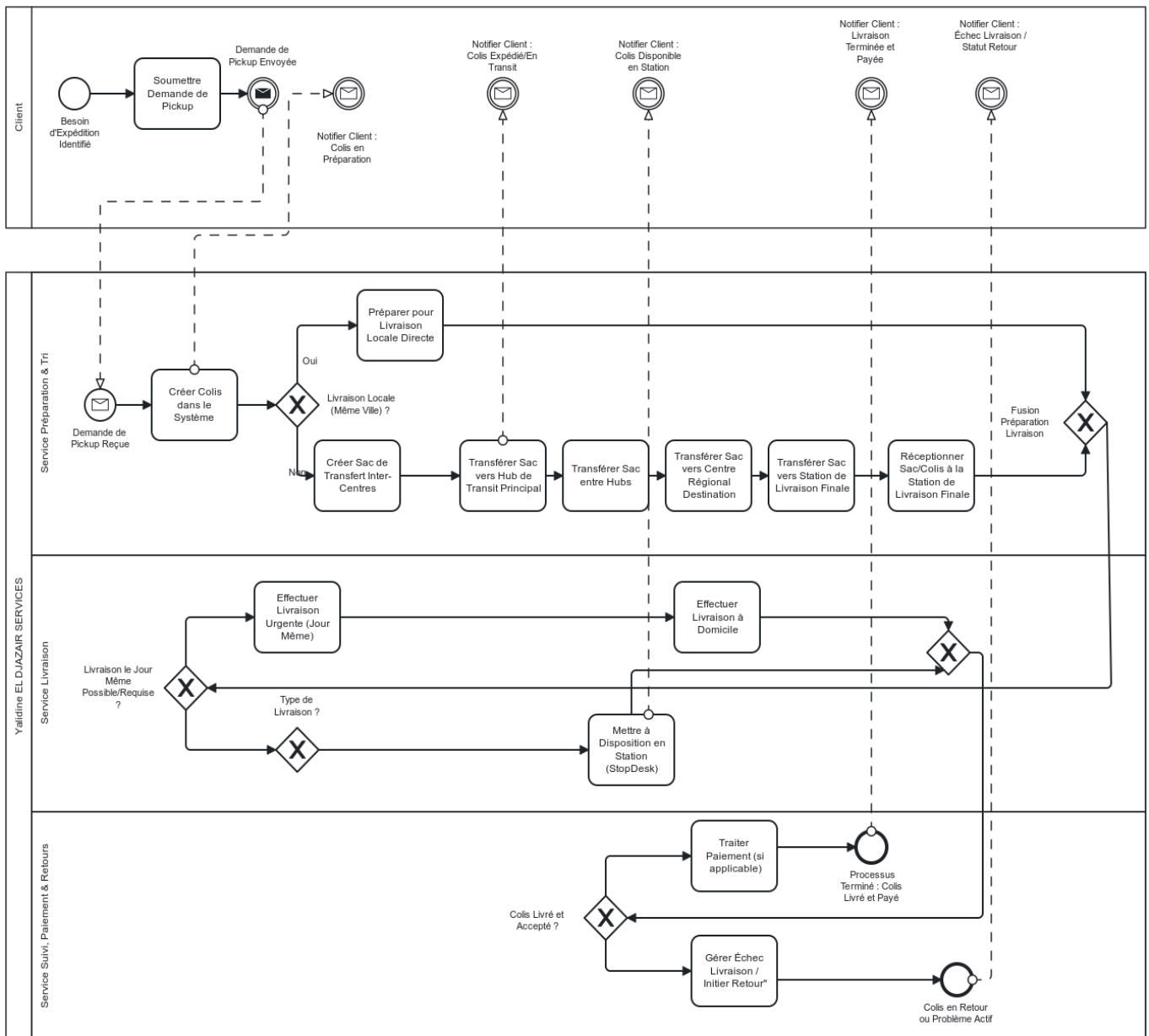


Figure 3.5 Diagramme BPMN du processus de livraison actuel de Yalidine

3.3.4. Cycle de vie d'un colis chez Yalidine

Le cycle de vie d'un colis chez Yalidine, illustré dans la **Figure 3.6**, est marqué par plusieurs changements. Chaque étape d'une livraison d'un colis déclenche une mise à jour de son statut. Ces différents statuts sont détaillés dans le **Tableau 3.1** ci-dessous :

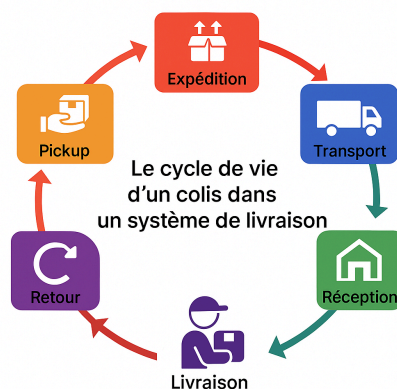


Figure 3.6 Cycle de vie d'un colis chez Yalidine

Statut	Description
En préparation	Le colis est enregistré et prêt pour l'expédition.
Expédié	Le colis a été pris en charge par Yalidine.
Vers Wilaya	Le colis est en route vers la wilaya de destination.
Centre	Le colis est arrivé ou transite dans un centre de tri ou de distribution.
Transfert	Le colis est en cours de transfert entre deux centres ou hubs.
En attente du client	Le colis est dans l'attente d'une action du client ou d'une tentative de livraison.
Tentative échouée (Client no-show)	Une tentative de livraison a eu lieu mais le client n'était pas disponible.
Retour	Le colis est en cours de retour à l'expéditeur suite à un échec de livraison définitif (wilaya d'expédition)
Livré	Le colis a été remis au destinataire final.

Tableau 3.1 : Statuts des colis et leurs descriptions

Cette chronologie et la traçabilité des mises à jour de statuts des colis forment une base de données essentielle pour notre projet. Nous l'utiliserons comme source de données dans la phase de conception, en nous concentrant particulièrement sur le statut "*Centre*" (*entrée du colis dans la station de destination*).

3.4. Problématique

Yalidine fait face à une sous-estimation chronique du nombre de livreurs nécessaires par station (182 à l'échelle nationale), faute de visibilité sur le volume quotidien exact des colis entrants. Cette lacune impacte directement la planification et la mobilisation des livreurs, majoritairement indépendants. Lors des pics de demande, où le volume par livreur peut doubler (70-80 colis contre 35-40 habituellement), la situation devient critique. Paradoxalement, ces pics de demande, qui devraient être synonymes de croissance du chiffre d'affaires, se transforment en une incapacité à gérer le volume, entraînant retards, dégradation du service et insatisfaction client.

La question centrale de ce projet est donc : Comment prédire précisément les demandes de livraison afin d'améliorer la prise de décision et l'efficacité logistique de Yalidine dans la gestion de ses ressources, notamment de ses livreurs ?

3.5. Objectifs finaux du système cible

L'analyse approfondie des besoins a permis de définir des objectifs clairs et mesurables pour améliorer la prise de décision concernant la planification des livreurs.

- **Amélioration de la prévision des volumes de colis**
 - Mettre en place des modèles de prévision hautement précis pour anticiper les volumes de colis entrants.
 - Minimiser les écarts entre les prévisions et la réalité opérationnelle pour une planification plus fiable.
- **Optimisation de l'affectation des livreurs et des ressources**
 - Automatiser l'assignation des livreurs aux zones de livraison en fonction des volumes prévus et des contraintes opérationnelles.
 - Répartir efficacement les ressources humaines et matérielles pour éviter la surcharge de travail, améliorer la productivité et réduire les délais de livraison.
- **Centralisation et accessibilité des données.**
 - Fournir une visibilité complète des performances opérationnelles grâce à des tableaux de bord détaillés et intuitifs.

3.6. Exigences Techniques et Ressources

Pour concrétiser ces objectifs et besoins, notre solution est conçue pour être exécutée localement, directement sur les postes de travail d'OURQUILANE. Voici ce qu'il faudra prévoir :

- **Ordinateur Performant** : Un PC (*Windows, macOS ou Linux*) avec au moins 16 Go de RAM et un processeur puissant. Un GPU (*carte graphique*) est un grand atout pour accélérer les calculs des modèles de prévision si la charge l'exige.
- **Environnement Python** : La version de Python que nous spécifions doit être installée. Toutes les bibliothèques Python nécessaires seront facilement installables via un fichier (*requirements.txt*) simple.

3.7. Parties prenantes du projet

La réussite de ce projet de système de prévision de colis repose sur une collaboration efficace et une compréhension mutuelle des attentes de toutes les parties prenantes. Nous les catégorisant comme suit dans le **Tableau 3.2** :

Acteur	Rôle clé	Impact principal du projet
Responsables Logistiques	Superviser les opérations logistiques (station/centre/hub).	meilleure gestion des flux et des ressources, prise de décision agile.
Data scientists	Générer, recevoir et analyser les prévisions de volume issues de la solution.	Automatisation de la génération des prévisions, amélioration de leur fiabilité, gain de temps significatif.
Direction Générale	Décision stratégique et suivi de la performance globale.	Visibilité sur la performance globale, aide à l'optimisation des coûts et à la définition de la stratégie.
Livreurs	Exécuter les livraisons jusqu'au client final (livraison "Home Desk").	Itinéraires optimisés, suivi facilité, meilleure répartition des tâches, respect accru des délais.
Département IT	Assurer l'infrastructure technique, le support et la sécurité.	Simplification (à terme) via un système unifié, sécurité des données.

Resp. Conformité/Juridique	Veiller à la conformité des processus logistiques et de la gestion des données (Loi 18-07).	Outils et processus intégrés facilitant le respect des réglementations et la réduction des risques de non-conformité.
-----------------------------------	---	---

Tableau 3.2: Rôles des acteurs et impacts attendus du projet

3.8. Gestion des prévisions existante

Chez Yalidine EL DJAZAIR SERVICES, la gestion des prévisions s'appuie sur une solution externe, **Tableau Cloud** (Figure 3.7). Pour comprendre comment les prévisions du nombre de colis à livrer sont générées, nous allons détailler l'utilisation de cette solution par Ourquilane.



Figure 3.7 Logo de Tableau Cloud

Le Diagramme BPMN (Figure 3.8) décrit le processus actuel de prévision du nombre de colis.

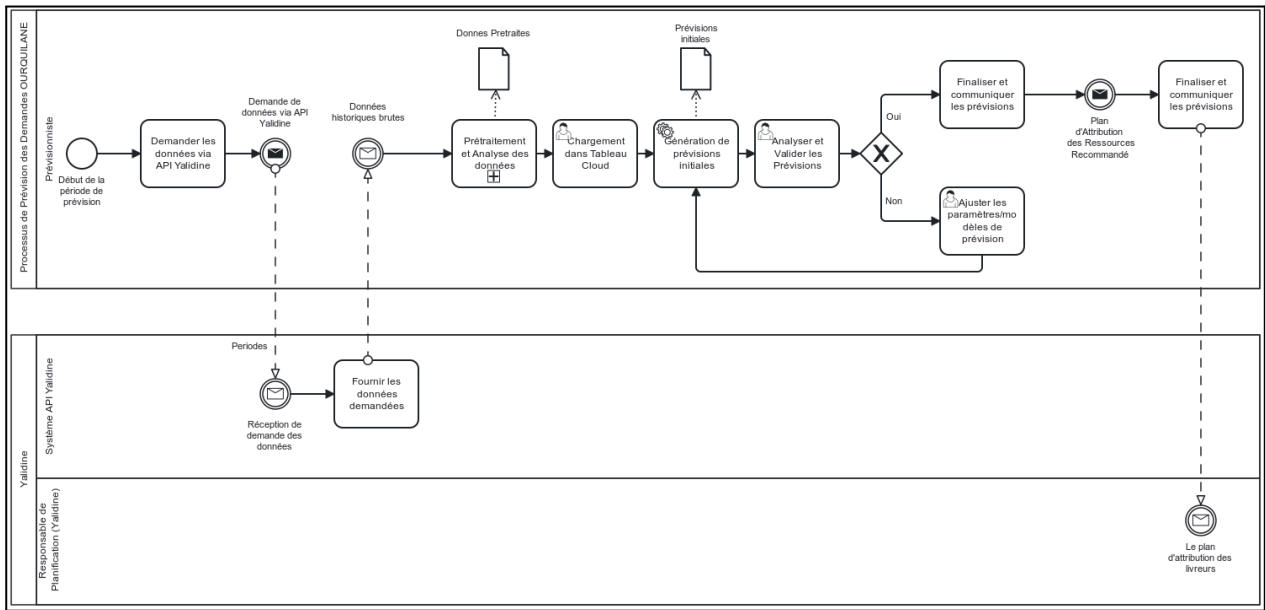
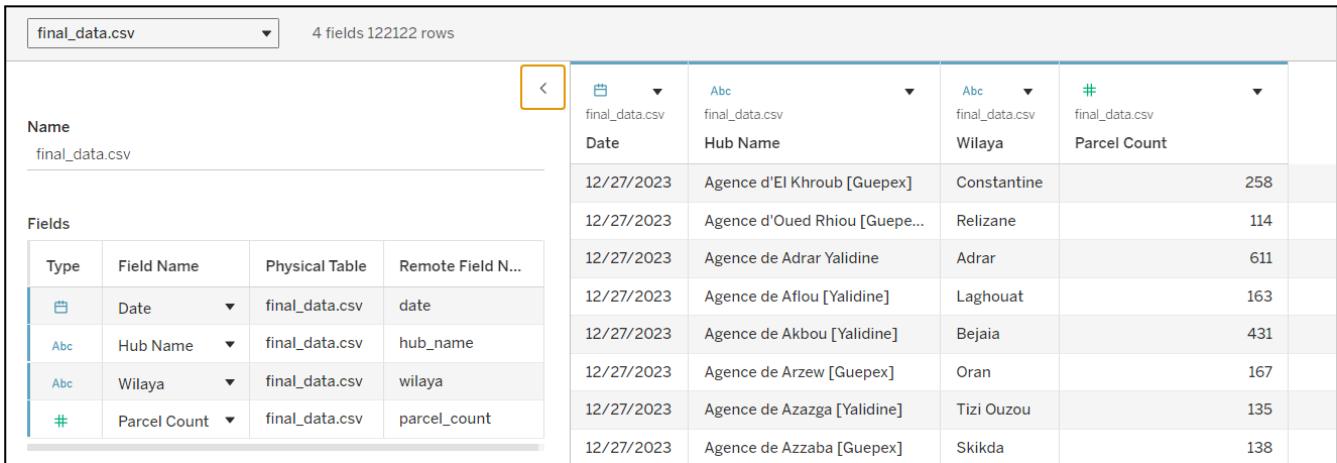


Figure 3.8 Diagramme BPMN du processus actuel de prévision du nombre de colis

Ce processus consiste à collecter et importer les données, puis à les traiter et les visualiser dans Tableau Cloud pour générer des prévisions. Ces prévisions sont régulièrement vérifiées et ajustées pour s'adapter à la réalité opérationnelle. Nous détaillerons ci-après chaque étape de ce dernier.

3.8.1.1. Importation des données

Les informations historiques nécessaires à la prévision sont collectées directement via l'API de Yalidine. Ces données vitales, qui décrivent les changements de statut des colis (comme vu précédemment), incluent les historiques de livraisons. Elles subissent ensuite des prétraitements (dont les modalités exactes ne sont pas détaillées par l'équipe d'Ourquilane) avant d'être importées. Ce processus est illustré dans la (Figure 3.9).



The screenshot shows the Tableau Cloud interface for importing data from a CSV file named 'final_data.csv'. The interface indicates that there are 4 fields and 122,122 rows of data. On the left, a 'Fields' pane lists the imported fields: Date, Hub Name, Wilaya, and Parcel Count, each with a corresponding icon and data type. On the right, a preview of the data table is shown, displaying columns for Date, Hub Name, Wilaya, and Parcel Count, with several rows of data.

Type	Field Name	Physical Table	Remote Field N...
Date	Date	final_data.csv	date
Hub Name	Hub Name	final_data.csv	hub_name
Wilaya	Wilaya	final_data.csv	wilaya
Parcel Count	Parcel Count	final_data.csv	parcel_count

Date	Hub Name	Wilaya	Parcel Count
12/27/2023	Agence d'El Khroub [Guepex]	Constantine	258
12/27/2023	Agence d'Oued Rhiou [Guepe...	Relizane	114
12/27/2023	Agence de Adrar Yalidine	Adrar	611
12/27/2023	Agence de Aflou [Yalidine]	Laghouat	163
12/27/2023	Agence de Akbou [Yalidine]	Bejaia	431
12/27/2023	Agence de Arzew [Guepex]	Oran	167
12/27/2023	Agence de Azazga [Yalidine]	Tizi Ouzou	135
12/27/2023	Agence de Azzaba [Guepex]	Skikda	138

Figure 3.9 Exemple d'importation des données de livraison sur Tableau Cloud

3.8.1.2. Traitement et visualisation des données

Une fois les données chargées dans Tableau Cloud, l'outil génère des tableaux de bord interactifs. Ces visualisations, qui utilisent divers types de graphiques (barres, cercles, etc.), sont essentielles pour les prévisionnistes. Elles leur permettent de mieux comprendre les données, d'analyser les tendances passées, de saisir les fluctuations et les variations saisonnières, et d'identifier les pics d'activité. Ce processus est illustré dans la (Figure 3.10)

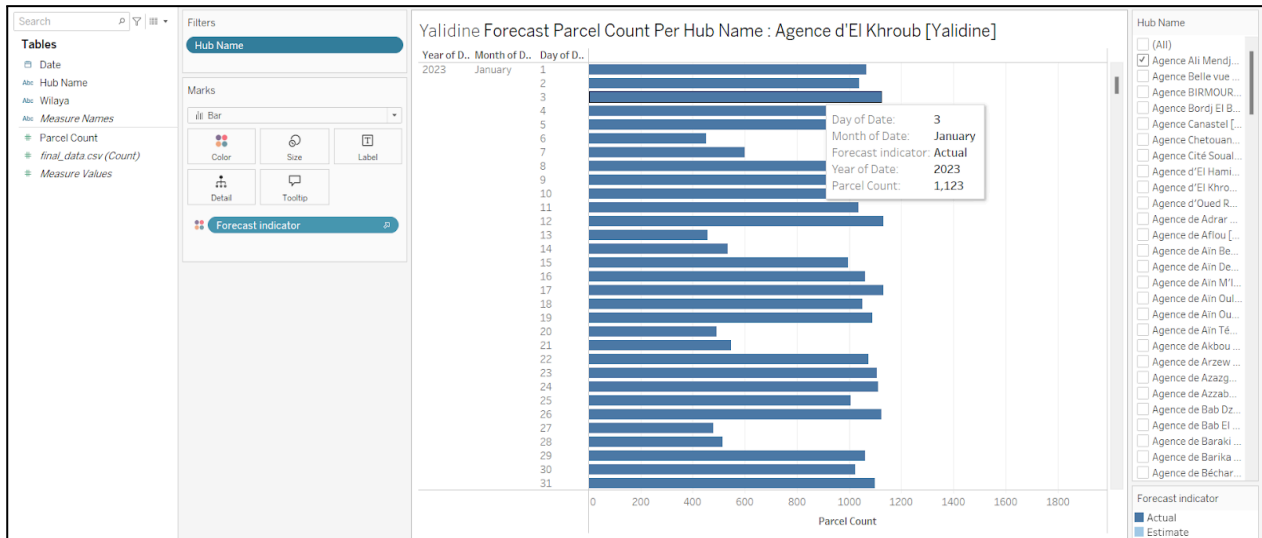


Figure 3.10 Exemple de visualisation des données sur Tableau Cloud.

3.8.1.3. La génération des prévisions et évaluation

Après la phase de visualisation et compréhension des données, le processus passe à la génération des prévisions. Pour cela, la variable cible est explicitement sélectionnée, qui est le nombre de colis par jour. L'outil génère alors une prévision basée sur les données disponibles et les modèles intégrés.

Deux exemples de prévisions générées sont présentés ci-dessous **Figure 3.11** et **Figure 3.12**, illustrant la capacité de l'outil à estimer le volume de colis pour différentes agences.

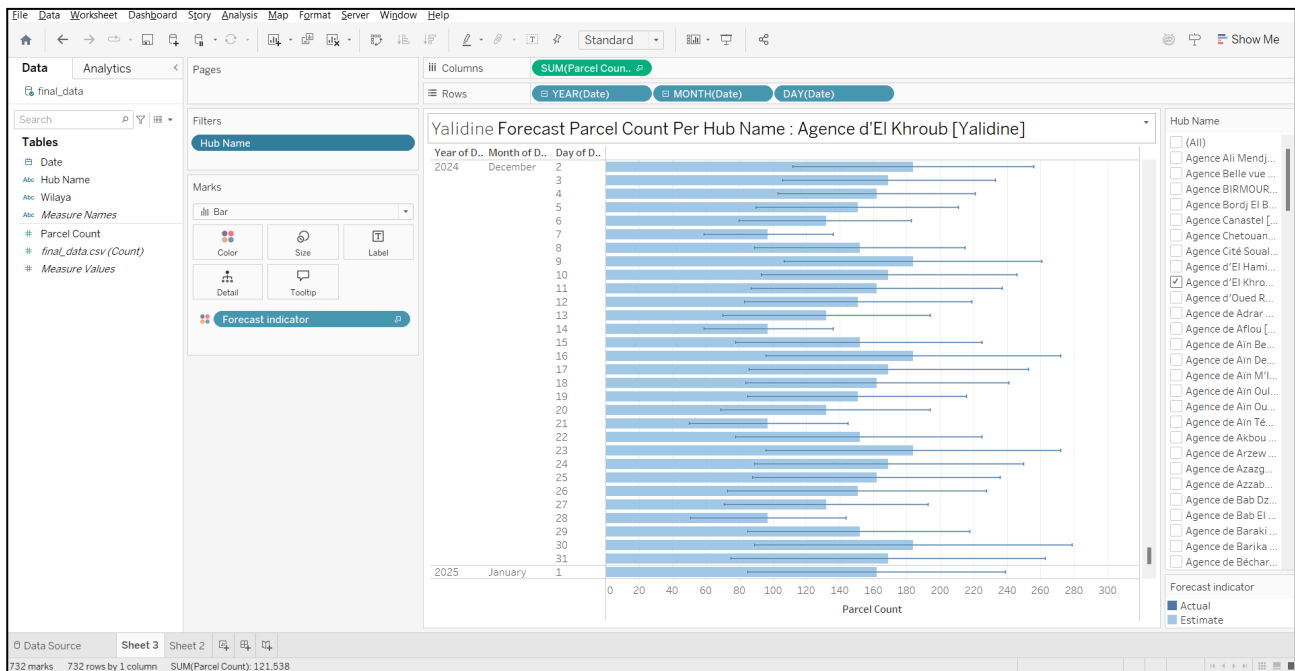


Figure 3.11 Exemple de prévision pour l'agence d'El Khroub sur Tableau cloud

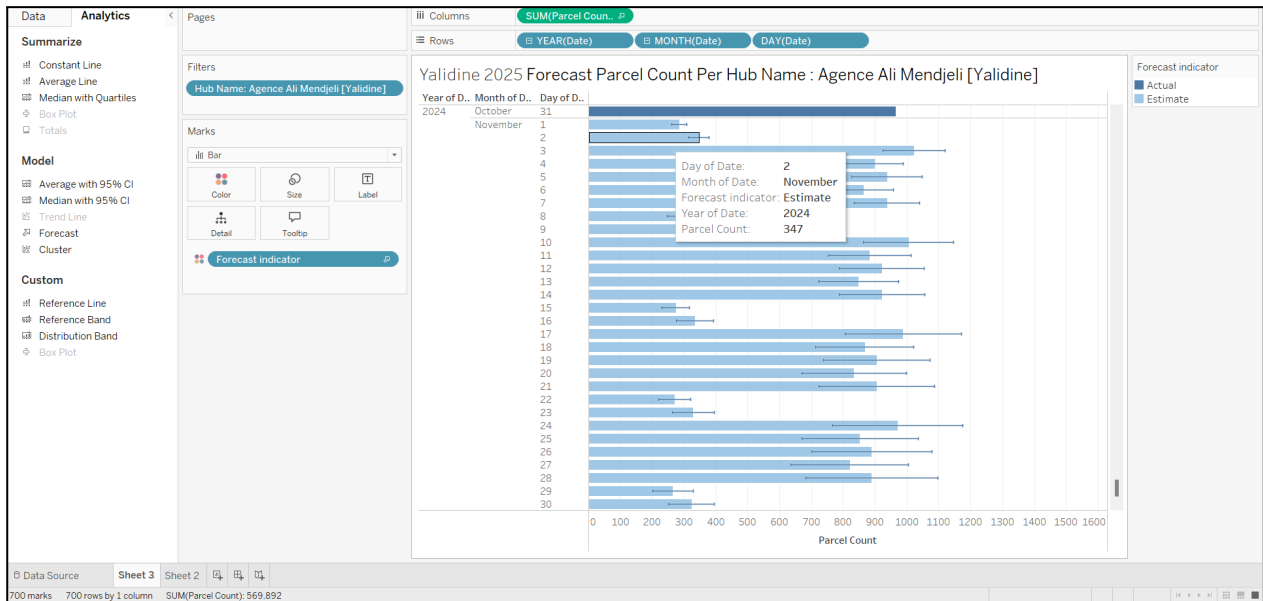


Figure 3.12 Exemple de prévision pour l'agence Ali Mendjeli sur Tableau cloud

Un point notable dans ces prévisions est la diminution significative des volumes de colis sur deux jours consécutifs, correspondant généralement au week-end. Cela s'explique par le fait que Yalidine n'effectue pas d'opérations de livraison durant ces jours, ce qui crée un "creux" évident dans les données et les prévisions

3.8.1.4. Analyse et Évaluation

Cette section synthétise les observations clés issues de l'analyse du forecast généré par le modèle actuel (dont les détails seront abordés ultérieurement), Pour évaluer la qualité et comprendre la structure de la prévision générée, nous nous appuyons sur les informations détaillées présentées dans la **Figure 3.13** (résumé de la prévision) et la **Figure 3.14** (détails du modèle et métriques).



Figure 3.13 Options de génération de la prévision avec Tableau cloud

La **Figure 3.13** explique les réglages de la prévision. Elle montre que le modèle a été entraîné sur des données allant du 1er janvier 2023 au 3 juillet 2024, puis a utilisé les données du 4 juillet au 31 octobre 2024 pour établir la base de la prévision. La prévision est ensuite générée pour un horizon de 62 jours, s'étendant jusqu'au 1er janvier 2025, et tient compte d'un cycle de 7 jours (*la semaine*). Ce cycle est important pour prévoir les variations hebdomadaires, comme la diminution des volumes le week-end.

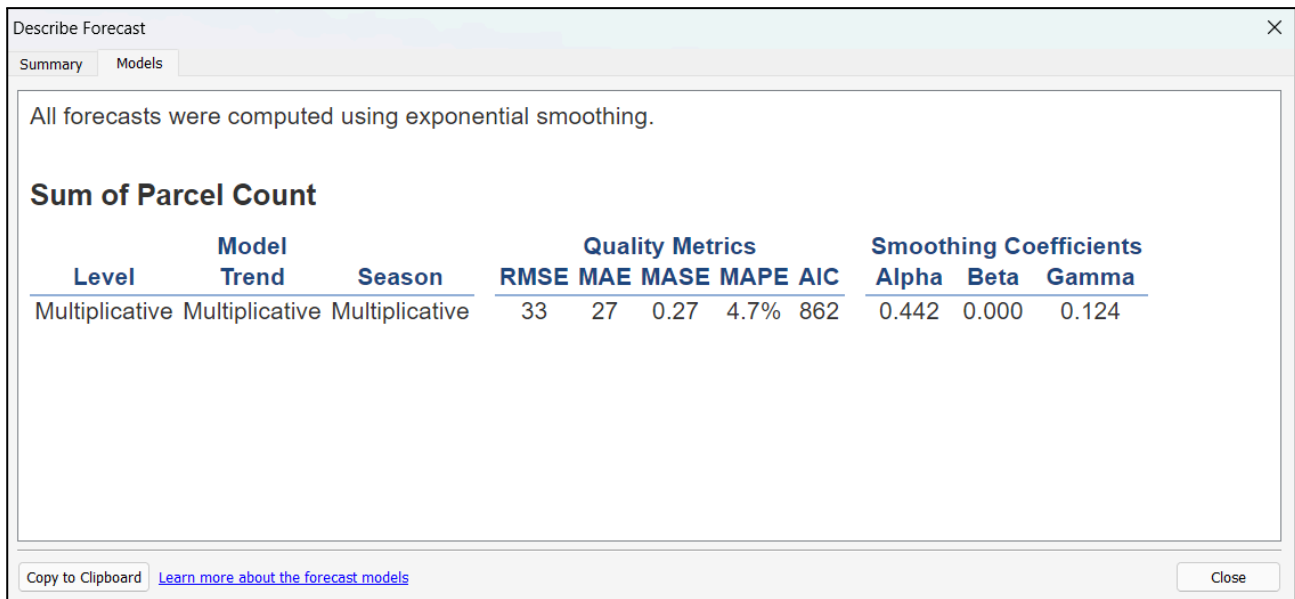


Figure 3.14 Modèle de prévision et métriques de qualité

La **Figure 3.14** montre que le modèle de prévision utilise une méthode de lissage. Elle indique que la prévision est assez précise, avec une erreur moyenne de 4.7% (MAPE). De plus, on voit que la saisonnalité (le fait que les volumes changent selon les jours de la semaine) est le facteur le plus important pour cette prévision, bien plus qu'une tendance générale.

3.8.1.5. Attribution des ressources

L'attribution des livreurs est une étape importante qui utilise les prévisions de colis.

Une fois les prévisions établies par le système et communiquées aux chefs de station, chaque dirigeant dispose d'une estimation claire du nombre de colis à recevoir. Le chef de station planifie alors et organise la répartition des colis entre ses livreurs.

Cette répartition respecte une norme opérationnelle clé : chaque livreur gère environ 35 à 40 colis par jour. Cette approche permet d'anticiper la charge de travail et d'ajuster les effectifs pour garantir l'efficacité des livraisons.

3.9. Diagnostic opérationnel et impact sur le processus de livraison.

Le système de prévision actuel de l'entreprise, bien que fonctionnel, présente des contraintes et des limites significatives qui ont un impact direct et négatif sur l'efficacité du processus de livraison de Yalidine.

3.9.1.1. Contraintes et Limites du Système Actuel

Nous allons ici présenter les principales lacunes du système actuel :

- **Dépendance à un outil externe:** L'usage de Tableau Cloud implique une dépendance à un prestataire extérieur, limitant la personnalisation et l'agilité face aux évolutions internes ou du marché.
- **Risque de non-conformité réglementaire :** Le stockage des données à l'étranger via Tableau Cloud pose des risques de non-conformité avec la législation algérienne (*Loi n° 18-07 relative à la protection des données personnelles*).
- **Coûts élevés :** L'abonnement onéreux à Tableau Cloud (*environ 115 \$ par mois et par utilisateur*), représente une lourde charge financière et limite l'accès à l'outil pour l'ensemble du personnel concerné.

3.9.1.2. Problèmes Opérationnels et Stratégiques

Ces limites et contraintes engendrent plusieurs problématiques opérationnelles et stratégiques, affectant directement la performance de livraison de Yalidine :

- **Difficultés d'adaptation aux variations de la demande :** Le système peine à s'adapter rapidement aux pics soudains (*ex: Ramadan*). Cela crée un déséquilibre critique entre colis et livreurs, entraînant retards de livraison, surcharge des livreurs et insatisfaction client.
- **Manque de flexibilité et de personnalisation :** L'impossibilité d'adapter les modèles ou d'intégrer de nouvelles données (*événements locaux*) rend les prévisions moins précises. Les livraisons sont moins optimisées, augmentant les coûts opérationnels et réduisant l'efficacité.
- **Risque de non-conformité réglementaire :** Le non-respect de la loi algérienne 18-07 peut entraîner des sanctions légales et nuire à la réputation de l'entreprise, affectant indirectement la confiance des clients.
- **Freins à l'expansion et à l'efficacité :** Le coût élevé limite l'adoption de la solution à l'échelle du réseau Yalidine. Cela empêche une optimisation générale des livraisons, maintenant des inefficacités coûteuses dans les stations non couvertes par l'outil.

3.10. Recensement des Besoins Client

Le recensement des besoins a été une étape fondamentale de notre démarche. Il a été mené de manière collaborative, notamment à travers des réunions de travail en format JAD (*Joint Application Design* – une méthode de travail collaboratif qui impliquent plusieurs rôles tels que Le Facilitateur (*Animateur*), Le Sponsor, Le Scribe (*Modélisateur*), les Participants Métiers (*Experts du domaine*)) avec des responsables logistiques de Yalidine ainsi que les prévisionnistes d'OURQUILANE.

Cette démarche nous a permis de confirmer les lacunes actuelles de la solution existante et d'exprimer les attentes des clients vis-à-vis d'une future solution. Les principaux besoins identifiés sont les suivants :

- **Automatisation des prévisions** : Mettre fin à la mise à jour manuelle fastidieuse des fichiers de prévisions et optimiser l'attribution des livreurs pour une meilleure efficacité.
- **Conformité réglementaire** : Assurer que le stockage et le traitement des données personnelles respectent strictement la loi algérienne 18-07 sur la protection des données.
- **Réduction des coûts** : Trouver une alternative à Tableau Cloud, dont les abonnements sont jugés coûteux.
- **Précision et flexibilité du système de prévisions** : Développer un système intégrant des modèles adaptés aux spécificités locales et aux variations de la demande, offrant une grande précision.

3.11. Conclusion

Ce chapitre a été consacré à l'analyse des processus logistiques de Yalidine. Nous avons identifié les problématiques clés de l'entreprise et évalué la solution de prévision existante (Tableau Cloud), en mettant en lumière ses limites, notamment en termes de coûts et de conformité réglementaire. Ce diagnostic a permis de définir des objectifs clairs pour un nouveau système, dont la conception sera détaillée dans le chapitre suivant.

Chapitre 04

Conception

4.1. Introduction

Ce chapitre présente la conception du système développé. S'appuyant sur les besoins précédemment identifiés, elle décrira la solution mise en place et justifiera les choix de conception qui la sous-tendent, afin d'assurer son efficacité et son adaptation aux spécificités du contexte logistique de Yalidine.

4.2. Méthodologie de travail

Pour organiser notre travail et avancer de manière structurée, on a choisi de suivre la méthodologie *CRISP-DM* “*Cross-Industry Standard Process for Data Mining*”. Ce modèle, comme illustré dans la **Figure 4.1**, est particulièrement adapté à notre problématique. Il nous permet de couvrir l'ensemble du cycle de vie du projet, de la compréhension du besoin métier jusqu'à la phase de déploiement. Grâce à sa logique par étapes et sa flexibilité entre les phases, nous pourrons traiter efficacement les données et construire des modèles prédictifs parfaitement adaptés à la réalité terrain de Yalidine.

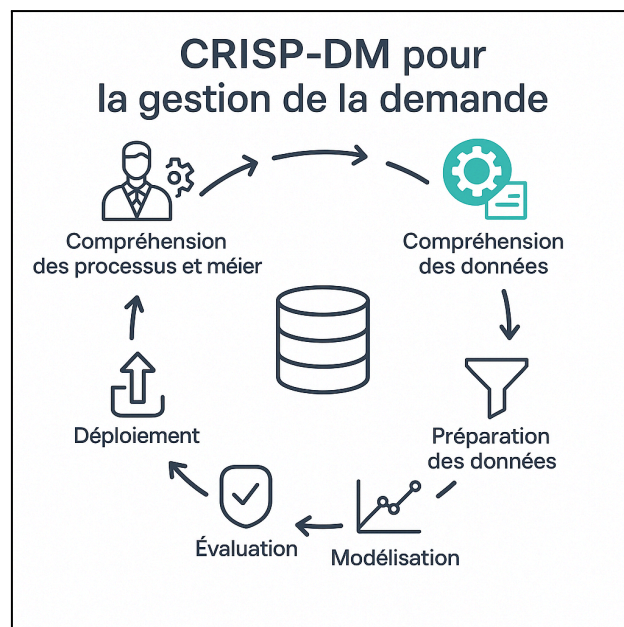


Figure 4.1 Méthodologie CRISP-DM

4.2.1. Compréhension du métier

Pour bien cerner les enjeux, nous avons d'abord essayé de comprendre les besoins réels de Yalidine ainsi que leurs processus logistiques, notamment celui de livraison et leur défi majeur (le manque de visibilité sur le nombre de colis pour chaque station).

4.2.2. Compréhension des données

Il s'agit de comprendre les données mises à notre disposition. Cette étape nous a aidé à savoir quelles variables sont utiles, et à anticiper le prétraitement à faire plus tard.

4.2.3. Préparation des données

Cette étape constitue l'une des phases les plus longues de la méthode. Le nettoyage des données ainsi que l'enrichissement avec de nouvelles caractéristiques sont réalisés dans cette étape.

4.2.4. Modélisation

La phase de modélisation consiste à sélectionner, paramétrer et entraîner différents algorithmes de prédiction en appliquant des techniques d'apprentissage automatique, optimisant leurs hyperparamètres et évaluant comparativement leurs performances pour identifier les modèles les plus adaptés au contexte logistique.

4.2.5. Évaluation

La phase d'évaluation consiste à comparer les modèles entraînés en utilisant la métrique de l'erreur quadratique moyenne (*MSE*) pour mesurer leur précision et sélectionner le plus performant.

4.2.6. Déploiement

Le système sera conçu pour être exportable et réutilisable, notre système intégrera un mécanisme de détection de concept drift pour s'adapter aux évolutions constantes. Face à l'absence d'infrastructure serveur chez Yalidine, la solution est développée pour être **utilisée directement par un Data Scientist**. Ce choix garantit la conformité, la confidentialité des données et la génération experte des prévisions.

4.3. Architecture Globale

La solution que nous proposons de développer dans ce projet pour répondre aux besoins de Yalidine reposera sur une architecture modulaire intelligente structurée en quatre modules principaux qui coopéreront pour créer un système de prévision optimal dédié à l'anticipation des demandes de livraison. Il est illustré dans la **Figure 4.2**.

Module de collecte et traitement des données

Ce module intégrera les données historiques de colis enrichies par des informations contextuelles (*météorologiques, événementielles*) et assurera leur prétraitement rigoureux, incluant le nettoyage et le "feature engineering", afin de garantir une qualité optimale des données d'entrée.

Module d'entraînement et d'apprentissage

Dans ce module, cinq modèles distincts (*ARIMA, XGBoost, LSTM, MLP, MLP Sul*) seront entraînés pour extraire les patterns temporels complexes, l'innovation majeure résidant dans un algorithme de sélection automatique qui optimisera leurs hyperparamètres, évaluera leurs performances et choisira le modèle optimal de manière adaptative pour chaque station.

Module de prédiction et visualisation (Statistiques)

Ce module générera les prévisions de demande à partir des modèles sélectionnés et fournira des outils d'analyse comparative de leurs performances pour toutes les stations.

Module de détection du concept Drift

Le dernier module surveillera en continu l'évolution des patterns de données pour identifier les changements significatifs nécessitant une adaptation ou un réentraînement des modèles.

Le choix d'une approche de prévision granulaire, station par station, plutôt qu'une approche globale, est une justification clé de notre architecture détaillée. Cette stratégie est essentielle pour capturer les spécificités locales de chacune des 182 stations du réseau Yalidine, qui évoluent dans des contextes démographiques, économiques et événementiels très divers. En permettant une adaptation algorithmique personnalisée, cette approche maximise la précision des prédictions pour chaque site, tout en garantissant une architecture cohérente et scalable pour l'ensemble du réseau logistique.

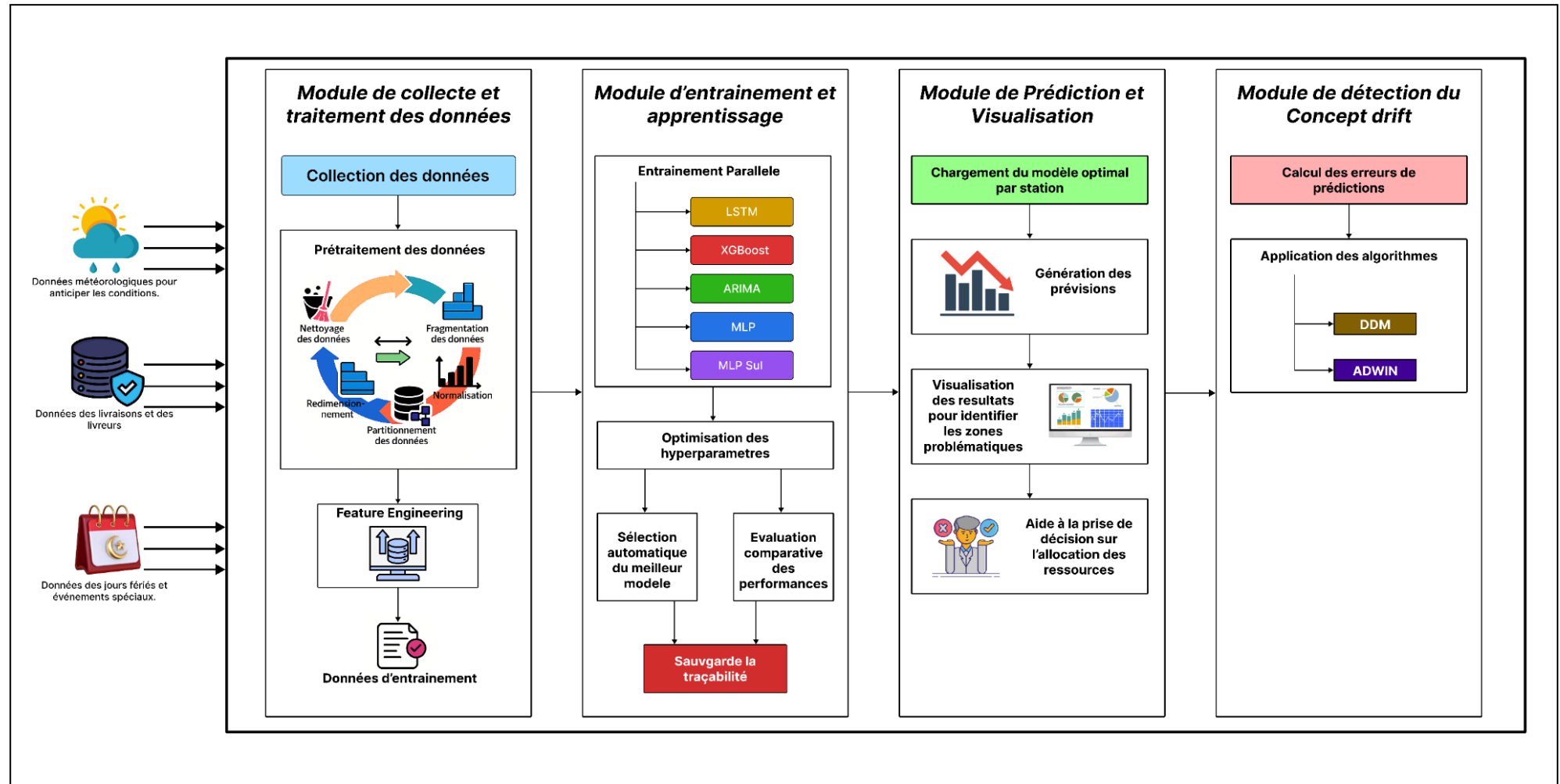


Figure 4.2 Architecture globale du système de prévision des volumes de livraison

4.4. Architecture Détaillée

Le but de cette étape est de bien comprendre comment chaque composant fonctionne en représentant de manière formelle les différentes tâches.

4.4.1. Module de collecte et traitement des données

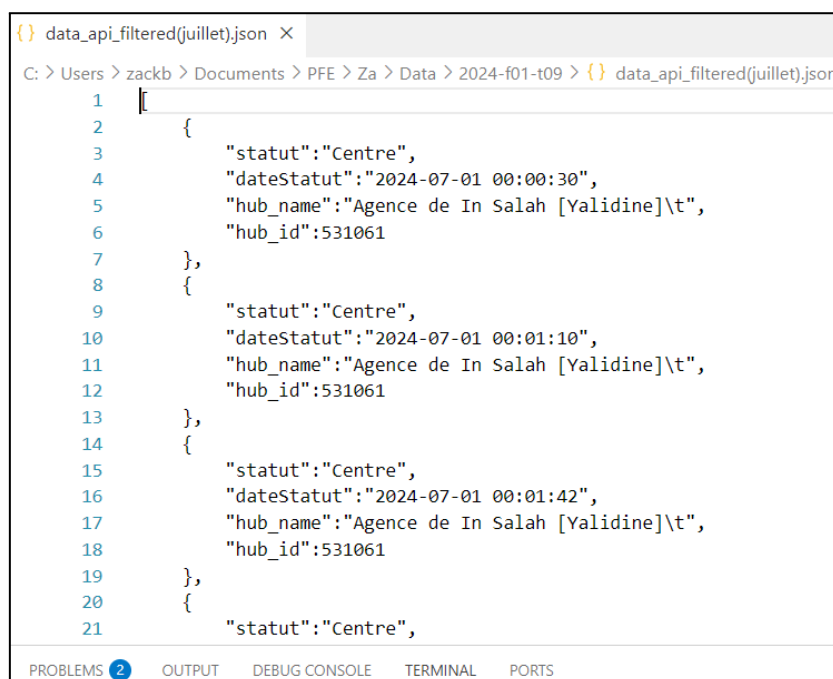
Dans cette section, nous allons comprendre les données brutes et détailler le processus qui les transformera en un format prêt pour nos modèles de prédiction.

4.4.1.1. Description des données

La qualité et la structure des données sont cruciales pour la fiabilité de toute prédiction. Cette section vise à comprendre en profondeur les données disponibles et à les préparer rigoureusement via une phase de transformation, afin qu'elles soient parfaitement exploitables pour nos modèles.

Notre principale source de données est l'API interne de Yalidine, qui enregistre mensuellement l'état de chaque colis sous forme de fichiers JSON. Pour garantir la conformité et la confidentialité, l'équipe technique de Yalidine a spécifiquement préparé ces données. Ils nous ont fourni une API dédiée contenant uniquement les informations relatives aux colis et à leurs statuts, sans aucune donnée personnelle. Cette démarche assure le respect strict de la Loi 18-07 sur la protection des données personnelles et des exigences de la certification ISO 27001 en matière de sécurité de l'information.

La **Figure 4.3** présente un exemple de ces données au format JSON pour le mois de juillet 2024.



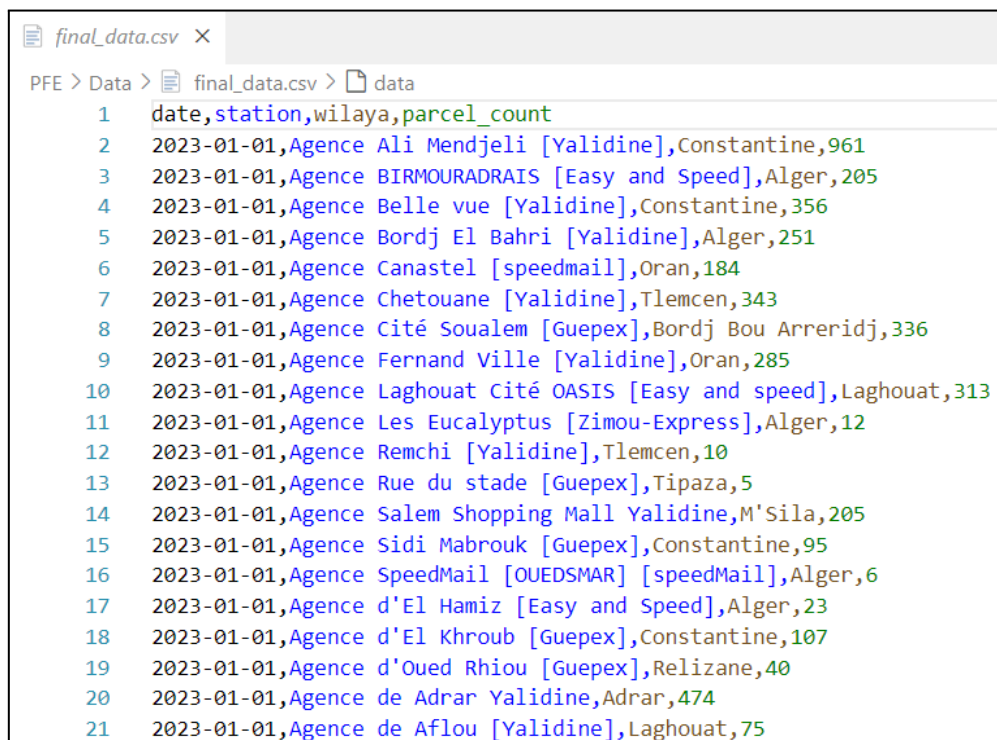
```
{
  "statut": "Centre",
  "dateStatut": "2024-07-01 00:00:30",
  "hub_name": "Agence de In Salah [Yalidine]\\t",
  "hub_id": 531061
},
{
  "statut": "Centre",
  "dateStatut": "2024-07-01 00:01:10",
  "hub_name": "Agence de In Salah [Yalidine]\\t",
  "hub_id": 531061
},
{
  "statut": "Centre",
  "dateStatut": "2024-07-01 00:01:42",
  "hub_name": "Agence de In Salah [Yalidine]\\t",
  "hub_id": 531061
},
{
  "statut": "Centre",
```

Figure 4.3 Exemple de représentation des données brutes depuis l'API sous format JSON

Comme mentionné dans le chapitre précédent (Etude de l'existant), notre analyse se concentre uniquement sur les colis ayant le statut "centre", cette focalisation est essentielle car elle marque l'arrivée des colis en station, prêts pour la livraison. En conséquence, un filtre est appliqué pour ne retenir que les lignes de données correspondant à ce statut.

Bien que les données soient initialement en JSON, elles sont converties au format CSV. Ce choix est motivé par la compatibilité et l'efficacité, car la plupart des bibliothèques de traitement de données et d'apprentissage automatique sont optimisées pour travailler avec les fichiers CSV.

Nous allons transformer ces événements bruts en une table agrégée par : Station logistique "hub_name" et Jour calendaire "dateStatut". Chaque ligne représentera alors une unité de prévision : le nombre de colis livrés ou traités dans une station donnée à une date donnée "parcel_count". Ce format tabulaire sera exporté en CSV. Un exemple de la structure finale obtenue est présenté ci-dessous dans la **Figure 4.4**:



1	date,station,wilaya,parcel_count
2	2023-01-01,Agence Ali Mendjeli [Validine],Constantine,961
3	2023-01-01,Agence BIRMOURADRAIS [Easy and Speed],Alger,205
4	2023-01-01,Agence Belle vue [Validine],Constantine,356
5	2023-01-01,Agence Bordj El Bahri [Validine],Alger,251
6	2023-01-01,Agence Canastel [speedmail],Oran,184
7	2023-01-01,Agence Chetouane [Validine],Tlemcen,343
8	2023-01-01,Agence Cité Soualem [Guepex],Bordj Bou Arreridj,336
9	2023-01-01,Agence Fernand Ville [Validine],Oran,285
10	2023-01-01,Agence Laghouat Cité OASIS [Easy and speed],Laghouat,313
11	2023-01-01,Agence Les Eucalyptus [Zimou-Express],Alger,12
12	2023-01-01,Agence Remchi [Validine],Tlemcen,10
13	2023-01-01,Agence Rue du stade [Guepex],Tipaza,5
14	2023-01-01,Agence Salem Shopping Mall Validine,M'Sila,205
15	2023-01-01,Agence Sidi Mabrouk [Guepex],Constantine,95
16	2023-01-01,Agence SpeedMail [OUEDSMAR] [speedMail],Alger,6
17	2023-01-01,Agence d'El Hamiz [Easy and Speed],Alger,23
18	2023-01-01,Agence d'El Khroub [Guepex],Constantine,107
19	2023-01-01,Agence d'Oued Rhiou [Guepex],Relizane,40
20	2023-01-01,Agence de Adrar Validine,Adrar,474
21	2023-01-01,Agence de Aflou [Validine],Laghouat,75

Figure 4.4 Exemple de structure des données agrégées par jour et par station sous format CSV

4.4.1.2. Analyse Exploratoire des Données (EDA)

Une fois les données collectées et transformées, l'Analyse Exploratoire des Données (EDA) s'impose. Elle est indispensable pour comprendre les données brutes, en dégager les motifs et anomalies, et ainsi guider efficacement leur prétraitement et la modélisation.

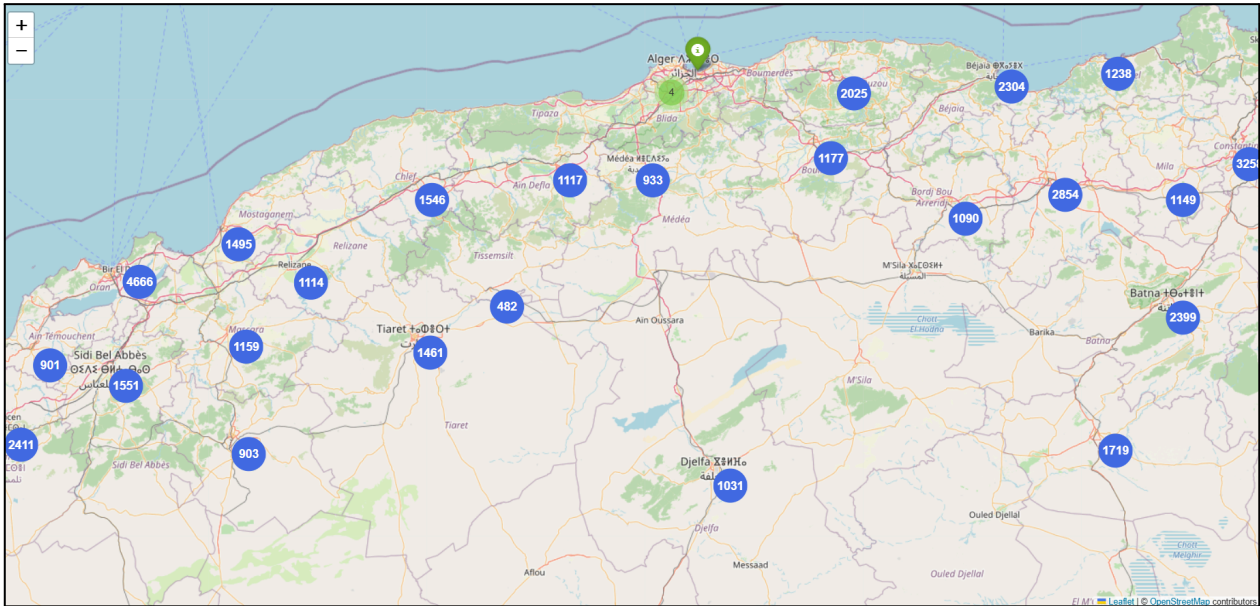


Figure 4.5 Moyenne quotidienne des colis de Yalidine traités par wilaya et par jour

Cette carte visualise la moyenne quotidienne des colis reçus ou traités par les différentes wilayas. On observe clairement une forte concentration de l'activité de livraison dans les grandes villes du nord du pays comme illustré dans la **Figure 4.5**. Des pôles majeurs comme Oran et Tlemcen à l'ouest, Alger au centre, et Batna, Sétif et Constantine à l'est se distinguent par des volumes élevés. Inversement, les régions plus intérieures ou du sud montrent des volumes de colis reçus plus faibles. Cette vue d'ensemble est essentielle pour comprendre la répartition géographique de la demande et optimiser la logistique quotidienne.

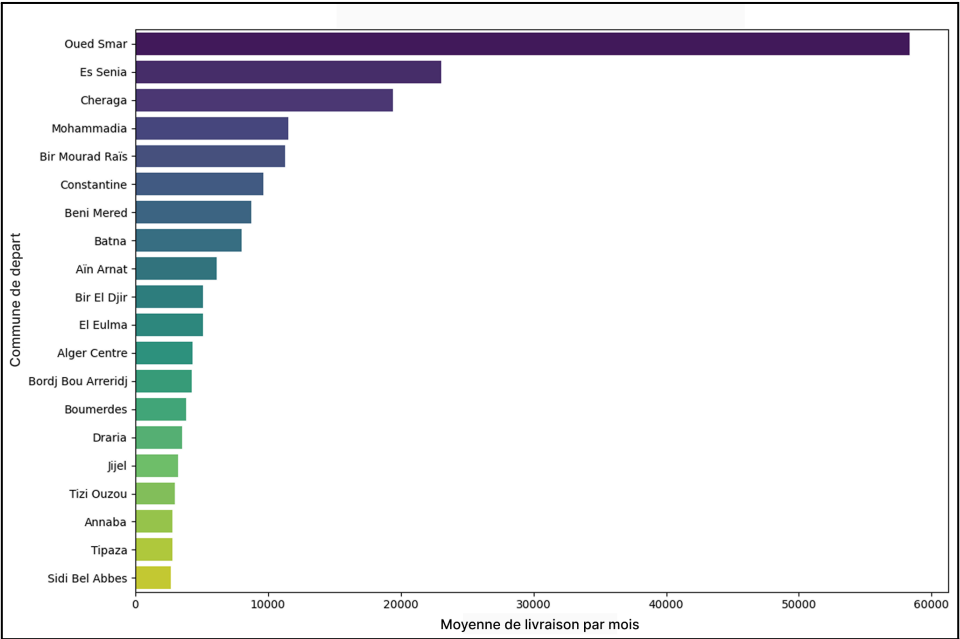


Figure 4.6 Moyenne des livraisons mensuelles par commune de départ

La **Figure 4.6** identifie les points d'origine les plus importants. On voit clairement que Oued Smar, Cheraga, Mohammadia et Bir Mourad Raïs (*toutes dans la Wilaya d'Alger*), Es Senia (*Oran*) et Constantine sont les communes d'où partent le plus de colis.

Cette information, combinée à la **Figure 4.5** des réceptions quotidiennes, est essentielle pour notre étude. Elle permet de révéler les différents schémas de flux (patterns) entre les départs et les points de réception, et de comprendre la spécificité des volumes et des dynamiques propres à chaque wilaya. Ces deux figures sont des exemples clés de notre Analyse Exploratoire des Données (EDA), qui a un rôle fondamental pour identifier et comprendre les points de fortes demandes (*wilayas/centres régionaux*) les plus importants en termes de volume de livraisons au sein du réseau logistique de Yalidine

4.4.1.3. Prétraitement des données

Une fois les données collectées, transformées et explorées, un prétraitement est essentiel pour les rendre exploitables par notre système de prévision. Cette étape garantit la qualité des données d'entrée, comme illustré dans la **Figure 4.7**.



Figure 4.7 Pipeline de prétraitement des données

4.4.1.3.1. Nettoyage des données

Les données initialement extraites depuis l'API pourront être bruitées et pourront comporter des doublons, des valeurs aberrantes ou manquantes. Une première étape va donc consister à vérifier et à corriger ces anomalies. Le nettoyage portera notamment sur:

- La vérification et la conversion des dates au format **datetime** pour un traitement temporel précis.
- Le filtrage des stations non pertinentes pour la prédiction, tel que le "*Centre colis en souffrance (HUB) [Yalidine Centre virtuel]*", car il s'agit d'un centre virtuel qui sert à collecter les colis introuvables et non d'un point de livraison réel à prédire.

4.4.1.3.2. Fragmentation des données

Pour une approche de prédiction personnalisée et plus fine, les données seront fragmentées par station "*hub_name*". Chaque station disposera de son propre fichier de livraisons quotidiennes sur deux ans (2023-2024). Cette organisation permet un

entraînement indépendant des modèles, une meilleure adaptation aux spécificités locales, et facilite la détection du “*concept drift*”.

4.4.1.3.3. Redimensionnement

Les données seront redimensionnées pour s'adapter aux entrées des modèles. Pour les modèles séquentiels comme LSTM, elles seront structurées en fenêtres glissantes (ex: 10 jours consécutifs pour prédire le 11ème). Pour MLP et XGBoost, des caractéristiques à décalage temporel (lag features) seront générées pour capturer les tendances passées.

4.4.1.3.4. Normalisation

La normalisation a pour but d'optimiser la performance et la stabilité de nos modèles prédictifs, nous allons procéder comme suit. Cela mettra toutes les variables, y compris le nombre de colis et les autres caractéristiques, sur une échelle comparable afin d'éviter qu'une seule ne domine l'apprentissage.

4.4.1.3.5. Le partitionnement des données

Les données seront divisées chronologiquement en un jeu d'entraînement (80%) pour l'entraînement des modèles et un jeu de test (20%) pour évaluer leur performance.

4.4.1.4. Ingénierie et Enrichissement des Caractéristiques

Cette section est dédiée à l'ingénierie et l'enrichissement des caractéristiques, visant à doter nos modèles d'informations contextuelles, telles que la météo et les événements, afin d'optimiser leur capacité prédictive.

Pour enrichir nos données temporelles, nous concevons plusieurs types de caractéristiques contextuelles. Les caractéristiques temporelles incluront les indicateurs de jour de la semaine et de weekend, permettant de capturer les variations hebdomadaires dans les habitudes de livraison. Les informations météorologiques, notamment la température, seront intégrées pour refléter l'impact des conditions climatiques sur les volumes de commandes et les capacités logistiques comme illustré dans la **Figure 4.8**.

```
PFE > Data > aggregated_data.csv
```

	date	station	wilaya	parcel_count	day_of_week	is_weekend	month	is_holiday	avg_temp
1	2023-01-01	Agence Ali Mendjeli [Validine]	Constantine	961	6	1	1	1	12
2	2023-01-01	Agence BIRMOURADRAIS [Easy and Speed]	Alger	205	6	1	1	1	15
3	2023-01-01	Agence Belle vue [Validine]	Constantine	356	6	1	1	1	12
4	2023-01-01	Agence Bordj El Bahri [Validine]	Alger	251	6	1	1	1	15
5	2023-01-01	Agence Canastel [speedmail]	Oran	184	6	1	1	1	16
6	2023-01-01	Agence Chetouane [Validine]	Tlemcen	343	6	1	1	1	14
7	2023-01-01	Agence Cité Soualem [Guepex]	Bordj Bou Arreridj	336	6	1	1	1	10
8	2023-01-01	Agence Fernand Ville [Validine]	Oran	285	6	1	1	1	16
9	2023-01-01	Agence Laghouat Cité OASIS [Easy and speed]	Laghouat	313	6	1	1	1	8
10	2023-01-01	Agence Les Eucalyptus [Zimou-Express]	Alger	12	6	1	1	1	15
11	2023-01-01	Agence Remchi [Validine]	Tlemcen	10	6	1	1	1	14
12	2023-01-01	Agence Rue du stade [Guepex]	Tipaza	5	6	1	1	1	13
13	2023-01-01	Agence Salem Shopping Mall Validine	M'Sila	205	6	1	1	1	11
14	2023-01-01	Agence Sidi Mabrouk [Guepex]	Constantine	95	6	1	1	1	12
15	2023-01-01	Agence SpeedMail [OUEDSMAR] [speedMail]	Alger	6	6	1	1	1	15
16	2023-01-01	Agence d'El Hamiz [Easy and Speed]	Alger	23	6	1	1	1	15
17	2023-01-01	Agence d'El Khroub [Guepex]	Constantine	107	6	1	1	1	12
18	2023-01-01	Agence d'Oued Rhiou [Guepex]	Relizane	40	6	1	1	1	13
19	2023-01-01	Agence de Adrar Validine	Adrar	474	6	1	1	1	20
20	2023-01-01	Agence de Aflou [Validine]	Laghouat	75	6	1	1	1	8
21	2023-01-01	Agence de Aflou [Validine]	Laghouat	75	6	1	1	1	8

Figure 4.8 Données en format csv après Enrichissement

Nous développerons également des caractéristiques événementielles spécifiques au contexte algérien (illustré dans la **Figure 4.9**), incluant les jours fériés nationaux, les fêtes religieuses et les événements sociaux majeurs qui influencent significativement les changements de consommation. Enfin, les caractéristiques saisonnières permettront de modéliser les variations cycliques liées aux différentes saisons de l'année, capturant ainsi les fluctuations naturelles de la demande logistique selon les périodes climatiques et culturelles.

1	"Date", "Événement", "Catégorie"
2	"2023-01-01", "Jour de l'An", "Civil & National"
3	"2023-01-12", "Yennayer 2973 (Nouvel An amazigh)", "Civil & National"
4	"2023-03-23", "Début du Ramadan", "Religieux"
5	"2023-04-21", "Aïd el-Fitr (fin du Ramadan)", "Religieux"
6	"2023-06-06", "Brevet d'Enseignement Moyen (BEM)", "Scolaire & Académique"
7	"2023-06-11", "Baccalauréat", "Scolaire & Académique"
8	"2023-06-29", "Aïd al-Adha (fête du sacrifice)", "Religieux"
9	"2023-07-04", "Début des vacances d'été", "Scolaire & Académique"
10	"2023-07-28", "Achoura", "Religieux"
11	"2023-09-20", "Rentrée scolaire 2023-2024", "Scolaire & Académique"
12	"2023-09-27", "Mawlid al-Nabawi", "Religieux"
13	"2023-10-29", "Vacances d'automne", "Scolaire & Académique"
14	"2023-11-01", "Fête de la Révolution", "Civil & National"
15	"2023-12-19", "Vacances d'hiver", "Scolaire & Académique"
16	

Figure 4.9 Événements - Année 2023

4.4.2. Module d'Apprentissage et Entraînement

Le module d'entraînement, cœur du système de prédiction, gère l'apprentissage parallèle et la sélection adaptative du modèle optimal pour chacune des 182 stations du réseau Yalidine.

4.4.2.1. Algorithme de Sélection

Cette section détaillera le processus par lequel le modèle de prévision optimal est choisi et ses hyperparamètres ajustés pour chaque station.

4.4.2.1.1. Principe de l'Algorithme

L'algorithme de sélection automatique entraîne simultanément cinq modèles candidats sur les mêmes données historiques pour chaque station. Le modèle optimal est choisi automatiquement en sélectionnant celui qui présente la valeur minimale de l'erreur quadratique moyenne (MSE), garantissant ainsi une adaptation précise aux spécificités locales sans intervention humaine. comme décrit dans l'algorithme dans la **Figure 4.10** ci

Algorithm 1 Algorithme de Sélection Automatique de Modèle par Station

Entrées:

- S_{liste} : Ensemble de toutes les stations.
- M_{types} : Liste des types de modèles candidats.
- D_s : Données historiques disponibles pour chaque station $s \in S_{liste}$.
- $P_{metrique}$: Métrique de performance pour l'évaluation.

Sorties:

- M_s^* : Le modèle optimal sélectionné pour chaque station $s \in S_{liste}$.

```

1: for all station  $s$  dans  $S_{liste}$  do
2:    $ModelesEntraines_s \leftarrow \emptyset$   $\triangleright$  Pour stocker les modèles entraînés pour la
    station  $s$ 
3:    $Performances_s \leftarrow \emptyset$   $\triangleright$  Pour stocker les performances des modèles pour
    la station  $s$ 
4:    $DonneesStation_s \leftarrow \text{RécupérerDonnéesHistoriques}(s)$ 
   Phase 1: Entraînement parallèle des modèles candidats
5:   for all typeDeModele  $mt$  dans  $M_{types}$  do
6:      $modele_{s,mt} \leftarrow \text{EntraînerModèle}(mt, DonneesStation_s)$   $\triangleright$ 
    L'entraînement peut être parallélisé
7:     Ajouter  $modele_{s,mt}$  à  $ModelesEntraines_s$ 
8:   end for
   Phase 2: Évaluation comparative
9:   for all modeleEntraîne  $m$  dans  $ModelesEntraines_s$  do
10:     $perf_m \leftarrow \text{CalculerPerformance}(m, DonneesStation_s, P_{metrique})$ 
11:     $Performances_s[m] \leftarrow perf_m$ 
12:   end for
   Phase 3: Sélection automatique du meilleur modèle
13:    $M_s^* \leftarrow \text{IdentifierMeilleurModèle}(Performances_s, P_{metrique})$ 
14:   Enregistrer  $M_s^*$  comme modèle sélectionné pour la station  $s$ .
15: end for
16: return  $M_s^*$  pour toutes les stations  $s \in S_{liste}$ .
```

dessous :

Figure 4.10 Algorithme de sélection automatique de modèle par station

Pour offrir une vision d'ensemble, la **Figure 4.11** illustre graphiquement le flux complet et les interactions entre les différentes étapes de l'algorithme, après l'exploration détaillée de chaque phase

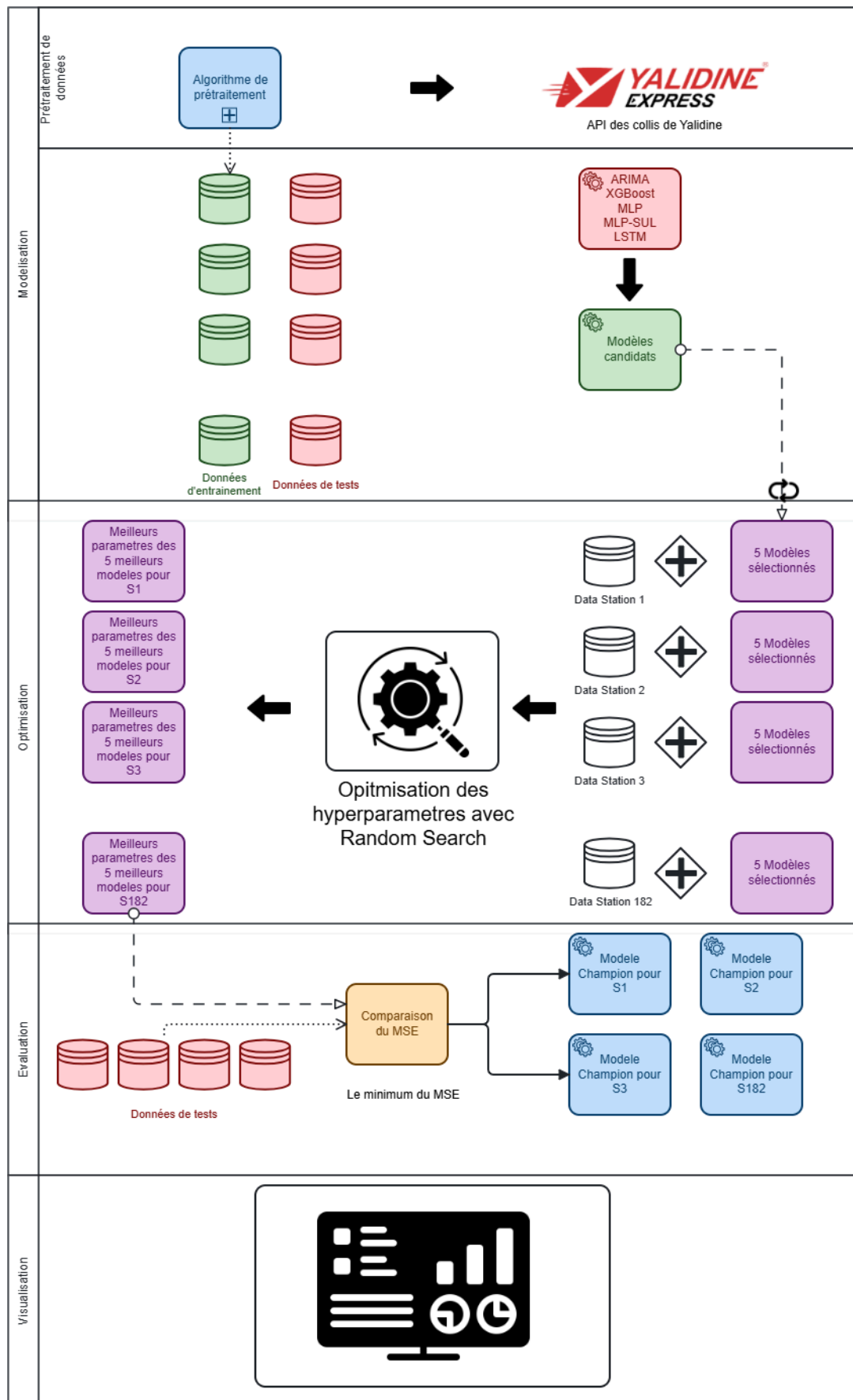


Figure 4.11 Vue globale de l'algorithme du sélection des modèles

4.4.2.1.2. Optimisation des Hyperparamètres

L'optimisation des hyperparamètres est une étape préalable à la sélection finale. Pour chaque modèle candidat, le système procédera à une exploration de l'espace des paramètres en utilisant des techniques de recherche aléatoire, permettant d'identifier automatiquement la configuration optimale pour chaque type de modèle.

L'approche sera basée sur la recherche aléatoire (*Random Search*) comme illustré dans la **Figure 4.12** telle qu'introduite par Bergstra et Bengio (2012) plutôt que sur une méthode exhaustive comme la recherche en grille (*Grid Search*), cette dernière s'avérant trop coûteuse en ressources computationnelles. La recherche aléatoire présente plusieurs avantages : une couverture plus efficace de l'espace des paramètres, un coût computationnel réduit, et une efficacité démontrée dans l'optimisation des hyperparamètres.

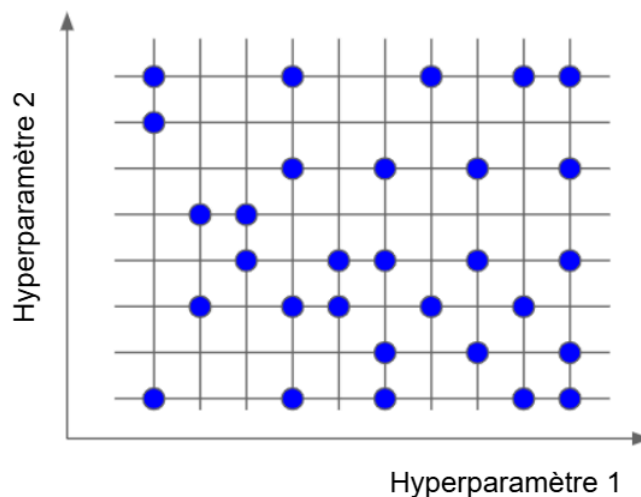


Figure 4.12 Illustration du principe de *Random Search* pour l'optimisation des hyperparamètres

Le processus d'optimisation explorera les paramètres critiques de chaque algorithme : les ordres autorégressifs et de moyenne mobile pour *ARIMA*, la profondeur des arbres et le taux d'apprentissage pour *XGBoost*, le nombre de couches et de neurones pour les réseaux de neurones, ainsi que les paramètres de régularisation. Chaque modèle sera évalué avec différentes combinaisons de paramètres générées aléatoirement, et la configuration optimale sera retenue selon la métrique MSE.

4.4.2.2. Justification du choix des modèles candidats

Le choix des modèles utilisés repose sur une étude approfondie de la littérature, ciblant les modèles performants dans le domaine des séries temporelles et prévisions de charge logistique :

- **ARIMA** : Référence pour les séries temporelles linéaires et stationnaires.
- **XGBoost** : Modèle de boosting puissant pour capter des non-linéarités, robuste aux valeurs aberrantes
- **MLP** : Réseau de neurones dense, apte à modéliser des relations complexes.
- **MLP-SUL** : Variante améliorée du MLP, intégrant des mécanismes de sélection et de mise à jour des poids pour une adaptation dynamique.
- **LSTM** : Modèle récurrent spécialisé dans l'apprentissage des dépendances temporelles à long terme.

Ces modèles offriront une complémentarité en termes de capacité à gérer la linéarité, la non-linéarité, la dynamique temporelle et les structures complexes.

4.4.2.3. Métrique d'évaluation

Nous avons choisi d'opter pour l'erreur moyenne quadratique "*Mean Squared Error*" (MSE) comme critère principal d'évaluation des performances pour plusieurs raisons :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Il est largement utilisé dans les problèmes de régression.
- Il pénalise fortement les grandes erreurs, ce qui sera crucial dans un contexte logistique où une mauvaise prévision pourrait causer des déséquilibres importants dans la répartition des ressources.
- Il est simple à interpréter et à comparer entre modèles.

L'usage du MSE permettra donc une sélection objective du modèle le plus performant pour chaque station.

4.4.2.4. Sauvegarde des Meilleurs Modèles

Le module d'entraînement sauvegardera les 5 meilleurs modèles pour chaque combinaison station-modèle. Pour chaque station, le système conservera le meilleur modèle optimal ainsi que la sauvegarde des statistiques telles que la moyenne, la variance, l'écart-type, le nombre de features et le nombre d'échantillons pour les modèles. Un cache des hyperparamètres sera également sauvegardé.

Cette organisation assurera une gestion efficace, une traçabilité complète et facilitera le déploiement rapide des configurations optimales pour l'ensemble du réseau de 182 stations Yalidine.

4.4.3. Module de Prédiction et Visualisation

Ce module sera conçu autour d'un gestionnaire central qui chargera automatiquement le modèle optimal pour chaque station en consultant les métadonnées de performance sauvegardées. Un système de normalisation appliquera la normalisation appropriée selon le type de modèle sélectionné, garantissant la cohérence des transformations de données. Le moteur de prédiction comme illustré dans la **Figure 4.13** unifié générera les prévisions en adaptant sa stratégie selon l'algorithme utilisé, tout en orchestrant le traitement simultané des 182 stations. Enfin, une partie d'analyse comparative et de visualisation produira les graphiques et tableaux nécessaires à l'évaluation des performances et à la prise de décision.

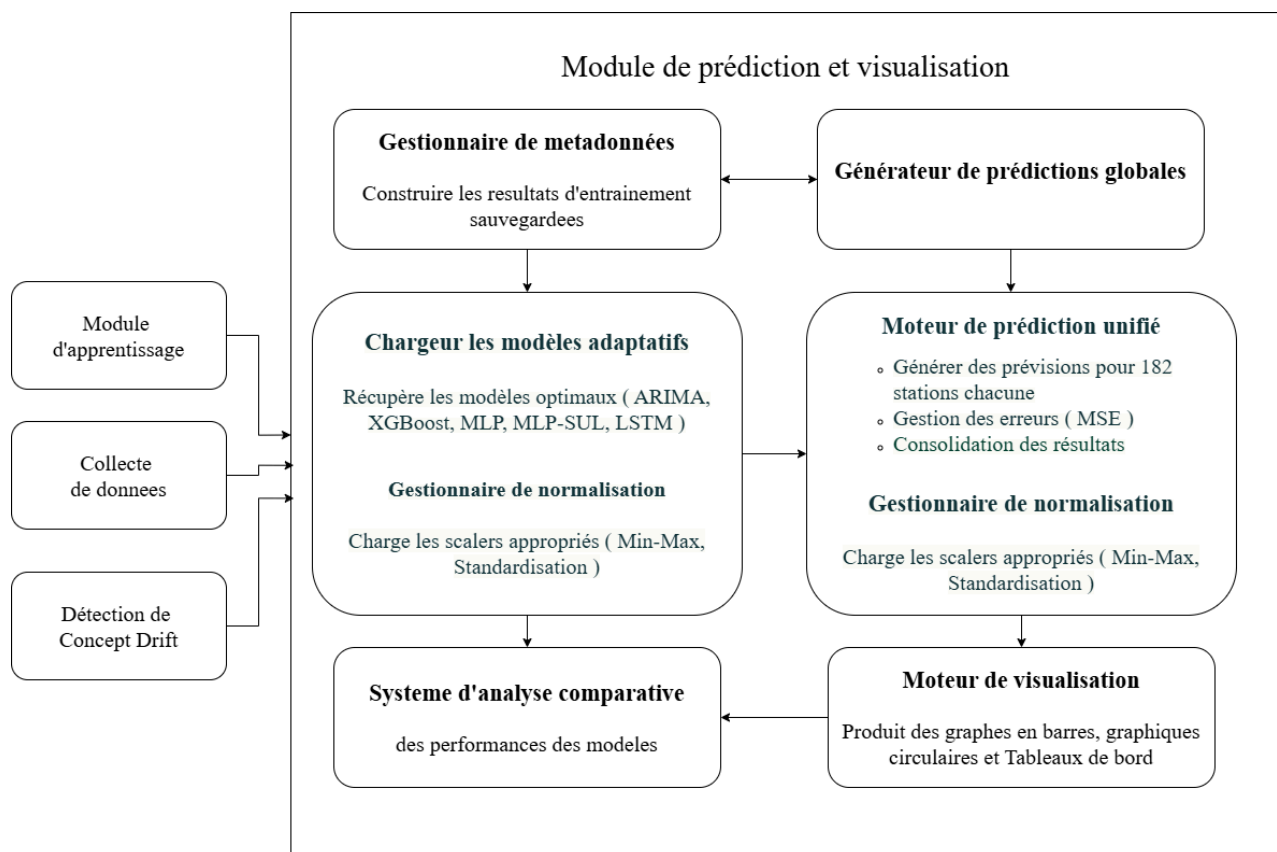


Figure 4.13 Module de Prédiction et Visualisation

4.4.4. Module de détection du Concept drift

Notre solution intègre une gestion proactive du concept drift pour maintenir la robustesse et la pertinence des modèles face aux évolutions des comportements logistiques. Ce module intégrera des mécanismes de détection et d'adaptation pour anticiper les changements dans les schémas de volume de colis, garantissant la fiabilité des prévisions sur le long terme comme illustré dans la **Figure 4.14**

Pour répondre aux défis spécifiques de la détection de drift dans le contexte logistique de Yalidine, nous avons conçu et appliqué deux méthodes complémentaires : DDM “*Drift Detection Method*” et ADWIN “*Adaptive Windowing*”. Cette approche bi-algorithmique permettra de couvrir un spectre large de types de dérives tout en s'adaptant aux caractéristiques variées des 182 stations du réseau.

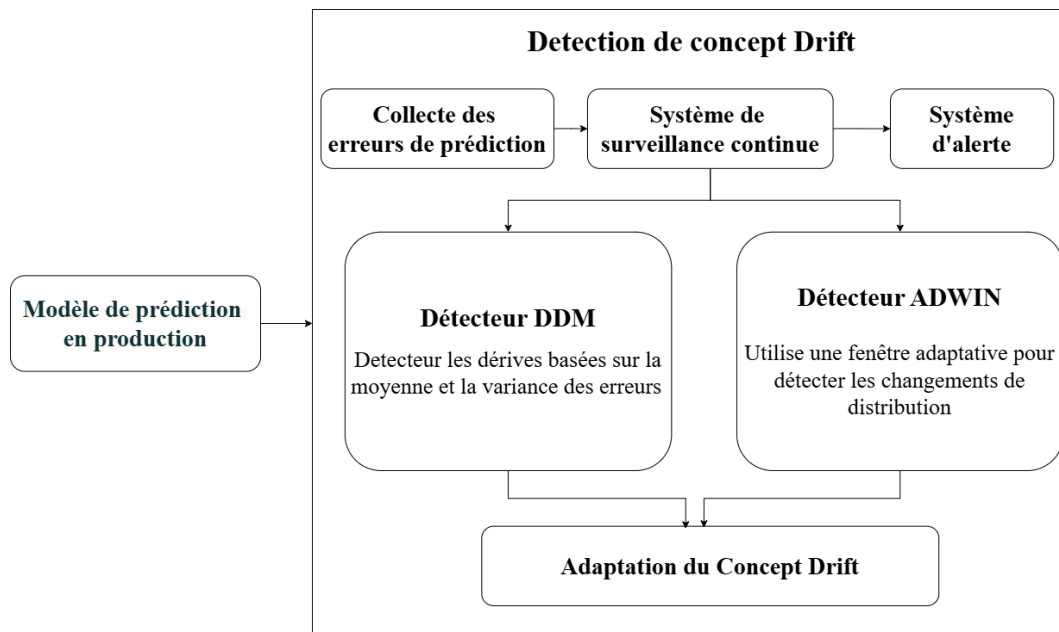


Figure 4.14 Module de détection du Concept drift

4.4.4.1. Méthode DDM

Nous avons sélectionné l'algorithme DDM pour sa capacité remarquable à détecter les dérives basées sur l'évolution statistique des erreurs de prédiction. Cette méthode s'avère particulièrement adaptée au contexte logistique de Yalidine car elle surveille simultanément la moyenne et la variance des erreurs, permettant d'identifier les changements dans la qualité prédictive des modèles.

L'algorithme DDM présente l'avantage majeur de fonctionner avec des seuils statistiquement fondés, calculés à partir de l'écart-type des erreurs observées. Cette approche garantit une détection objective des dérives sans nécessiter de calibrage manuel complexe pour chaque station. Dans notre contexte, DDM surveillera l'évolution du rapport entre les MSE de test (*calculés lors de l'entraînement*) et les MSE en production (*calculés sur les données réelles*) comme illustré dans la **Figure 4.15**, permettant d'identifier rapidement les dégradations de performance significatives.

La robustesse de DDM face aux variations naturelles des données logistiques constitue un atout essentiel. L'algorithme distingue efficacement les fluctuations normales liées à

la saisonnalité ou aux événements ponctuels des véritables dérives structurelles nécessitant une intervention. Cette capacité de discrimination s'avère cruciale dans un environnement où les volumes de colis peuvent varier significativement selon les périodes (*Ramadan, fêtes, événements économiques*).

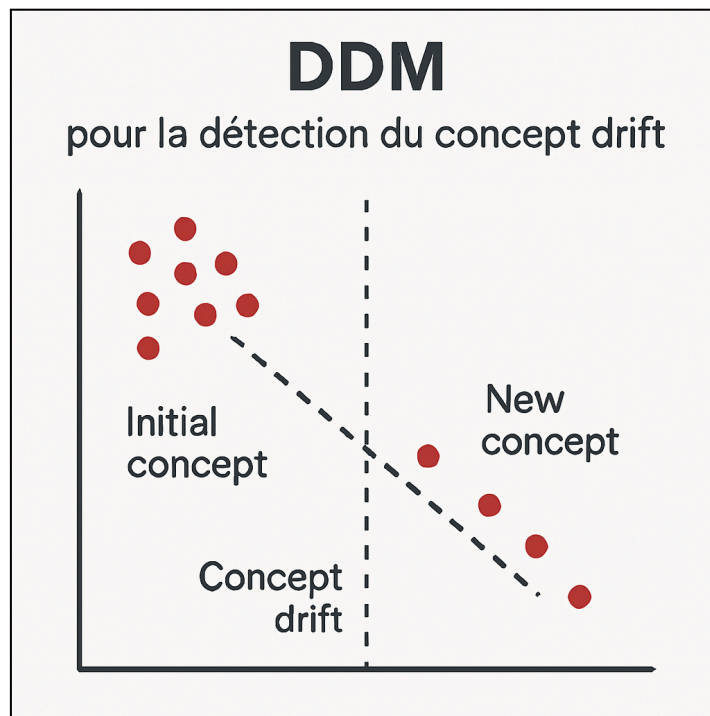


Figure 4.15 Algorithme de DDM

4.4.4.2. Méthode ADWIN

L'algorithme *ADWIN* complète notre stratégie de détection par sa capacité unique à adapter dynamiquement sa fenêtre d'observation en fonction de l'évolution des données. Cette caractéristique s'avère particulièrement précieuse dans le contexte de Yalidine où les patterns de demande peuvent évoluer graduellement ou subir des changements abrupts selon les stations et les périodes.

ADWIN excelle dans la détection des changements de distribution et de moyenne, même lorsque ces changements s'opèrent de manière progressive comme illustré dans la **Figure 4.16**. Cette capacité répond directement aux défis posés par l'évolution des comportements de consommation algériens, où les changements peuvent être subtils mais persistants. L'algorithme ajuste automatiquement sa sensibilité en fonction de la stabilité récente des données, offrant une détection fine sans générer d'alertes excessives.

La flexibilité de *ADWIN* dans la gestion des fenêtres temporelles permet une adaptation optimale aux caractéristiques spécifiques de chaque station. Les stations urbaines à forte

activité bénéficieront de fenêtres plus courtes pour une détection réactive, tandis que les stations rurales à activité modérée utiliseront des fenêtres plus larges pour une analyse statistiquement significative. Cette adaptabilité automatique élimine le besoin de paramétrage manuel station par station, simplifiant considérablement la maintenance du système.

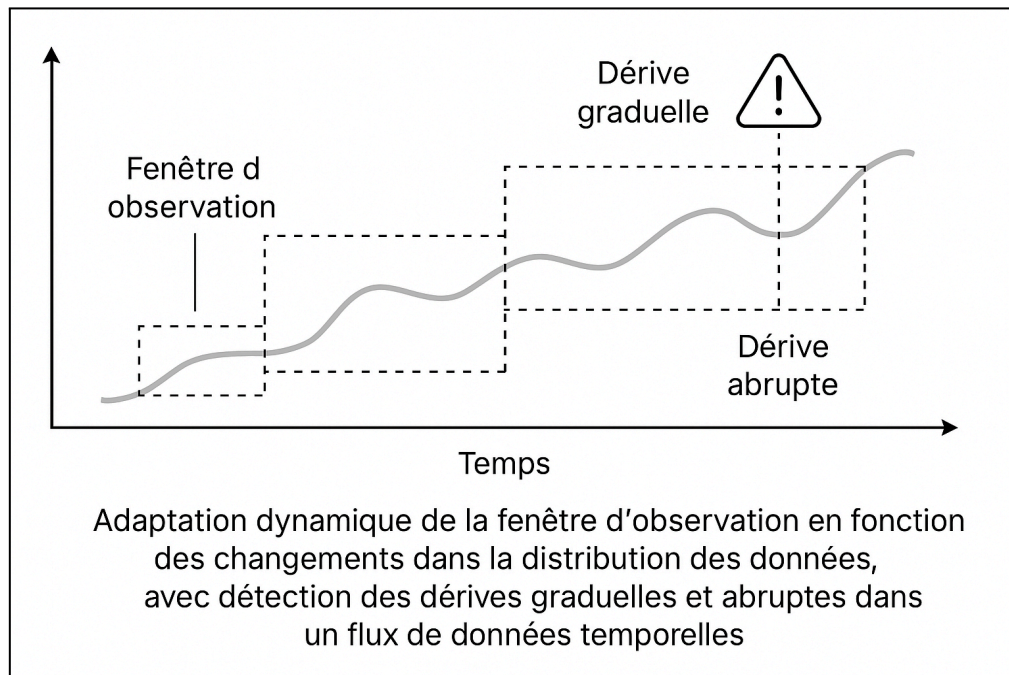


Figure 4.16 Algorithme de ADWIN

4.5. Conclusion

Ce chapitre pose les fondations d'un système prédictif performant, extensible et conforme aux besoins métiers, prêt à être mis en œuvre concrètement et intégré dans l'environnement opérationnel de l'entreprise.

Chapitre 05

Réalisation

5.1. Introduction

Ce chapitre est dédié à la mise en œuvre concrète de notre solution de prévision de la demande de colis. Nous y détaillons les choix technologiques et les outils spécifiques que nous avons employés pour construire la plateforme. L'objectif principal est de présenter l'architecture technique complète, ainsi que les processus d'entraînement et d'optimisation des modèles, tous essentiels pour garantir la précision, la robustesse et la performance de notre système.

5.2. Choix technologiques

5.2.1. Langages et frameworks utilisés

La conception de notre solution de prévision a nécessité une sélection rigoureuse des technologies, en privilégiant la performance, la flexibilité et la compatibilité avec l'écosystème existant. Ces choix ont été guidés par des critères clairs pour assurer la robustesse et l'efficacité de notre système.

5.2.1.1. Langage de programmation



Figure 5.1 Logo de Python

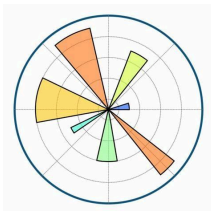
Pour l'analyse et la modélisation des données, ainsi que pour la construction de la logique de prévision, nous avons opté pour le langage de programmation Python. Ce choix stratégique repose sur plusieurs avantages clés :

- **Accessibilité et coût** : Python est un langage open-source et entièrement gratuit, réduisant les coûts de développement.
- **Écosystème riche** : Il offre une vaste collection de bibliothèques et de packages spécialisés en statistiques, en apprentissage automatique et en science des données.

- **Capacités de prévision** : Sa robustesse en fait un choix idéal pour le développement de modèles de prévision sophistiqués.
- **Simplicité et Apprentissage** : Sa syntaxe claire et intuitive rend Python facile à apprendre et à utiliser, accélérant le développement.
- **Extensibilité** : Il permet d'intégrer facilement des modules développés dans d'autres langages (comme C++ ou C) pour des performances optimisées si nécessaire.
- **Soutien communautaire** : Une communauté très active et une documentation exhaustive garantissent un support constant et une résolution rapide des problèmes.

5.2.1.2. Packages et librairies

Dans cette section, nous examinons les différentes bibliothèques Python pour les données de séries chronologiques que nous avons utilisées pour l'analyse et l'entraînement et l'implémentation des modèles.

Package	Description
 <i>Figure 5.2 PyTorch</i>	Une bibliothèque d'apprentissage profond essentielle, PyTorch a été choisie pour sa capacité à gérer des graphes de calcul dynamiques, facilitant le débogage et la personnalisation des modèles. Elle offre également une accélération GPU via CUDA et un moteur de différenciation automatique performant, crucial pour l'entraînement des réseaux neuronaux.
 <i>Figure 5.3 NumPy</i>	Fondamentale pour le calcul numérique et scientifique, NumPy fournit un objet tableau N-dimensionnel puissant, facilitant les opérations sur les matrices, l'algèbre linéaire, les transformées de Fourier et les calculs statistiques.
 <i>Figure 5.4 Matplotlib</i>	Matplotlib est une bibliothèque de traçage Python qui produit des figures de qualité dans une variété de formats papier et d'environnements interactifs sur toutes les plateformes. Matplotlib peut être utilisé dans les scripts Python, les shells Python et IPython, le bloc-notes Jupyter, serveurs d'applications Web.

 <i>Figure 5.5 Scikit-learn</i>	Scikit-learn est un outil simple et efficace pour l'exploration de données et l'analyse des données. En particulier, cette bibliothèque implémente une gamme d'algorithmes d'apprentissage automatique, prétraitement, validation croisée et visualisation utilisant une interface unifiée.
 <i>Figure 5.6 River</i>	River est une bibliothèque Python pour le machine learning en flux, conçue pour traiter des données séquentielles. Elle permet de construire des modèles qui apprennent et se mettent à jour en continu au fur et à mesure que de nouvelles données arrivent.

Tableau 5.1 Liste des principales bibliothèques Python utilisées et leurs descriptions

5.2.1.3. Autres outils

Outil	Description
 <i>Figure 5.7 VSCode</i>	VSCode est un éditeur de code source polyvalent de Microsoft c'était notre environnement de développement principal. Ses fonctionnalités, telles que le débogage intégré, la complétion intelligente, la mise en évidence syntaxique et l'intégration Git, ont optimisé notre productivité.
 <i>Figure 5.8 GitHub</i>	En tant que plateforme de contrôle de version et de collaboration, GitHub a été indispensable pour la gestion du code source. Il a permis un suivi efficace des modifications, un travail d'équipe fluide grâce aux dépôts, branches et pull-requests, et une gestion centralisée des tâches.
 <i>Figure 5.9 draw.io</i>	Cette application web gratuite a été utilisée pour la création de diagrammes (architecture, flux de données, etc.) et de dessins vectoriels, facilitant la visualisation et la communication des concepts techniques.

 <i>Figure 5.10 Google Drive</i>	Nous avons tiré parti de Google Drive pour le stockage et le partage sécurisé de documents et de fichiers du projet, assurant une synchronisation et un accès facilités pour l'équipe.
 <i>Figure 5.11 Zotero</i>	Zotero est un outil gratuit, facile à utiliser et open source, conçu pour aider les chercheurs à collecter, organiser, annoter, citer et partager leurs travaux de recherche. Il permet de gérer des données bibliographiques et des documents de recherche tels que des fichiers PDF
 <i>Figure 5.12 LaTeX</i>	Cette plateforme collaborative en ligne pour LaTeX a été choisie pour la rédaction des algorithmes, des tableaux ainsi que les équations mathématiques de ce rapport. Elle a permis une édition simultanée, un contrôle de version intégré et une gestion simplifiée de la mise en forme académique

Tableau 5.2 : Outils logiciels utilisés pour le développement et la gestion du projet

5.2.1.4. Matériels et ressources utilisées

Pour la phase de développement et d'implémentation de la solution, nous avons eu recours à nos machines personnelles. Leurs spécifications sont détaillées ci-dessous, permettant une compréhension des ressources informatiques utilisées avant le déploiement final au sein de l'organisme d'accueil :

PC1	Processeur	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz 2.59 GHz
	RAM	24.0 GB (23.9 GB usable)
	GPU	gtx-1050 4GB Vram
PC2	Processeur	Intel(R) Core(TM) i5-1135G7 CPU 2.4GHz
	RAM	24.0 GB (23.9 GB usable)
	GPU	Iris Xe Graphics

Tableau 5.3 : Spécifications matérielles des machines utilisées lors du développement

Bien que ces configurations aient permis un développement fonctionnel, il est essentiel de souligner que l'entraînement de modèles de prévision de la demande, notamment ceux de “*Deep Learning*”, est une tâche intensive en ressources. Les spécifications de nos machines personnelles, en particulier au niveau des GPU et de la RAM, ont représenté un facteur limitant. Cela s'est traduit par des temps d'entraînement parfois longs et la nécessité d'ajuster la taille des jeux de données ou la complexité des architectures pour rester dans les limites de nos capacités. Ces contraintes mettent en lumière l'importance d'une infrastructure de calcul plus puissante pour les phases de production et de montée en charge.

5.2.2. Outils de gestion des données

La gestion des données joue un rôle fondamental dans tout projet de Science de données, notamment dans les projets de prévision à partir de séries temporelles. Pour assurer une préparation efficace, une manipulation flexible et un prétraitement structuré des données, plusieurs outils ont été utilisés dans notre solution.

5.2.2.1. Pandas



Figure 5.13 Pandas

La bibliothèque Pandas est l'un des piliers de l'analyse de données en Python. Elle permet de manipuler facilement des structures de données complexes telles que les DataFrames, qui sont particulièrement adaptés aux jeux de données tabulaires.

- Lecture et écriture de fichiers CSV et JSON.
- Agrégation, filtrage, jointures, tri des données.
- Création de colonnes dérivées et manipulation temporelle “*datetime*”.
- Structure pivot pour l'ensemble des transformations effectuées sur les données.

5.2.2.2. Formats de données

Deux formats principaux de fichiers ont été utilisés dans notre projet : le format JSON pour l'importation des données brutes depuis l'API interne de *Yalidine*, et le format CSV obtenu après transformation tabulaire via *Pandas*. Le CSV constitue le format de travail principal, étant léger, lisible et parfaitement compatible avec les bibliothèques Python utilisées pour l'entraînement des modèles.

5.2.3. Prétraitement et transformation des données

Pour préparer les données en vue de l'entraînement des modèles, plusieurs étapes de prétraitement ont été nécessaires. Ces opérations ont été réalisées grâce à différentes bibliothèques Python spécialisées, chaque approche étant adaptée aux exigences spécifiques de chaque famille d'algorithmes.

5.2.3.1. Prétraitement pour les Modèles ARIMA

Le prétraitement des données pour *ARIMA* suit une approche statistique rigoureuse adaptée aux exigences des modèles de séries temporelles classiques.

Notre pipeline vérifie d'abord la stationnarité de la série temporelle en appliquant le test **Augmented Dickey-Fuller**, un test statistique standard pour détecter la présence de racines unitaires. La fonction `check_stationarity()` utilise un seuil de significativité de 5% : si la **p-value** est inférieure à **0.05**, les données sont considérées comme stationnaires.

Lorsque la série n'est pas stationnaire ($p\text{-value} \geq 0.05$), nous appliquons une différenciation d'ordre 1 pour stabiliser la moyenne et la variance. Cette transformation, implémentée par la fonction `make_stationary()`, calcule la différence première de la série temporelle, éliminant ainsi les tendances linéaires.

La normalisation s'effectue avec un `StandardScaler` qui centre les données autour de zéro avec un écart-type unitaire, préservant la distribution gaussienne souvent requise par les modèles *ARIMA*. La division **train/test** respecte un ratio de **95/5%** pour maintenir suffisamment de données d'entraînement tout en conservant un ensemble de test représentatif.

5.2.3.2. Prétraitement pour les Modèles XGBoost

Le prétraitement *XGBoost* transforme la série temporelle en un problème de régression supervisée en créant des variables de décalage "lag features". Cette approche exploite la capacité de *XGBoost* à modéliser les relations non-linéaires entre les valeurs passées et futures. Les données sont normalisées avec **MinMaxScaler** et divisées selon un ratio 80/20% en respectant l'ordre temporel (`shuffle=False`) pour préserver la structure séquentielle. Le reshape final garantit la compatibilité avec l'interface `scikit-learn` de *XGBoost*.

5.2.3.3. Prétraitement pour les Modèles LSTM

Le prétraitement pour *LSTM* adopte une approche orientée séquences temporelles, créant des fenêtres glissantes de 10 jours pour capturer les dépendances temporelles à court et moyen terme. Notre pipeline transforme la série temporelle unidimensionnelle en séquences d'entrées multidimensionnelles. La normalisation utilise un **MinMaxScaler**

qui transforme les données dans l'intervalle , particulièrement adapté aux fonctions d'activation des réseaux de neurones. Les données sont organisées en DataLoader PyTorch avec un batch_size de 32, optimisant l'entraînement par mini-lots et permettant une utilisation efficace de la mémoire GPU.

5.2.3.4. Prétraitement pour les Modèles MLP et MLP Sul

Le prétraitement pour les perceptrons multicouches suit une approche similaire à XGBoost, créant des fenêtres de 10 valeurs passées comme variables d'entrée. Cette méthode permet aux réseaux de neurones feedforward de capturer les patterns temporels sans architecture récurrente explicite. La normalisation **MinMaxScaler** assure une convergence stable des algorithmes de gradient descent, tandis que l'encapsulation dans des DataLoader PyTorch facilite l'entraînement par lots avec mélange aléatoire des échantillons d'entraînement.

5.3. Architecture technique de la solution

Dans la *Figure 5.14*, on détaillera comment on a pu utiliser les outils technologiques pour la réalisation de ce système :

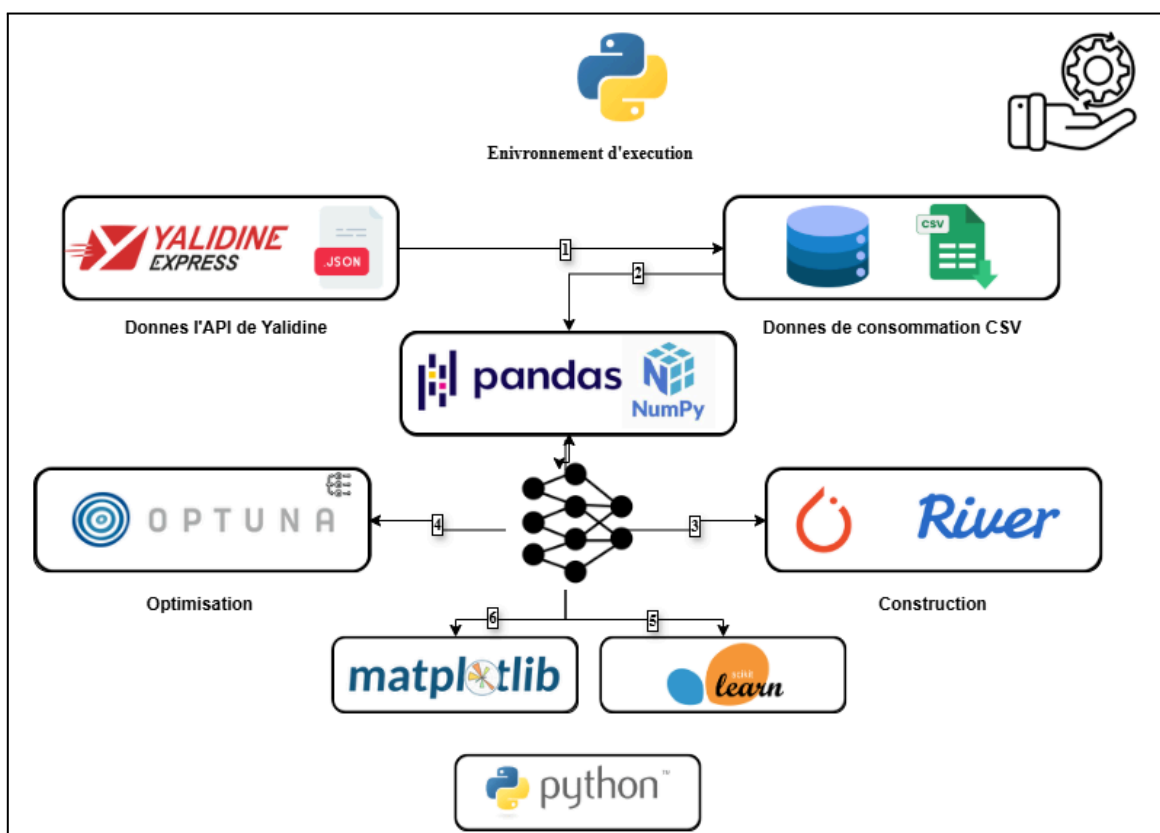


Figure 5.14 Architecture Technique (Applicative) de notre solution

Flux 1 : Collecte: Les données sont extraites de plusieurs sources (*API Yalidine*) au format brut (*JSON*).

Flux 2 : Prétraitement: Nettoyage, normalisation et transformation des données avec *Pandas* et *NumPy* pour obtenir un jeu de données exploitable.

Flux 3 et 4 : Entraînement et Optimisation: Les modèles sont entraînés par *PyTorch* et optimisés automatiquement avec **Optuna**. *River* sert à la détection du drift et à l'apprentissage incrémental..

Flux 5 et 6 : Prédiction et Visualisation: Génération des prévisions, visualisation des résultats via *Matplotlib* et *Scikit-learn* pour l'analyse et l'aide à la décision.

5.4. Implémentation des modèles de prédiction

Cette section détaille l'architecture logicielle et la méthodologie rigoureuse mises en œuvre pour l'entraînement, l'optimisation et la gestion des modèles de prédiction. L'ensemble du processus a été conçu pour fonctionner de manière parallèle et indépendante pour chaque station, garantissant ainsi une exécution rapide et une scalabilité pour l'ensemble du réseau Yalidine.

5.4.1. Méthode d'entraînement

Notre implémentation du processus d'entraînement basée sur notre conception avec des fonctions spécialisées pour chaque type de modèle. Le système exécute l'entraînement en parallèle pour optimiser les performances computationnelles.

5.4.1.1. Entraînement LSTM

Le modèle *LSTM* est entraîné pour chaque station en utilisant *PyTorch*. L'architecture comprend une couche *LSTM* (dont la taille du vecteur caché est optimisée entre 16 et 256), un dropout à 0.2 pour limiter le surapprentissage, puis une couche dense de sortie. Un scheduler réduit le learning rate de 50% toutes les 100 époques pour affiner la convergence. La régularisation *L2* est appliquée.

5.4.1.2. Entraînement MLP et MLP Sul

Les *MLP* classiques utilisent plusieurs couches denses avec activation *ReLU*. La version *MLP-SUL* ajoute des couches de normalisation batch et du dropout (*par exemple 0.3*) pour améliorer la robustesse et la généralisation. Les architectures sont optimisées pour chaque station.

5.4.1.3. Entraînement XGBoost et ARIMA

L'algorithme *XGBoost* est intégré via l'API *scikit-learn*, tirant parti de ses capacités de gradient boosting distribué et de son efficacité computationnelle pour les tâches de régression sur séries temporelles.

ARIMA, quant à lui, est implémenté via la bibliothèque *pmdarima* pour l'auto-détection de son ordre optimal (p, d, q). Cette approche permet de déterminer automatiquement les trois paramètres clés qui définissent la structure du modèle pour chaque station

5.4.2. Optimisation des hyperparamètres

L'ensemble des modèles (*LSTM*, *MLP*, *MLP-SUL*, *XGBoost*) bénéficient d'une optimisation automatique des hyperparamètres avec Optuna : Chaque modèle dispose d'une fonction objectif spécifique, qui reçoit des suggestions de valeurs pour les hyperparamètres (*nombre d'époques*, *learning rate*, *taille des couches cachées pour les réseaux de neurones ... etc*).

Optuna construit et met à jour un modèle probabiliste de la fonction objectif afin de sélectionner les prochaines combinaisons à tester, maximisant ainsi l'efficacité de la recherche.

L'optimisation est parallélisée via *ThreadPoolExecutor*, ce qui permet de lancer plusieurs études d'optimisation en simultané sur différentes stations ou modèles, accélérant considérablement le processus global.

Un mécanisme de cache est mis en place pour sauvegarder et recharger les meilleurs hyperparamètres trouvés, évitant de refaire inutilement la recherche lors de futurs entraînements.

5.4.3. Stockage des Résultats

5.4.3.1. Sauvegarde des Modèles

Les modèles de deep learning (*LSTM*, *MLP*, *MLP-SUL*) sont sauvegardés au format natif *PyTorch* (.pth), incluant architecture et poids.

Les modèles *XGBoost* et *ARIMA* sont sérialisés avec *Pickle* (.pkl), ce qui permet de les recharger ultérieurement pour l'inférence ou l'analyse

Trois types de scalaires sont gérés et sauvegardés selon les besoins de chaque modèle (par exemple, *MinMaxScaler* pour les réseaux de neurones, scaler spécifique pour *XGBoost* si besoin), assurant la cohérence du prétraitement lors de la prédiction

Les métriques de performance (MSE pour chaque modèle et chaque station, meilleur modèle identifié) sont consolidées dans un fichier CSV structuré. L'accès concurrent à ce fichier est sécurisé par des verrous pour éviter toute corruption lors de la parallélisation

5.4.3.2. Traitement Parallèle Multi-Stations

L'architecture exploite le module multiprocessing de *Python* pour répartir le traitement sur plusieurs cœurs.

Chaque station est traitée par un processus indépendant, que ce soit pour l'entraînement, l'optimisation des hyperparamètres ou la détection de dérive. Cela permet de maximiser l'utilisation des ressources CPU et de réduire significativement le temps total d'exécution, notamment lors du traitement de grandes volumétries de données.

Un pool de processus est créé, avec un nombre de workers adapté au nombre de cœurs disponibles (*typiquement jusqu'à 8 pour éviter la saturation*). Les tâches (*par exemple, entraînement ou détection de drift par station*) sont distribuées dynamiquement dans ce pool. Les accès concurrents aux ressources partagées (*fichiers de résultats, modèles, scalaires*) sont protégés par des verrous (*locks*), garantissant l'intégrité des données produites lors de l'exécution parallèle.

5.4.4. Détection du concept drift

5.4.4.1. DDM

La méthode DDM (*Drift Detection Method*) compare les erreurs quadratiques moyennes (*MSE*) observées en production à celles enregistrées lors de l'entraînement. Les scalars sauvegardés sont systématiquement recherchés pour garantir la cohérence des transformations appliquées aux données. Le système fusionne les données d'entraînement et de test, extrait automatiquement le *MSE* du meilleur modèle pour chaque station, puis applique des seuils statistiques pour classifier les états de dérive. Un ratio de *MSE* supérieur à 2.0 signale une dérive (*DRIFT*), entre 1.5 et 2.0 un avertissement (*WARNING*), et inférieur à 1.5 un état stable (*STABLE*). Les résultats sont archivés dans un fichier CSV pour assurer la traçabilité et l'analyse ultérieure

5.4.4.2. ADWIN

L'algorithme *ADWIN*, implémenté via la bibliothèque *River*, détecte en ligne les changements dans la distribution des erreurs en utilisant une fenêtre adaptative. La sensibilité (*delta*) est ajustée en fonction du volume de données de chaque station : un *delta* de 0.01 pour plus de 100 jours de données, 0.005 pour 50 à 100 jours, et 0.003 pour moins de 50 jours. Cette approche garantit une détection fine et adaptée à la volumétrie de chaque station, permettant de réagir rapidement en cas de dérive significative des performances

5.5. Conclusion

Dans ce chapitre, nous avons détaillé l'architecture technique de notre solution, en commençant par la présentation des outils et des langages de programmation qui composent notre environnement de développement. Nous avons également abordé les différentes étapes d'implémentation, allant du prétraitement des données jusqu'à la mise en place et l'optimisation des modèles prédictifs ainsi que la détection du concept drift.

Dans le chapitre suivant, nous discuterons en détail les résultats obtenus au cours de nos expérimentations.

Chapitre 06

Tests et Résultats

6.1. Introduction

Ce chapitre présente les tests et validations réalisés pour évaluer la performance de notre système de prévision de la demande de colis. Nous y abordons les performances de nos modèles, l'analyse du concept drift, ainsi que les statistiques principales permettant de mesurer la robustesse et la précision de notre approche.

6.2. Apprentissage des modèles

6.2.1. Performance du modèle final

Pour analyser les performances de notre modèle, nous allons montrer les résultats des valeurs de la perte sur les données d'entraînement et les données de validation.

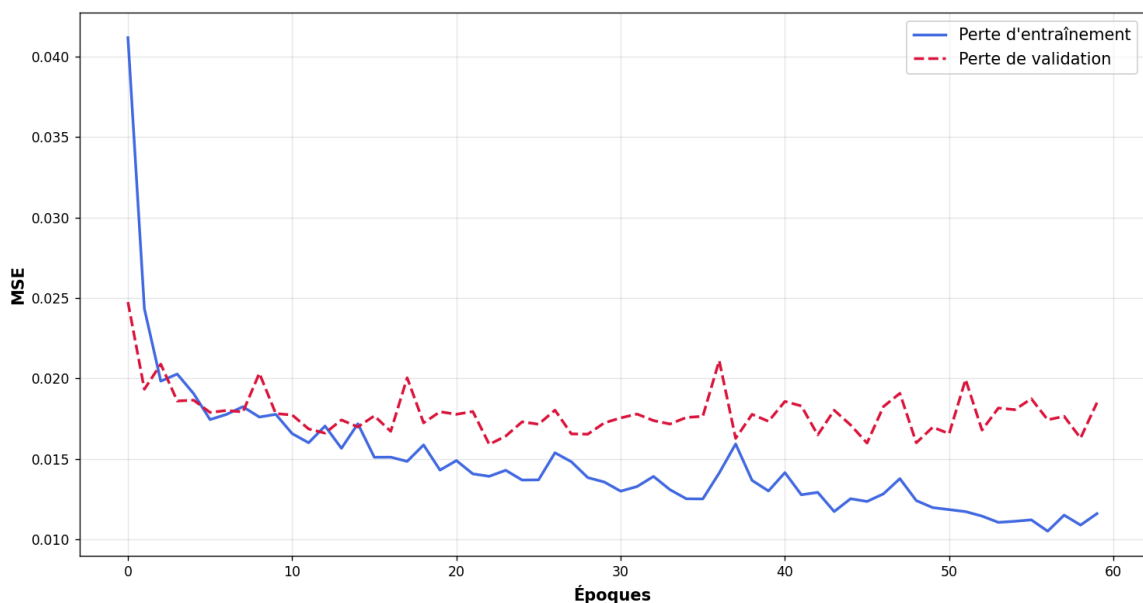


Figure 6.1 Perte d'entraînement du MLP-SUL – Corner El Harrach

Comme illustrée dans la **Figure 6.1**, qui présente l'évolution des pertes d'entraînement et de validation pour un exemple de la station Corner El Harrach, reflète en réalité une tendance généralisée à l'ensemble des stations étudiées. De manière globale, les courbes MSE diminuent rapidement au début de l'apprentissage puis se stabilisent, ainsi que les pertes de validation suivent une trajectoire similaire, avec un écart faible et stable entre les deux. Cette convergence régulière témoigne d'une bonne capacité de généralisation

des modèles déployés, sans surapprentissage marqué. Ainsi, bien que l'illustration soit ici donnée pour une station particulière, ce comportement s'observe de façon systématique sur l'ensemble du réseau, confirmant la robustesse de la démarche d'entraînement et la fiabilité des prédictions générées sur des données nouvelles. Par ailleurs, la proximité des valeurs de *MSE* entre les différents modèles impose de maintenir une traçabilité rigoureuse afin de toujours pouvoir identifier et charger le modèle le plus performant lors de la mise en production.

Le **Tableau 6.1** présenté ci-dessus expose les résultats de performance du modèle final pour un échantillon de stations sélectionnées:

Station	MSE LSTM	MSE_M LP	MSE MLP Sul	MSE XGBoost	MSE ARIMA	Best Model
Agence Cité Soualem	0.011	0.0113	0.0113	0.0117	1.4286	LSTM
Corner El Harrach	0.0173	0.0152	0.0147	0.0176	1.118	MLP-Sul
Agence de Aïn Benian	0.019	0.0128	0.0128	0.0142	1.0527	MLP-Sul

Tableau 6.1 : Résultats de performance des modèles pour un échantillon de stations

6.2.2. Algorithmes d'optimisation

Dans le cadre de ce travail, nous avons adopté une démarche comparative entre les principaux algorithmes d'optimisation utilisés dans l'entraînement des réseaux de neurones, afin de sélectionner l'optimiseur le plus performant et le plus stable pour l'ensemble des stations et des architectures testées. Cette comparaison a été systématiquement réalisée pour chaque station, en évaluant notamment SGD, RMSprop et Adam, qui figurent parmi les méthodes les plus reconnues dans le domaine.

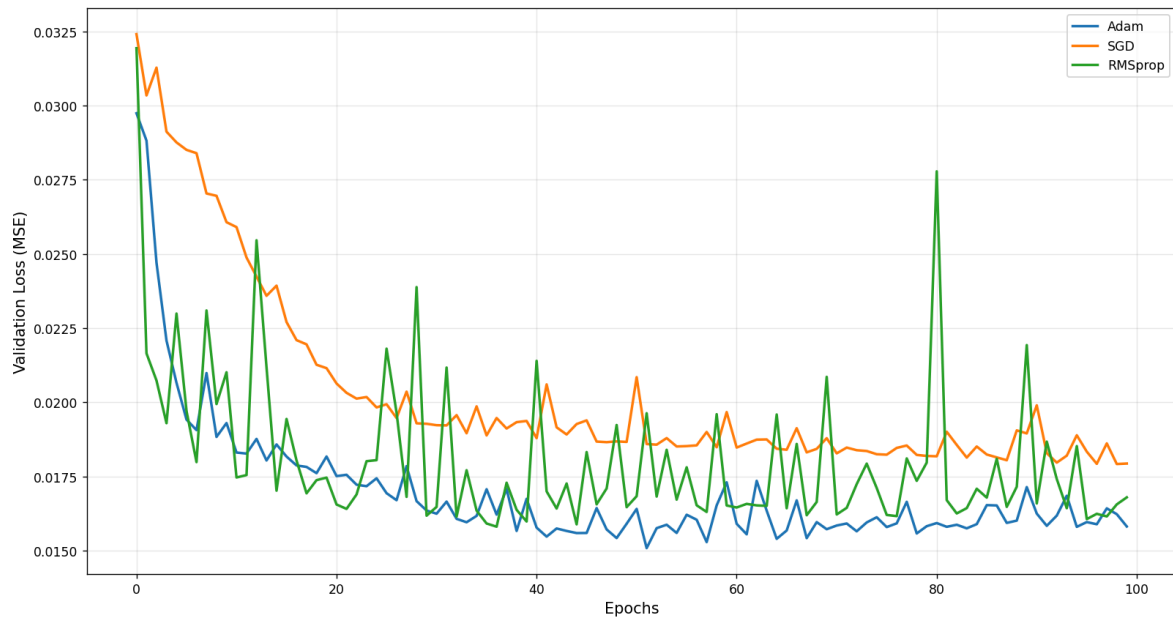


Figure 6.2 Comparaison des optimiseurs (perte de validation du MLP-SUL) – Corner El Harrach

À titre d'exemple, la **Figure 6.2** présente une comparaison de l'évolution de la perte de validation (MSE) au fil des epochs pour trois optimisateurs – Adam, SGD et RMSprop lors de l'entraînement du modèle MLP Sul sur la station Corner El Harrach. Sur l'axe horizontal figurent les epochs, représentant le nombre d'itérations complètes sur les données, tandis que l'axe vertical indique la valeur de la perte de validation. On observe que la courbe bleue, correspondant à Adam, descend rapidement et atteint une valeur de perte plus basse que les autres, tout en restant stable et peu sujette aux fluctuations. À l'inverse, la courbe orange de SGD montre une descente beaucoup plus lente, avec une perte qui demeure nettement supérieure sur l'ensemble de l'entraînement. La courbe verte de RMSprop, quant à elle, affiche une diminution rapide de la perte au début, mais se caractérise par d'importantes oscillations, surtout en fin d'entraînement, traduisant une certaine instabilité. Cet exemple illustre la tendance générale observée sur toutes les stations et architectures testées : Adam s'impose systématiquement comme l'optimiseur le plus efficace et le plus robuste, justifiant son choix pour l'entraînement de l'ensemble des modèles de l'étude.

6.2.3. Feature Engineering

Le feature engineering est essentiel pour optimiser les performances des modèles d'apprentissage automatique en convertissant les données brutes en attributs plus significatifs et prédictifs.

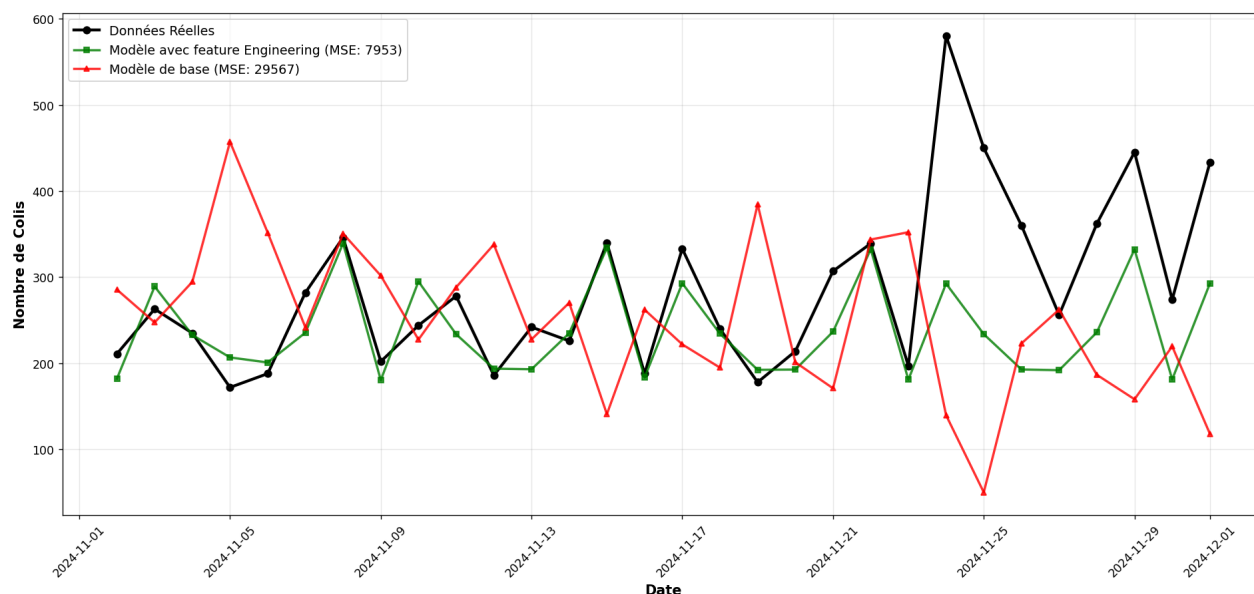


Figure 6.3 Comparaison des prédictions avec et sans feature engineering

La **Figure 6.3** présente une comparaison directe entre les prédictions d'un modèle de base et celles d'un modèle enrichi par feature engineering pour la prévision du nombre de colis sur une série temporelle. Sur ce graphique, les courbes illustrent respectivement les données réelles (*en noir*), les prédictions du modèle de base (*en rouge*) et celles du modèle intégrant le feature engineering (*en vert*). Le modèle de base, avec un MSE de 29 567, montre des fluctuations importantes et des écarts marqués par rapport à la réalité, notamment lors de pics où il surestime fortement la demande. À l'inverse, le modèle avec feature engineering, dont le MSE chute à 7 953, parvient à suivre de près les variations réelles, réagissant de façon plus précise aux changements soudains et limitant nettement les erreurs extrêmes. Cette amélioration technique majeure témoigne de l'impact du feature engineering : en construisant et sélectionnant des variables pertinentes, on dote le modèle d'une compréhension affinée des dynamiques sous-jacentes, ce qui se traduit par une réduction de l'erreur quadratique moyenne de 73 %. Ce résultat met en lumière la capacité du feature engineering à transformer des données brutes en informations exploitables, permettant au modèle de mieux intégrer la complexité et la réalité opérationnelle du terrain.

6.3. Tests sur les prédictions

La phase de prédiction permet de valider l'efficacité opérationnelle des modèles développés, en confrontant leurs résultats aux nouvelles données sur le court terme (10 jours) et long terme (30 jours). Cette analyse duale révèle à la fois la capacité des modèles à s'adapter aux fluctuations locales et leurs limites face à la complexité inhérente aux séries temporelles.

6.3.1. Court terme

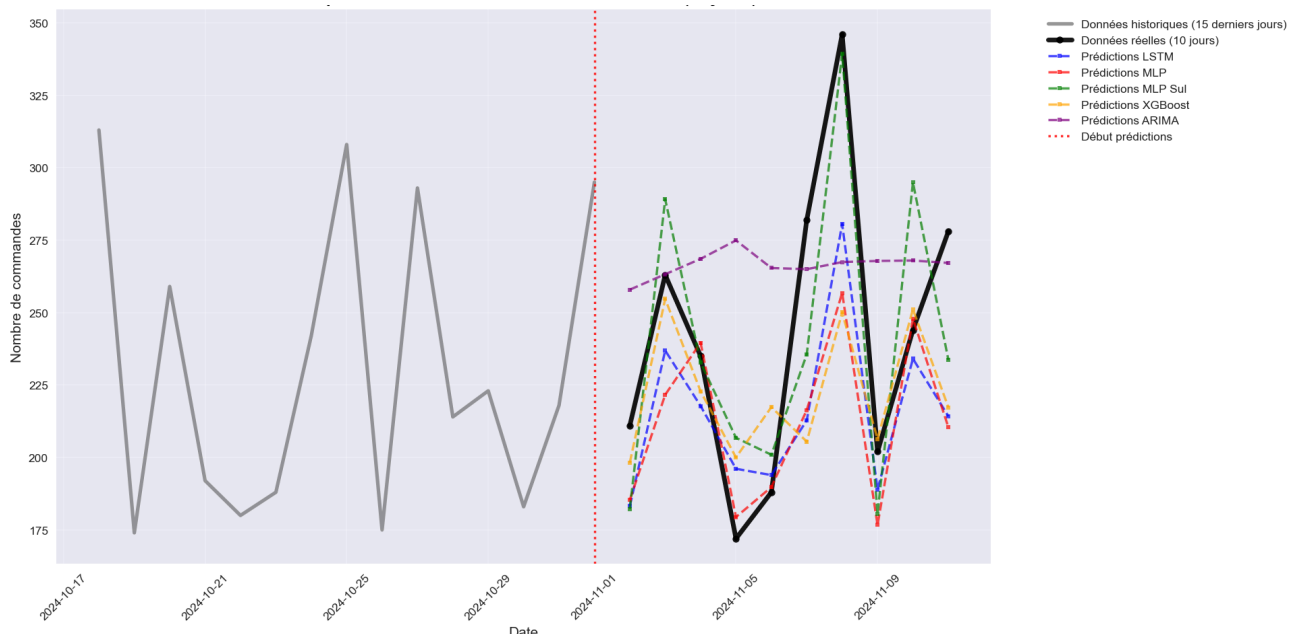


Figure 6.4 Comparaison des prédictions des modèles sur 10 jours – Station Corner El Harrach

La **Figure 6.4** présente les prédictions de cinq modèles (*LSTM*, *MLP*, *MLP Sul*, *XGBoost*, *ARIMA*) comparées aux données réelles sur la station Corner El Harrach du 1er au 10 novembre 2024. La courbe noire des données réelles montre une forte variabilité, oscillant entre 175 et 350 commandes, avec des transitions abruptes notamment entre les 5 et 7 novembre, période correspondant au weekend où l'agence de livraison ne travaille pas. Le modèle *MLP Sul* (courbe verte) suit de très près la trajectoire réelle, reproduisant fidèlement les pics à 350 commandes et les creux à 175 commandes avec une précision remarquable. *LSTM* (courbe bleue) et *MLP* (courbe rouge) maintiennent un suivi correct mais présentent des décalages lors des variations rapides, particulièrement visibles durant la période du 5-7 novembre. *XGBoost* (courbe jaune) tend à atténuer l'amplitude des variations, lissant les pics et créant un retard dans la capture des changements brusques. *ARIMA* (courbe violette) montre le comportement le plus défaillant avec un lissage excessif qui ignore complètement les variations critiques, restant proche d'une valeur moyenne constante.

6.3.2. Long terme

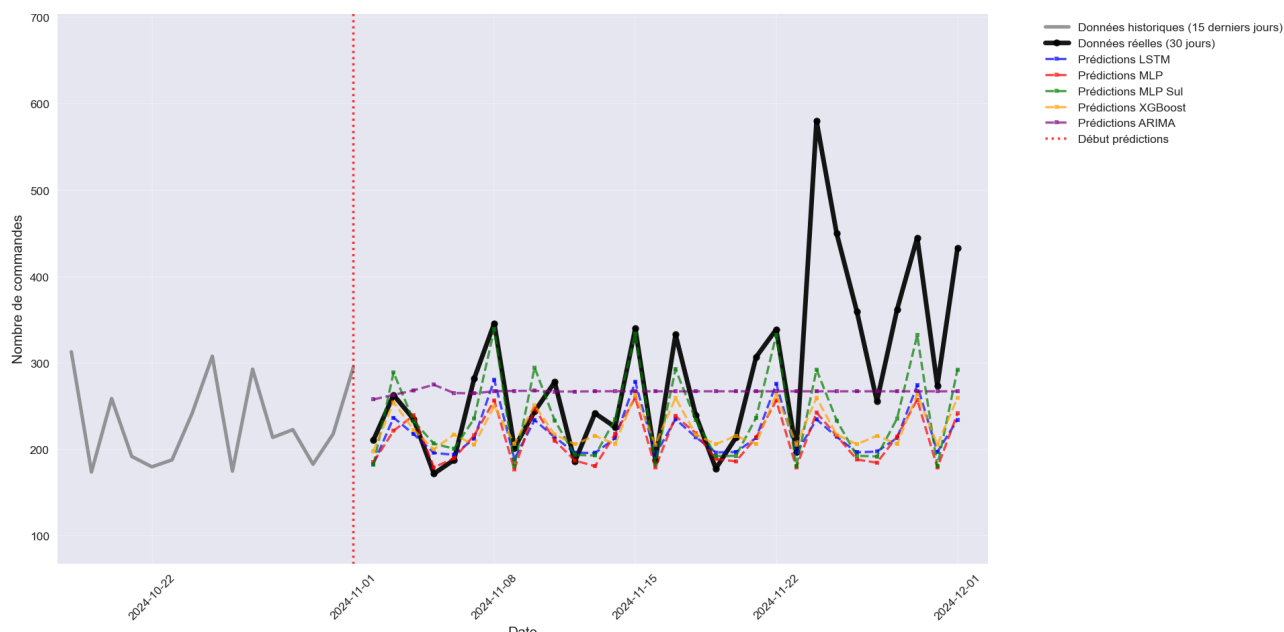


Figure 6.5 Prédictions des modèles sur 30 jours – Corner El Harrach

La **Figure 6.5** montre que, sur 30 jours, tous les modèles sous-estiment l'amplitude réelle des pics de commandes, notamment lors des hausses soudaines au-delà de 600. MLP Sul et LSTM suivent correctement la tendance générale et réagissent mieux aux variations, mais restent en dessous des valeurs extrêmes observées. XGBoost et MLP lissent davantage les fluctuations, tandis que ARIMA reste pratiquement plat et insensible aux pics. Notre solution permet donc de capter la dynamique globale de la demande, mais la prévision des extrêmes reste un défi sur le long terme.

6.3.3. Meilleur modèle vs Prophet par station

Dans le cadre de notre étude, nous avons systématiquement comparé, pour chaque station, les performances du meilleur modèle identifié lors de la phase de développement à celles du modèle Prophet, afin d'évaluer leur capacité respective à prédire la demande journalière de colis. Pour chaque station, les deux modèles ont été entraînés et testés sur les mêmes jeux de données, puis leurs résultats ont été confrontés à la réalité du terrain.

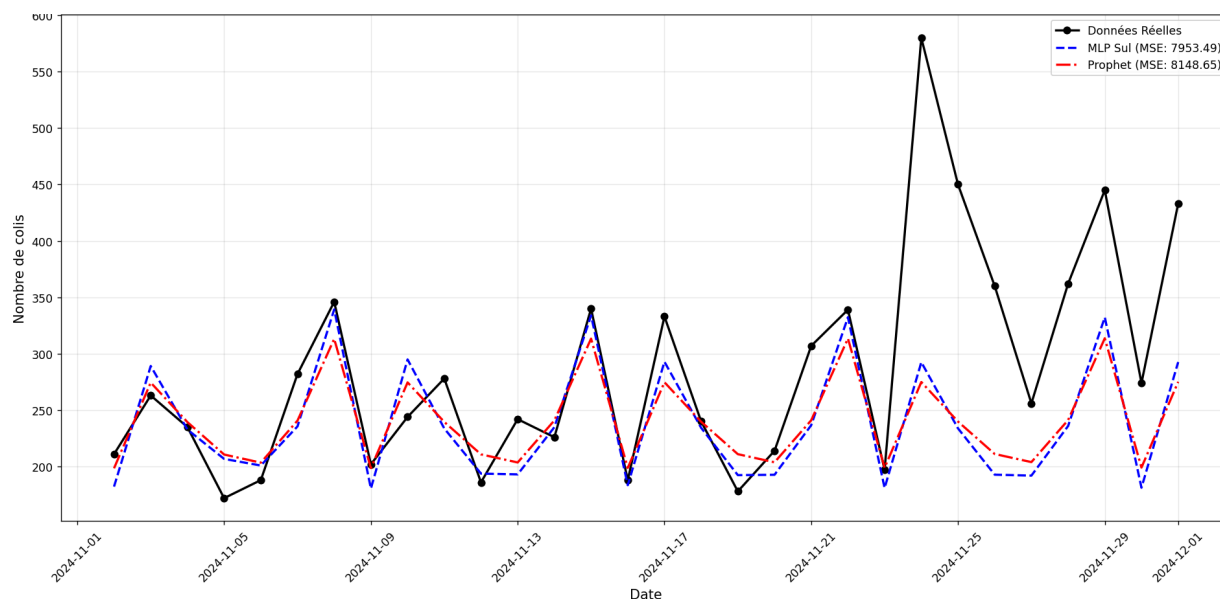


Figure 6.6 Prédictions MLP-SUL vs Prophet – Corner El Harrach

La **Figure 6.6** illustre cette comparaison pour la station Corner El Harrach, où le *MLP-SUL* s'est imposé comme le meilleur modèle. On constate que le *MLP-SUL* (courbe bleue pointillée) épouse plus fidèlement les variations réelles de la demande, notamment lors des pics, tandis que Prophet (courbe rouge pointillée) a tendance à lisser davantage les fluctuations et à sous-estimer certains extrêmes. Le *MSE* obtenu par le *MLP-SUL* est inférieur à celui de Prophet, confirmant ainsi la supériorité du *MLP-SUL* pour cette station.

Il convient toutefois de souligner que Prophet demeure un outil indispensable dans le domaine de la prévision des séries temporelles. Sa robustesse, sa flexibilité face aux données manquantes, son interprétabilité et sa capacité à modéliser naturellement les tendances, les saisonnalités et les événements en font une référence incontournable pour de nombreux cas d'usage, notamment en contexte métier ou pour des séries présentant des cycles marqués.

6.3.4. Performances entre développement et production

Avant de déployer un modèle en production, il est essentiel d'évaluer sa robustesse et sa capacité à généraliser, non seulement sur les données de test et validation (*développement*), mais aussi sur des nouvelles prédictions en conditions opérationnelles (*production*). Pour illustrer cette démarche comparative, nous avons sélectionné la station Corner El Harrach comme cas d'étude représentatif. Les deux figures ci-dessous présentent cette analyse comparative pour cette station spécifique :

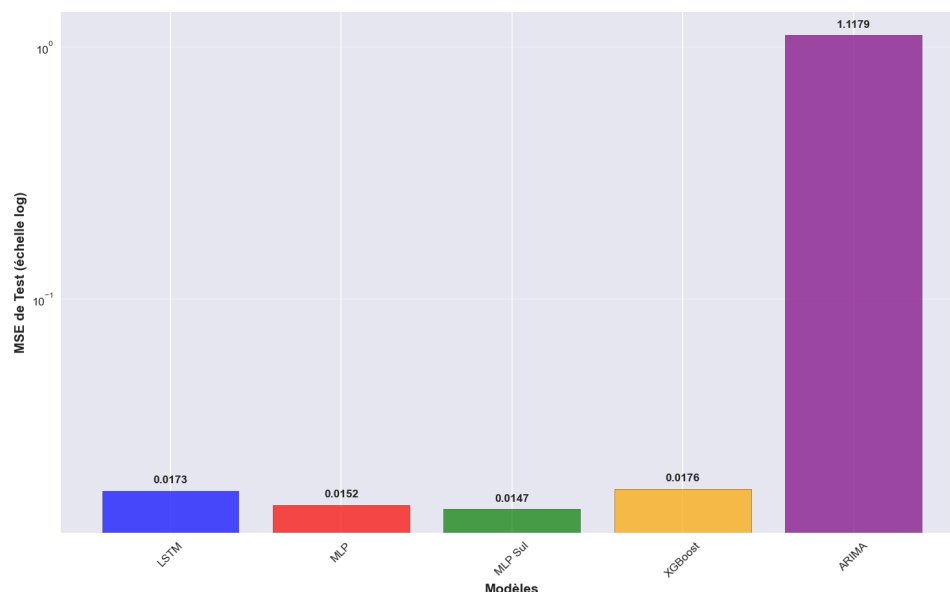


Figure 6.7 Performance réelle en développement – Corner El Harrach

La **Figure 6.7** montre que, lors de la phase de développement sur la station Corner El Harrach, les modèles *MLP Sul*, *MLP* et *LSTM* obtiennent les plus faibles erreurs quadratiques moyennes, *XGBoost* restant proche, tandis que *ARIMA* affiche une erreur nettement plus élevée. Cela indique que les modèles de réseaux de neurones sont plus efficaces pour apprendre les dynamiques des séries temporelles sur les données historiques de cette station.

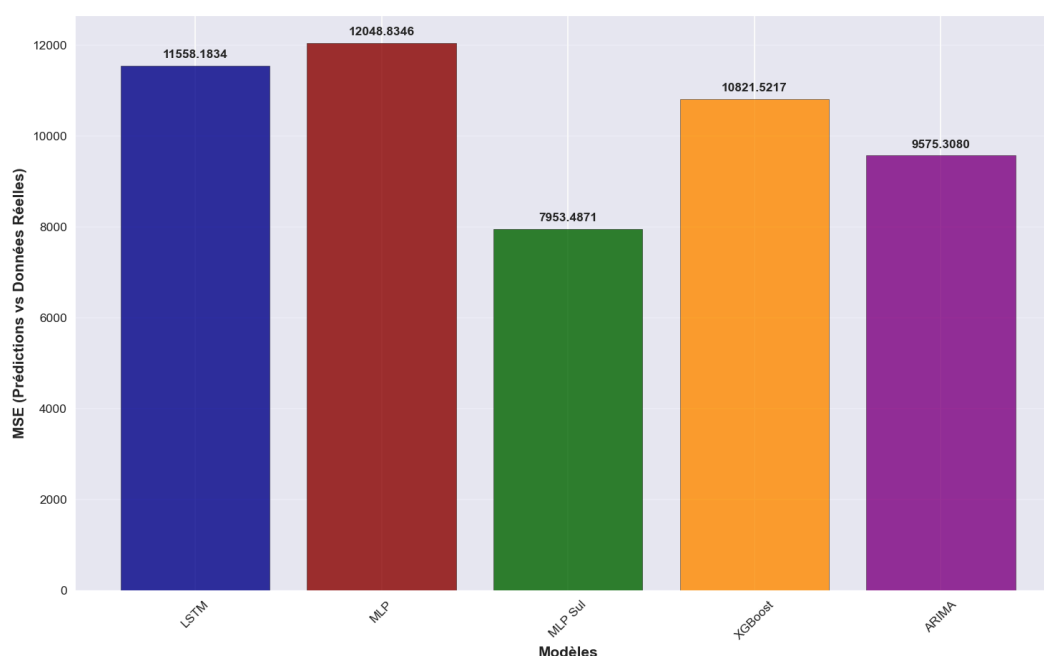


Figure 6.8 Performance réelle en production – Corner El Harrach

La **Figure 6.8** met en lumière la performance réelle en production de ces mêmes modèles sur la station Corner El Harrach, évaluée sur un horizon de 30 jours de prédictions

réelles. Ici, *MLP Sul* conserve son avantage, suivi par *ARIMA*, *XGBoost*, *LSTM* et *MLP*. On constate que l'écart entre les modèles se resserre et que la difficulté de prédire les valeurs futures réelles augmente pour tous, mais *MLP Sul* reste le plus robuste pour cette station.

6.3.5. Statistiques sur les modèles

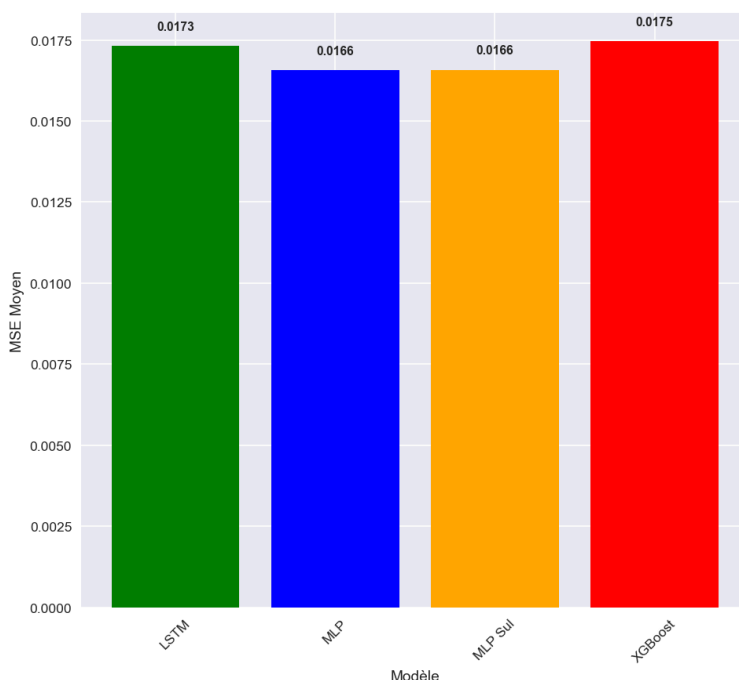


Figure 6.9 Comparaison du MSE des modèles – Corner El Harrach

La **Figure 6.9** présente la comparaison des erreurs quadratiques moyennes (MSE moyen) obtenues par chaque modèle sur l'ensemble des stations, à l'exclusion d'ARIMA. Quatre modèles ont été évalués : *LSTM*, *MLP*, *MLP Sul* et *XGBoost*. Le graphique met en évidence une convergence remarquable des performances entre ces modèles, avec des valeurs de *MSE* très proches. *ARIMA* n'a pas été inclus dans cette comparaison car sa moyenne de *MSE* est très élevée, bien supérieure à celle des autres modèles, ce qui rendrait sa présence sur le graphique non pertinente et écraserait la lisibilité des résultats des modèles performants.

L'écart minimal de 0,0009 entre le meilleur et le moins performant modèle indique que le problème de prévision de la demande de colis présente une complexité bien maîtrisée par l'ensemble des approches testées. Cette homogénéité suggère que :

- Les données sont bien préparées et les features pertinentes.
- Les modèles plus simples (*MLP*) sont aussi efficaces que les approches sophistiquées.

Cette similarité de performance libère le choix du modèle des contraintes de précision pure, permettant de privilégier des critères opérationnels comme la vitesse d'exécution, la facilité de maintenance ou les ressources computationnelles requises.

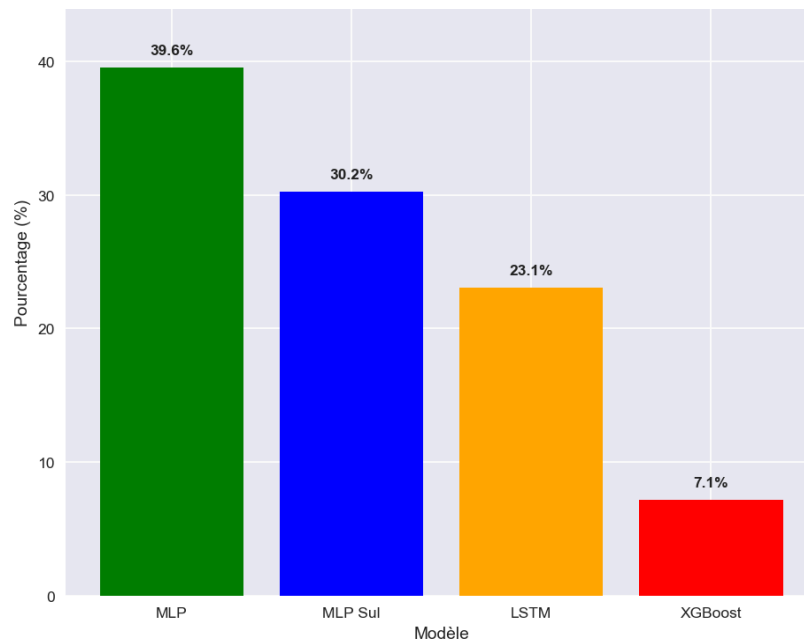


Figure 6.10 Répartition des modèles sélectionnés comme meilleurs prédicteurs

La **Figure 6.10** présente la distribution du modèle le plus performant, identifié station par station parmi les 182 stations analysées. Le graphique à barres indique que le MLP est désigné comme meilleur prédicteur dans 39,6 % des cas, suivi du MLP Sul avec 30,2 %, du LSTM avec 23,1 % et enfin de XGBoost avec 7,1 %. Cette répartition met en évidence la prédominance du MLP, qui s'impose comme solution optimale pour près de deux stations sur cinq, traduisant sa robustesse et sa capacité d'adaptation à des contextes locaux variés. Le MLP Sul et le LSTM confirment également leur compétitivité, tandis que XGBoost, bien qu'efficace dans certains scénarios, n'est sélectionné que dans une minorité de cas. Cette analyse comparative souligne l'importance d'une évaluation station par station pour identifier le modèle le plus adapté à chaque environnement opérationnel.

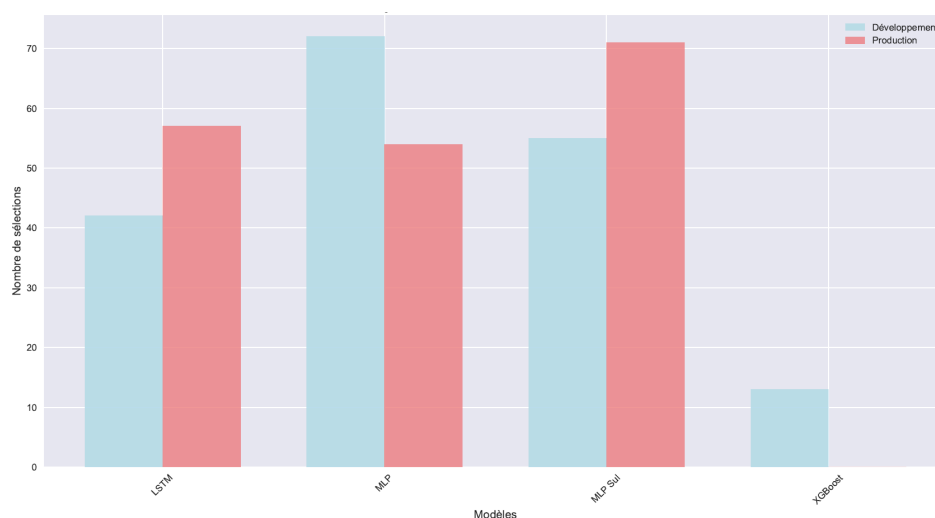


Figure 6.11 Sélections des modèles (développement et production)

La **Figure 6.11** compare le nombre de stations où chaque modèle a été sélectionné comme optimal en développement (*bleu clair*) versus production (*rose*). *LSTM* passe de 42 à 58 stations (+16), *MLP Sul* de 54 à 72 stations (+18), tandis que *MLP* chute de 72 à 54 stations (-18) et *XGBoost* disparaît complètement (13 à 0 stations).

Deux modèles gagnent en production (*LSTM* et *MLP Sul*) tandis que deux perdent (*MLP* et *XGBoost*). Le *MLP Sul* devient le modèle dominant avec 72 sélections, dépassant le *MLP* qui était leader en développement. *XGBoost* s'effondre totalement, révélant son inadaptation aux conditions réelles. Ces écarts démontrent que la performance en développement ne prédit pas la robustesse opérationnelle, justifiant une validation systématique sur données de production avant déploiement.

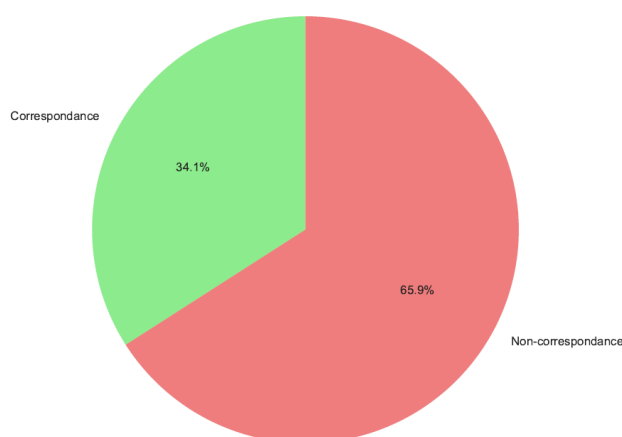


Figure 6.12 Taux de correspondance des meilleurs modèles (développement vs production)

La **Figure 6.12** montre que seulement 34,1 % des stations présentent une correspondance entre le modèle choisi en phase de développement et celui finalement retenu en production, contre 65,9 % de non-correspondance. Cette faible concordance ne se traduit pas par une perte de performance notable : l'analyse des erreurs (*MSE*) en production indique que les différences de performance entre les modèles restent généralement faibles. Ainsi, plusieurs modèles peuvent offrir des résultats similaires sur les données réelles, ce qui justifie une approche flexible et adaptative dans le choix du modèle en production.

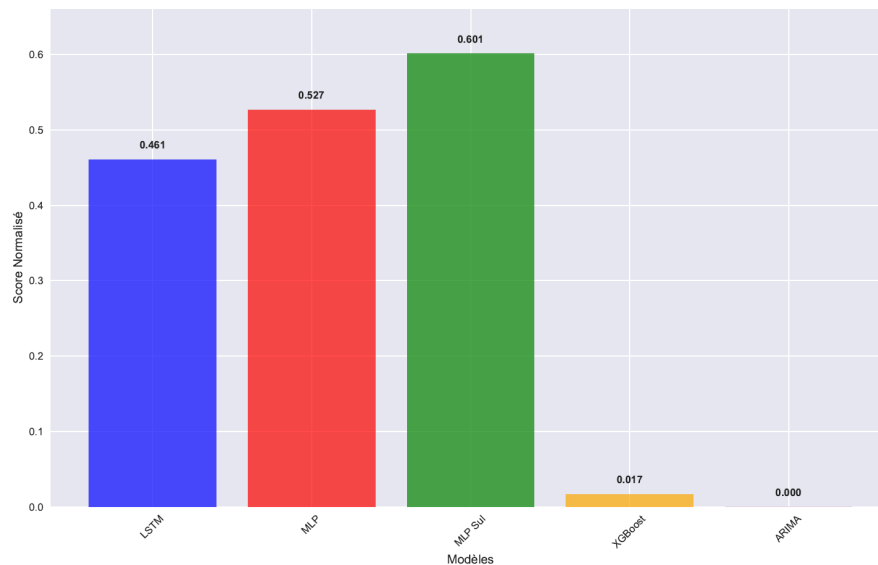


Figure 6.13 Scores pondérés MMR par modèle

La **Figure 6.13** présente les scores pondérés normalisés obtenus par chaque modèle selon la métrique *MRR*. Le *MLP Sul* atteint le score le plus élevé (0,601), suivi du *MLP* (0,527) et du *LSTM* (0,461), tandis que *XGBoost* (0,017) et *ARIMA* (0,000) affichent des performances très faibles. Cette évaluation met en évidence la supériorité des modèles neuronaux, en particulier du *MLP Sul*, selon ce critère global de performance.

6.4. Détection du Concept Drift

Les tests de détection du concept drift (*DDM*, *ADWIN*) montrent que la majorité des stations restent stables, avec quelques cas de drift ou warning détectés, principalement lors de changements brusques de comportement. Cela confirme la capacité du système à surveiller et anticiper les évolutions, garantissant ainsi la robustesse à long terme.

6.4.1. DDM

L'algorithme *DDM* a été appliqué sur l'ensemble des 182 stations en comparant les *MSE* de validation (calculés lors de l'entraînement) avec les *MSE* de production (calculés sur les données réelles après application des transformations de normalisation). La détection utilise des seuils adaptatifs basés sur le rapport *MSE* production/*MSE* validation : un ratio supérieur à 2.0 déclenche un statut "*DRIFT*", un ratio entre 1.5 et 2.0 génère un "*WARNING*", tandis qu'un ratio inférieur à 1.5 correspond à un état "*STABLE*". L'analyse révèle une distribution variée des états de dérive : la station **Corner El Harrach** illustre un cas typique de "*WARNING*" avec un *MSE* de validation de 0.0147 et un *MSE* de production de 0.0252 (ratio de 1.71), témoignant d'une dégradation modérée nécessitant une surveillance renforcée. D'autres stations comme Agence Ali Mendjeli présentent des cas de "*DRIFT*" avec des ratios dépassant 3.0 (*MSE* validation : 0.007, *MSE* production :

0.022), nécessitant une intervention, tandis que des stations comme Agence Bir Mourad Rais restent "STABLE" avec des performances cohérentes (*MSE validation* : 0.008, *MSE production* : 0.007). Ces résultats confirment la capacité de DDM à identifier les dégradations de performance et à classifier les niveaux de criticité selon des seuils statistiquement fondés, permettant une gestion proactive et graduée de la qualité prédictive du système.

Station	Validation MSE	Production MSE	Drift Status
Agence Ali Mendjeli	0.007	0.022	DRIFT
Corner El Harrach	0.0147	0.0252	WARNING
Agence Bir Mourad Rais	0.008	0.007	STABLE
Agence Bellevue	0.024	0.017	STABLE
Agence Bordj El Bahri	0.008	0.017	DRIFT
Agence Canastel	0.012	0.019	WARNING
Agence Chetouane	0.013	0.019	STABLE
Agence Cité Soualem	0.010	0.006	STABLE
Agence Fernand ville	0.018	0.026	STABLE

Tableau 6.2 Statut de dérive détecté par DDM sur un échantillon de stations

6.4.2. ADWIN

L'analyse ADWIN sur les 182 stations de Yalidine (5460 lignes sur 30 jours) n'a détecté aucun concept drift, contrairement à DDM. Cela s'explique par la fenêtre d'observation limitée et le paramètre delta conservateur fixé à 0.003, qui renforce la robustesse d'ADWIN face aux fausses alertes. ADWIN détecte les changements de distribution réelle, alors que DDM se base sur les erreurs de prédiction. Leur complémentarité permet une détection à la fois sensible (DDM) et robuste (ADWIN), assurant une surveillance équilibrée du concept drift.

6.5. Conclusion

Ce chapitre a détaillé l'évaluation exhaustive de notre système de prévision multi-modèles, confirmant sa robustesse et l'efficacité de son approche individualisée par station pour répondre aux besoins logistiques de Yalidine en Algérie.

Conclusion Generale

Ce projet a marqué une étape décisive pour Yalidine EL DJAZAIR SERVICES en développant les fondations d'un système interne de prévision des volumes de livraison, conçu pour surpasser les solutions externes en termes de précision, d'agilité et de pertinence métier. En adoptant la méthodologie rigoureuse CRISP-DM, nous avons transformé une problématique opérationnelle complexe en une feuille de route technique claire et validée, débutant par une compréhension approfondie des processus logistiques de Yalidine pour identifier les facteurs clés de la demande.

La contribution majeure de ce travail réside dans la conception et la validation d'une approche multi-modèles innovante. Il a été démontré qu'une solution unifiée n'est pas optimale ; la performance réside plutôt dans la capacité à sélectionner dynamiquement le meilleur algorithme (*MLP, LSTM, XGBoost, etc.*) pour chacune des 182 stations du réseau, en fonction de ses caractéristiques propres. Cette analyse a également mis en exergue le rôle critique de la dérive de concept dans un secteur logistique intrinsèquement volatile, établissant ainsi la nécessité impérative d'un système adaptatif pour garantir une fiabilité et une pertinence à long terme des prévisions.

Les modèles prédictifs développés ont permis d'anticiper avec précision les volumes de colis par station, conduisant à des bénéfices tangibles et stratégiques : une optimisation significative de l'allocation des ressources (*livreurs, flottes, espaces de tri*), une aide précieuse à la décision tant opérationnelle que stratégique, et une amélioration notable de la satisfaction client grâce à des livraisons plus rapides et fiables. La garantie de la confidentialité des données par un déploiement local de la solution a renforcé la souveraineté numérique de l'entreprise.

Malgré ces réalisations, le parcours n'a pas été sans défis. L'accès et la confidentialité des données, souvent complexes en Algérie, ont représenté une phase initiale longue. La compréhension approfondie du métier de la logistique a nécessité un investissement conséquent en temps et en apprentissage. De plus, les contraintes de ressources matérielles, notamment pour l'entraînement des modèles de deep learning, ont souligné la nécessité d'une infrastructure plus robuste pour une mise en production à grande échelle.

Ces défis ouvrent la voie à des perspectives d'amélioration et de développement futur ambitieuses. La prochaine étape logique est le déploiement et l'industrialisation de la solution via une infrastructure MLOps dédiée, intégrant l'automatisation complète du pipeline, de l'ingestion de données au réentraînement et à la surveillance continue. L'intégration d'une interface graphique web conviviale est essentielle pour démocratiser l'accès aux prévisions pour les utilisateurs non-techniques. Enfin, une amélioration

continue de la gestion du concept drift est envisagée, incluant l'interprétation des résultats de dérive (via *SHAP* ou *LIME* pour identifier les causes) et la capacité à identifier et à gérer les types de dérive (*soudain*, *graduel*, *récurrent*) pour déclencher des stratégies de correction automatiques et adaptées.

En définitive, ce travail ne se limite pas à une exploration théorique ; il livre un prototype fonctionnel et une architecture éprouvée, posant les bases d'une solution prête à être industrialisée. Il offre à Yalidine non seulement un outil de prévision, mais aussi un levier de compétitivité durable, capable de s'adapter de manière proactive aux défis de la logistique moderne et de transformer les données historiques en un avantage stratégique pour l'avenir.

Bibliographie

1. Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(1), 281–305.
2. Munkhdalai, L., Park, K. H., Batbaatar, E., & Ryu, K. H. (2020). Deep learning-based demand forecasting for Korean postal delivery service. *IEEE Access*, 8, 188135–188145.
3. Stadler, H., Kilger, C., & Meyr, H. (Éds.) (2015). *Supply Chain Management and Advanced Planning: Concepts, Models, Software, and Case Studies* (5^e éd.). Springer. <https://doi.org/10.1007/978-3-642-55309-7>
4. Rowe, G., Wright, G., & Bolger, F. M. I. (1991). Delphi: A re-evaluation of research and theory. *Technological Forecasting and Social Change*, 39(3), 235–251. [https://doi.org/10.1016/0040-1625\(91\)90039-I](https://doi.org/10.1016/0040-1625(91)90039-I)
5. Green, K. C., & Armstrong, J. S. (2007). Structured analogies for forecasting. *International Journal of Forecasting*, 23(3), 365–376. <https://doi.org/10.1016/j.ijforecast.2007.05.005>
6. Brockwell, P. J., & Davis, R. A. (2016). *Introduction to Time Series and Forecasting* (3^e éd.). Springer. <https://doi.org/10.1007/978-3-319-29854-2>
7. Box, G. E. P., & Jenkins, G. M. (1970). *Time series analysis: Forecasting and control*. Holden-Day.
8. Kim, K.-j. (2003). Financial time series forecasting using support vector machines. *Neurocomputing*, 55(1), 307–319. [Financial time series forecasting using support vector machines - ScienceDirect](#)
9. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system (arXiv:1603.02754v3) [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.1603.02754>
10. Gifty, A., & Li, Y. (2024). A comparative analysis of LSTM, ARIMA, XGBoost algorithms in predicting stock price direction. *Engineering and Technology Journal*, 9(8), 4978–4986. <https://doi.org/10.47191/etj/v9i08.50>
11. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
12. Touzet, C. (1992). *Les réseaux de neurones artificiels : Introduction au connexionnisme : cours, exercices et travaux pratiques* (préface de J. Hérault). Éditions EC2.
13. Chen, S.-A., Li, C.-L., Yoder, N. C., Arik, S. Ö., & Pfister, T. (2023). TSMixer: An all-MLP architecture for time series forecasting (arXiv:2303.06053v5) [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2303.06053>
14. Olah, C. (2015, août 27). Understanding LSTM Networks. colah's blog. <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
15. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
16. Uber Engineering Team. (2017). Deep and confident prediction for time series at Uber. Uber Technologies Inc. <https://eng.uber.com/tsw2017/>

17. Taylor, S. J., & Letham, B. (2017, 13 janvier). Forecasting at scale [Manuscrit de travail, Facebook Core Data Science]. [Forecasting at Scale](#)
18. Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1), 3–73. <https://www.proquest.com/docview/1266805989>
19. Feng, T., Zheng, Z., Xu, J., Liu, M., Li, M., Jia, H., & Yu, X. (2022). The comparative analysis of SARIMA, Facebook Prophet, and LSTM for road traffic injury prediction in Northeast China. *Frontiers in Public Health*, 10, Article 946563. <https://doi.org/10.3389/fpubh.2022.946563>
20. Du, L., Yang, A., Zayats, A. V., & Yuan, X. (2019). Deep-subwavelength features of photonic skyrmions in a confined electromagnetic field with orbital angular momentum. *Nature Physics*, 15, 650–654. <https://doi.org/10.1038/s41567-019-0487-7>
21. Kingma, D. P., & Ba, J. (2017, 30 janvier). Adam: A method for stochastic optimization (Version 9) [Préprint, International Conference on Learning Representations]. arXiv. <https://doi.org/10.48550/arXiv.1412.6980>
22. Wang, B., Shen, Y., Yan, X., & Kong, X. (2024). An autoregressive integrated moving average and long short-term memory (ARIM-LSTM) hybrid model for multi-source epidemic data prediction. *PeerJ Computer Science*, 10, Article e2046. <https://doi.org/10.7717/peerj-cs.2046>
23. Shcherbakov, M. V., Brebels, A., Shcherbakova, N. L., Tyukov, A. P., Janovsky, T. A., & Kamaev, V. A. (2013). A survey of forecast error measures. *World Applied Sciences Journal*, 24(Information Technologies in Modern Industry, Education & Society), 171–176. <https://doi.org/10.5829/idosi.wasj.2013.24.itmies.80032>
24. Botchkarev, A. (2018). Performance metrics (error measures) in machine learning regression, forecasting and prognostics: Properties and typology [Manuscrit]. Recherche non publiée. <https://arxiv.org/pdf/1809.03006>
25. Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3), 1247–1250. <https://doi.org/10.5194/gmd-7-1247-2014>
26. Voorhees, E. M., & Tice, D. M. (1999). The TREC-8 question answering track report. In *Proceedings of the 8th Text Retrieval Conference (TREC-8)* (pp. 77–82), Gaithersburg, MD. National Institute of Standards and Technology
27. Tay, F. E. H., & Cao, L. J. (2001). Application of support vector machines in financial time series forecasting. *Omega*, 29(4), 309–317. [https://doi.org/10.1016/S0305-0483\(01\)00026-3](https://doi.org/10.1016/S0305-0483(01)00026-3)
28. Schlimmer, J. C., & Granger, R. H., Jr. (1986). Incremental learning from noisy data. *Machine Learning*, 1(3), 317–354. <https://doi.org/10.1007/BF00116895>
29. Ramírez-Gallego, S., Krawczyk, B., García, S., Woźniak, M., & Herrera, F. (2017). A survey on data preprocessing for data stream mining: Current status and future directions. *Neurocomputing*, 239, 39–57. <https://doi.org/10.1016/j.neucom.2017.01.078>

30. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2020). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
<https://doi.org/10.1109/TKDE.2019.2946069>
31. Lemaire, V., Salperwyck, C., & Bondu, A. (2014). A survey on supervised classification on data streams. In E. Zimányi & R.-D. Kutsche (Éds.), *Business Intelligence: 4th European Summer School, eBISS 2014, Berlin, Germany, July 6–11, 2014, Tutorial Lectures (Lecture Notes in Business Information Processing, vol. 205, pages 88–125)*. Springer.
https://doi.org/10.1007/978-3-319-17551-5_4
32. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 44:1–44:37.
<https://doi.org/10.1145/2523813>
33. Gama, J., & Castillo, G. (2006). Learning with local drift detection. Dans *Advanced Data Mining and Applications (p. 42–55)*. SBIA/ADMA 2006. LIACC, University of Porto.
34. Baena-Garcia, M., del Campo-Avila, J., Fidalgo, R., Bifet, A., Gavalda, R., & Morales-Bueno, R. (2006). Early Drift Detection Method. Dans *Proc. of the 4th International Workshop on Knowledge Discovery from Data Streams, Montréal, Canada*.
35. Frías-Blanco, I., del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., & Caballero-Mota, Y. (2015). Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3), 810–823.
<https://doi.org/10.1109/TKDE.2014.2345382>
36. Yu, H., Liu, T., Lu, J., & Zhang, G. (2021, 4 mai). Automatic learning to detect concept drift (arXiv:2105.01419v1) [Préprint]. arXiv. <https://doi.org/10.48550/arXiv.2105.01419>
37. Bifet, A., & Gavalda, R. (2007). Learning from time-changing data with adaptive windowing. Dans *Proceedings of the 7th SIAM International Conference on Data Mining (SDM 2007)* (pp. 443–448). SIAM. <https://doi.org/10.1137/1.9781611972771.42>
38. Komorniczak, J., & Ksieniewicz, P. (2024, 11 avril). Unsupervised concept drift detection based on parallel activations of neural network (arXiv:2404.07776v1) [Préprint]. arXiv. <https://doi.org/10.48550/arXiv.2404.07776>
39. Elwell, R., & Polikar, R. (2011). Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10), 1517–1531.
<https://doi.org/10.1109/TNN.2011.2160459>
40. Polikar, R., Upda, L., Upda, S. S., & Honavar, V. (2008). Incremental learning in non-stationary environments with concept drift. Dans *Proceedings of the 19th International Conference on Pattern Recognition (ICPR 2008)* (pp. 43–47). IEEE.
41. Karnick, M., Muhlbaier, M. D., & Polikar, R. (2008). Incremental learning in non-stationary environments with concept drift using a multiple classifier based approach. *Proceedings of the 19th IEEE International Conference on Tools with Artificial Intelligence*, 42–49. IEEE.
<https://doi.org/10.1109/ICTAI.2007.4418480>

42. Domingos, P., & Hulten, G. (2000, 1^{er} août). Mining high-speed data streams. Dans *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 71–80). Association for Computing Machinery.
<https://doi.org/10.1145/347090.347107>
43. Autorité de Régulation de la Poste et des Communications Électroniques – ARPCE. (2018, 10 juin). Loi n° 18-07 du 25 Ramadhan 1439 relative à la protection des personnes physiques dans le traitement des données à caractère personnel. ARPCE.
<http://www.arpce.dz/fr/pub/c7e6n6>
44. Laqrichi, S. (2015). Approach to build realistic models for estimating project effort/cost in an uncertain environment: Application to the software development field [Doctoral dissertation, Université Paris-Dauphine]. ResearchGate.
<https://www.researchgate.net/publication/303447451>

Annexes

7.1. Gestion de projet

Afin d’assurer le bon déroulement de notre travail et respecter les délais des différentes activités, tout en assurant une bonne qualité des livrables, l’adoption d’une stratégie de gestion de projet bien déterminée est indispensable.

7.1.1. Planning

Le projet s’est déroulé selon plusieurs phases depuis son commencement en Octobre 2024. La **Figure 7.1** suivante représente le planning prévisionnel élaboré au début du projet avec ses différentes phases.

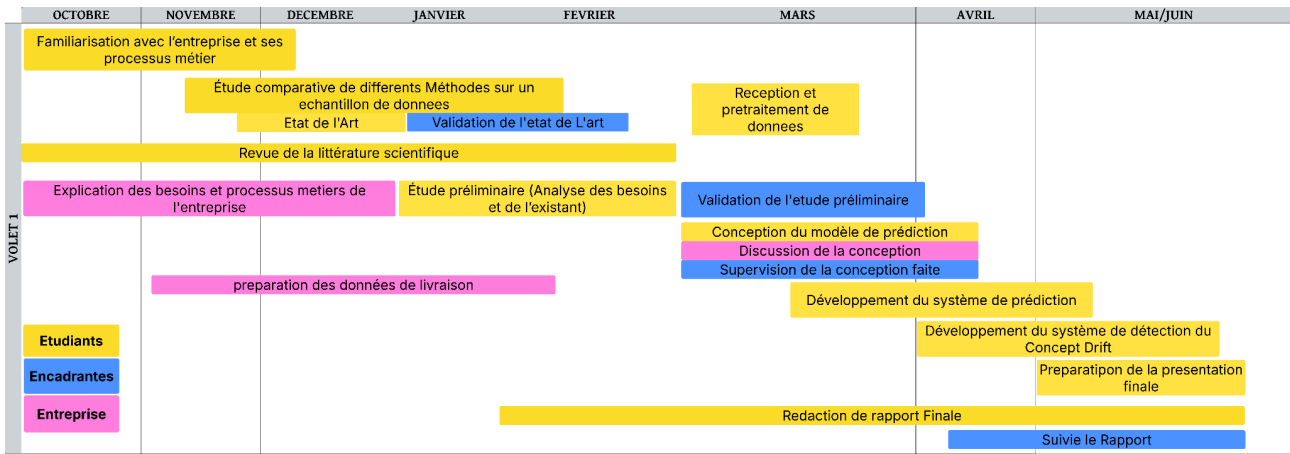


Figure 7.1 Planning prévisionnel du projet.

Le planning prévisionnel a connu diverses déviations en raison d’un ensemble de contraintes liées aux besoins, disponibilité des données et compréhension du contexte. Cependant, nous avons pu travailler en mode agile et livrer la solution proposée

7.1.2. Contraintes liées au projet

Au fur et à mesure de l’avancement de projet, nous avons rencontré un ensemble de contraintes affectant le déroulement du projet et la qualité des résultats :

- Manque d’un cahier des charges au début de projet.
- Accessibilité retardée aux données (Jusqu’au début du mois de Mars).
- Manque de ressources matérielles nécessaire à la mise en place des modèles.